

Московский государственный институт электронной техники
(технический университет)

Кафедра высшей математики - 1

Е.А. Коплович, Д.М. Коплович, С.В. Умняшкин

Теоретические основы цифровой обработки и представления сигналов: практикум

Москва, 2007.

Учебное пособие представляет собой методическое обеспечение для практических, лабораторных и самостоятельных работ по курсам «Математические основы цифровой обработки сигналов» и «Теоретические основы методов сжатия сигналов и данных». Пособие содержит краткое изложение теоретического материала для каждого аудиторного занятия, наборы задач для практических занятий, задания для лабораторных работ со списком контрольных вопросов.

Содержание

Часть 1. Практические занятия	4
Семинар №1. Линейные нормированные пространства. Ортогональные системы в гильбертовом пространстве	4
Семинар №2. Ряды Фурье. Частотное представление сигнала.....	15
Семинар №3. Быстрое дискретное преобразование Фурье	27
Семинар №4. Z-преобразование	37
Семинар №5. Линейные дискретные фильтры	42
Семинар №6. Источник дискретных сообщений без памяти. Энтропия. Эффективное кодирование	51
Семинар №7. Арифметическое кодирование. Кодирование длин серий.....	60
Семинар №8. Условная энтропия. Декоррелирующие преобразования. Дискретное косинусное преобразование	70
Часть 2. Лабораторный практикум.....	77
Лабораторная работа №1. Дискретизация и квантование сигналов.....	77
Лабораторная работа №2. Быстрое преобразование Фурье. Дискретная свертка	83
Лабораторная работа №3. Линейные дискретные фильтры. Синтез КИХ-фильтров по заданной частотной характеристике.....	97
Лабораторная работа №4. Словарные методы сжатия данных. Алгоритм LZW.....	106
Лабораторная работа №5. Арифметическое кодирование.....	114
Лабораторная работа №6. Сжатие изображений с использованием дискретного косинусного преобразования	124
Лабораторная работа №7. Знакомство с дискретным вейвлет-анализом	132

Часть 1. Практические занятия

Семинар №1. Линейные нормированные пространства. Ортогональные системы в гильбертовом пространстве

Нормированные пространства

Нормированные пространства

Линейное пространство E – **нормированное**, если $\forall x \in E$ поставлено в соответствие вещественное число $\|x\|$, называемое **нормой**, для которого выполняются аксиомы:

1. Невырожденность нормы: $\forall x \in E: \|x\| \geq 0, x = 0 \Leftrightarrow \|x\| = 0$.
2. Однородность нормы: $\forall \lambda \in \mathbb{R}, \forall x \in E: \|\lambda \cdot x\| = |\lambda| \cdot \|x\|$.
3. Неравенство треугольника: $\forall x, y \in E: \|x + y\| \leq \|x\| + \|y\|$.

Неравенство Минковского для интегралов:

пусть для $p \geq 1$: $\exists \int_a^b u(x)^p dx$ и $\exists \int_a^b v(x)^p dx$,

тогда выполняется неравенство

$$\left(\int_a^b (u(x) + v(x))^p dx \right)^{1/p} \leq \left(\int_a^b u(x)^p dx \right)^{1/p} + \left(\int_a^b v(x)^p dx \right)^{1/p}.$$

Пример нормированного пространства: $L_2[a, b]$ – пространство кусочно-непрерывных на отрезке $[a, b]$ вещественных функций с нормой $\|x\| = \sqrt{\langle x, x \rangle}$, где $\langle x, x \rangle = \int_a^b x^2(t) dt$.

Задание. $C[a, b]$ – множество непрерывных функций на отрезке $[a, b]$ с нормами $\|x\| = \max_{t \in [a, b]} |x(t)|$, $\|x\| = \left(\int_a^b |x(t)|^p dt \right)^{1/p}$, $p \geq 1$. Для $\|x\| = \left(\int_a^b |x(t)|^p dt \right)^{1/p}$ проверить выполнение аксиом нормы.

◀ 1) невырожденность нормы: $\forall x(t) \in C[a, b]: \|x(t)\| \geq 0$, при $x(t) \equiv 0 = q(t)$ $\|x(t)\| = 0$. Пусть $\exists q'(t) \neq q(t): \|q'(t)\| = 0$, тогда существует отрезок $[a'; b'] \subset [a, b]$, такой, что при $t \in [a'; b']$ $q'(t) \neq 0$, а $|q'(t)| > 0$. Следовательно,

$$\|q'(t)\| = \left(\int_{a'}^{b'} |q'(t)|^p dt \right)^{1/p} > 0 \text{ – получаем противоречие аксиоме 1;}$$

2) однородность нормы: очевидна в силу свойства линейности интеграла;

3) неравенство треугольника: используем неравенство Минковского

$$\|x(t) + y(t)\| = \left(\int_a^b |x(t) + y(t)|^p dt \right)^{1/p} \leq \left(\int_a^b |x(t)|^p dt \right)^{1/p} + \left(\int_a^b |y(t)|^p dt \right)^{1/p} = \|x(t)\| + \|y(t)\|. \blacktriangleright$$

Задание. Для пространства арифметических векторов $l^n : x = (x_1, \dots, x_n)^T$ с нормой $\|x\| = \sum_i |x_i|$ проверить выполнение аксиом нормы.

◀ 1) невырожденность нормы: очевидно;

2) однородность нормы: $\|l \cdot x\| = \sum_{i=1}^n |l \cdot x_i| = |l| \cdot \sum_{i=1}^n |x_i| = |l| \cdot \|x\|;$

3) неравенство треугольника:

$$\|x + y\| = \sum_{i=1}^n |x_i + y_i| \leq \sum_{i=1}^n (|x_i| + |y_i|) = \sum_{i=1}^n |x_i| + \sum_{i=1}^n |y_i| = \|x\| + \|y\|. \blacktriangleright$$

Полнота линейных нормированных пространств

Полнота пространства

Расстояние между элементами x, y линейного нормированного пространства E (ЛНП): $\rho(x, y) = \|x - y\|$ (**метрика** пространства).

Элемент $y \in E$ – **предел последовательности** $\{x_k\} \in E$, если $\lim_{k \rightarrow \infty} \rho(y, x_k) = 0 : \lim_{k \rightarrow \infty} x_k = y$.

Пусть X – линейное нормированное пространство (ЛНП). Последовательность $\{x_n\} \subset X$ называется **фундаментальной**, если $\forall \varepsilon > 0 \exists N = N(\varepsilon) : \forall n > N, \forall p \in \mathbb{N} \|x_{n+p} - x_n\| < \varepsilon$.

ЛНП называется **полным**, если в нем сходится любая фундаментальная последовательность. Полное ЛНП называется **банаховым**.

Задание. Доказать, что если $\{x_n\}, \{y_n\}$ – фундаментальные последовательности, то последовательность $z_n = \{x_n + y_n\}$ также фундаментальная.

◀ Необходимо показать, что $\forall \varepsilon > 0 \exists N = N(\varepsilon) : \forall n > N, \forall p \in \mathbb{N} \|z_{n+p} - z_n\| < \varepsilon$. Рассмотрим $\|z_{n+p} - z_n\| = \|(x_{n+p} + y_{n+p}) - (x_n + y_n)\| = \|(x_{n+p} - x_n) + (y_{n+p} - y_n)\| \leq \|x_{n+p} - x_n\| + \|y_{n+p} - y_n\|$. По условию, $\forall \frac{\varepsilon}{2} > 0 \exists N_x = N_x\left(\frac{\varepsilon}{2}\right)$ и

$\exists N_y = N_y\left(\frac{e}{2}\right)$ такие, что $\forall n > N = \max(N_x, N_y), \quad \forall p \in \mathbb{N}: \quad \|x_{n+p} - x_p\| < \frac{e}{2},$
 $\|y_{n+p} - y_p\| < \frac{e}{2}$ и, следовательно, $\|z_{n+p} - z_p\| < e. \blacktriangleright$

Евклидово и гильбертово пространства

Евклидово пространство

Линейное пространство E – **евклидово**, если $\forall x, y \in E$ поставлено в соответствие вещественное число $\langle x, y \rangle$, называемое **скалярным произведением**, для которого выполняются аксиомы:

1. $\forall x \in E: \langle x, x \rangle \geq 0, \langle x, x \rangle = 0 \Leftrightarrow x = \theta.$
2. $\forall x, y \in E: \langle x, y \rangle = \langle y, x \rangle.$
3. $\forall x, y \in E, \forall \lambda \in \mathbb{R}: \langle \lambda x, y \rangle = \lambda \langle x, y \rangle.$
4. $\forall x, y, z \in E: \langle x + y, z \rangle = \langle x, z \rangle + \langle y, z \rangle.$

Ортогональные элементы – $x, y \in E: \langle x, y \rangle = 0$

Ортогональная система в E – множество попарно ортогональных элементов $\{x_n\} \subset E.$

Пространство Гильберта H – евклидово пространство, полное в норме $\|x\| = \sqrt{\langle x, x \rangle}.$

Ортогональные системы в гильбертовом пространстве. Тригонометрическая система

Ортогональные системы

Ортогональная система $\{\varphi_k\}_{k=1}^{\infty} \subset H$ **полная** в гильбертовом пространстве H , если $\forall x$ из H существует разложение в **ряд Фурье**:

$$x = \sum_{k=1}^{\infty} \lambda_k \varphi_k,$$

где $\lambda_k = \frac{\langle x, \varphi_k \rangle}{\langle \varphi_k, \varphi_k \rangle}$ – **коэффициенты Фурье**.

Ортогональная система $\{\varphi_k\}_{k=1}^{\infty} \subset H$ **полная** $\Leftrightarrow \forall x \in H$ выполняется **равенство Парсеваля-Стеклова**:

$$\|x\|^2 = \sum_{j=1}^{\infty} \lambda_j^2 \|\varphi_j\|^2.$$

Примеры ортогональных систем в L_2 :

1. Тригонометрическая система функций

$\{1, \cos(2\pi kt/T), \sin(2\pi kt/T)\}_{k=1}^{\infty}$ – **полная** в $L_2[a, a+T]$.

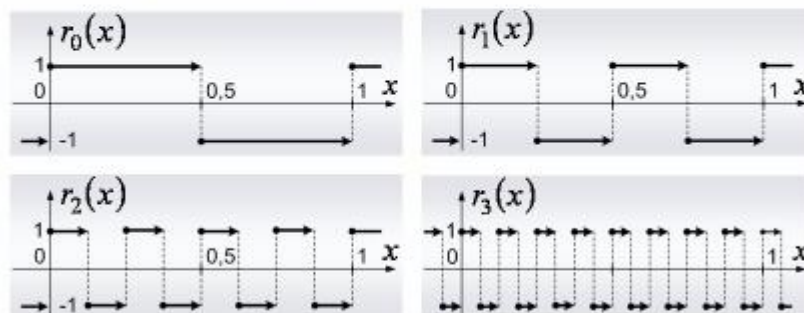
Ортогональные системы в гильбертовом пространстве: система Радемахера

Система функций Радемахера

2. Система функций Радемахера $\{r_k(x)\}_{k=0}^{\infty}$:

$r_0(x) = \begin{cases} 1, & x \in [0, 1/2) \\ -1, & x \in [1/2, 1) \end{cases}$ периодически продолженная на всю числовую ось.

$r_k(x) = r_0(2^k x), k = 1, 2, \dots$



Система не является полной в $L_2[0;1]$!

Задание. Разложить функцию $f(x) = r_0(x)r_1(x)$, убедиться, что не выполняется равенство Парсеваля.

$$\blacktriangleleft f(x) = r_0(x)r_1(x) = \begin{cases} 1, & x \in [0; 1/4) \cup [3/4; 1) \\ -1, & x \in [1/4; 3/4) \end{cases}, \|f(x)\| > 0. \text{ Имеем } \forall n \ r_n(x) \perp f(x),$$

следовательно, все коэффициенты $l_n = \frac{\langle f(x), r_n \rangle}{\langle r_n, r_n \rangle} = 0$ и $\sum l_n^2 \cdot \|r_n(x)\|^2 = 0 \neq \|f(x)\|^2$ –

система неполная. $\blacktriangleright$

Ортogonalные системы в гильбертовом пространстве: система Уолша



Задание. Используя функцию rademacher,

```
function res = rademacher(x, k)
% k-я функция Радемахера
res = sign(sin(2 * pi * 2^k * x));
res(find(res == 0)) = 1;
```

дописать функцию walsh, генерирующую k -ю функцию Уолша для заданных значений аргумента x , используя функцию dec2bin для нахождения двоичного разложения номера функции:

```
function res = walsh(x, k)
% k-я функция Уолша
res = ones(1, length(x));
bin = dec2bin(k);
for i = 1:length(bin)
    if bin(i) == '1'
        % вычислите k-ю функцию Уолша
    end
end
```


Построить несколько базисных функций Уолша, используя файл task_1_1.m:

```
function task_1_1
clear;
close all;
basis = 'walsh'; % базис (walsh, rademacher или haar)
total_funs = 12; % максимальный номер функции
T0 = 0; % начало периода
T1 = 1; % конец периода
points = 300; % число точек для вычисления функции
delay = 0.15; % задержка показа очередной функции, сек.
T = T1 - T0;
plot_time = linspace(T0, T1, points);
h = figure;
set(h, 'DoubleBuffer', 'on');
for i=0:total_funs
    fvalues = feval(basis, plot_time, i);
    plot(plot_time, fvalues, 'r', 'LineWidth', 2.5); grid;
    axis([T0, T1, 1.5 * [min(fvalues)-0.1,max(fvalues)+0.1]]);
    title(sprintf('%s-%d of %d', basis, i, total_funs));
    pause(delay);
end
```

Ортогональные системы в гильбертовом пространстве: система Хаара



Задание. Найти коэффициенты разложения функции $f(x) = \begin{cases} 2, & x \in [1/4; 3/4] \\ 0, & x \notin [1/4; 3/4] \end{cases}$ в ряд Фурье по системе Хаара, используя функцию `fseries`. Проверить выполнение равенства Парсеваля $\sum l_n^2 \cdot \|r_n(x)\|^2 = \|f(x)\|^2$, где $\|f(x)\| = \left(\int_{-\infty}^{\infty} f(x)^2 dx \right)^{1/2}$.

```
function c = fseries(target_fun, T0, T1, K, basis)
% Нахождение коэффициентов Фурье.
% Использование:
%   c = fseries(target_fun, T0, T1, K, basis)
%   target_fun - исходная функция
%   basis - 'fourier' (по умолчанию), 'walsh', 'rademacher'
%   T0, T1 - начало и конец периода функции
%   K - вектор номеров искоемых коэффициентов
%   c - вектор коэффициентов Фурье
%
% Пример:
% >> c = fseries('sin', 0, 1, [0:3], 'walsh')
% c =
%      0.4597    -0.2149    -0.1057    -0.0148
if (nargin < 5)
    basis = 'fourier';
end
switch lower(basis)
    case 'fourier'
        f_int = inline([target_fun, '(t) .* exp(-i * 2 * ...
            pi * k * t / (T1-T0))'], 't', 'k', 'T0', 'T1');
        fromZero = 0;
    case {'rademacher', 'walsh'}
        f_int = inline([target_fun, '(t) .* ', basis, '((t ...
            - T0)/(T1-T0), k)'], 't', 'k', 'T0', 'T1');
        fromZero = 1;
    case {'haar'}
        c = haar_coef('fun', K, T0, T1);
        return;
end
i = 1;
for k = K
    if (fromZero & k < 0)
        c(i) = 0;
    else
        c(i) = 1 / (T1 - T0) * quadl(@(t) f_int(t, k, T0, ...
            T1), T0, T1);
    end
    i = i + 1;
end
```

Функции Хаара:

```
function y = haar(t, n)
```

```

% k-я функция Хаара
L = length(t);
if (n == 0)
    y(1:L) = 1;
    return;
end
k = floor(log2(n));
m = n - 2^k;
y = zeros(1, L);
step_width = L / 2^(k+1);
support_start = floor(step_width * 2 * m) + 1;
support_middle = floor(step_width * (2*m+1));
support_end = floor(step_width * (2*m+2));
if (support_end > L)
    support_end = L;
end
y(support_start:support_middle) = sqrt(2^k);
y(support_middle + 1 : support_end) = -sqrt(2^k);

```

Нахождение коэффициентов ряда Фурье для системы Хаара:

```

function res = haar_coef(target_fun, K, T0, T1)

f_int = inline([target_fun, '(t)'], 't', 'T0','T1');
i = 1;
for n = K
    if n < 0
        res(i) = 0;
    elseif n == 0
        res(i) = quadl(@(t) f_int(t, T0, T1), T0, T1);
    else
        k = floor(log2(n));
        m = n - 2^k;
        step_width = 1 / 2^(k+1);
        support(1) = step_width * 2 * m;
        support(2) = step_width * (2*m+1);
        support(3) = step_width * (2*m+2);
        support = support * (T1-T0) + T0;
        res(i) = quadl(@(t) f_int(t, T0, T1), support(1), ...
            support(2));
        res(i) = res(i) - quadl(@(t) f_int(t, T0, T1), ...
            support(2), support(3));
        res(i) = res(i) * sqrt(2 ^ k);
    end
    i = i + 1;
end
res = res / (T1 - T0);

```

$$\blacktriangleleft \quad l_0 = 1, l_1 = 0, l_2 = -\frac{\sqrt{2}}{2}, l_3 = \frac{\sqrt{2}}{2}, \quad \text{при } n > 3 \quad l_n = 0. \quad \|f(x)\|^2 = \int_{1/4}^{3/4} 4dx = 2,$$

$$\forall n \quad \|h_n\| = 1 \text{ (система ортонормированная)}. \quad \sum l_n^2 = 1 + \frac{1}{2} + \frac{1}{2} = 2 = \|f(x)\|^2. \blacktriangleright$$

Задание. Представить синусоиду в виде последовательности частичных сумм ряда Фурье по системам Уолша и Хаара, используя программу task_1_2.m:

```
function task_1_2
clear;
close all;
total_num = 20; % всего коэффициентов Фурье
basis = 'walsh'; % базис (fourier, walsh или haar)
T0 = 0; % начало периода
T1 = 1; % конец периода
points = 300; % число точек для вычисления функции
T = T1 - T0;
plot_time = linspace(T0, T1, points);
h = figure;
set(h, 'DoubleBuffer', 'on');
K = [-total_num : total_num];
% Здесь нужно вычислить коэффициенты разложения функции fun
% в ряд Фурье.
% C = fseries(...);
fvalues = fun(plot_time);
for i = 3 : 2 : length(K)
    ind = [-floor(i/2) : floor(i/2)] + total_num + 1;
    % Здесь нужно вычислить частичную сумму ряда Фурье.
    % Индексы коэффициентов из вектора C, которые нужно
    % суммировать, содержатся в векторе ind.
    % Если задание выполнено верно, то при нажатии F5 вы
    % увидите, как происходит приближение функции fun
    % частичными суммами ряда Фурье
    % в выбранном базисе
    % S = real(fsum(...));
    plot(plot_time, fvalues, 'k', plot_time, S, 'r', 'LineWidth', 2);
    grid;
    axis([T0 T1 min(fvalues)-0.1 max(fvalues)+0.1]);
    title('Аппроксимация частичными суммами ряда Фурье')
    drawnow;
end
```

Функция для нахождения частичных сумм:

```
function Sn = fsum(c, K, T0, T1, t, basis)
% частичная сумма ряда Фурье
% Использование:
%   Sn = fsum(c, K, T0, T1, t, basis)
%   c – вектор коэффициентов Фурье
%   T0, T1 – начало и конец периоды функции
%   t – вектор отсчетов времени
```

```

% basis - базис 'fourier' (по умолчанию), 'walsh',
%          'rademacher'
% Sn - частичная сумма
%
% Пример:
% c = [0.4597, -0.2149, -0.1057, -0.0148];
% K = [0 : 3];
% >> plot(fsum(c, K, 0, 1, [0:0.01:1], 'walsh'))
if (nargin < 6)
    basis = 'fourier';
end
Sn = zeros(1, length(t));
switch lower(basis)
    case {'rademacher', 'walsh', 'haar'}
        basefun = inline([basis, '(t, k)'], 't', 'k');
        i = 1;
        for k = K
            w(i,:) = feval(basefun, (t - T0) / (T1 - T0), k);
            i = i + 1;
        end
        for m = 1 : length(t)
            Sn(m) = sum(c .* w(:,m)');
        end
    case 'fourier'
        for k = 1 : length(t)
            Sn(k) = sum(c .* exp(j * 2 * pi * K * t(k)...
                / (T1 - T0)));
        end
end
end

```

Домашнее задание

1. Докажите, что $\|x - y\| \geq \left| \|x\| - \|y\| \right|$.
2. Докажите выполнение аксиом нормы в пространстве $C^n : x = (x_1, \dots, x_n)^T$, $\|x\| = \max_i |x_i|$.
3. Покажите, что если последовательность $\{x_k\}$ – фундаментальная, то последовательность $\{l x_k\}$ – тоже фундаментальная.
4. Докажите, что если $\{x_k\}_{k=1}^{k=m}$ – ортогональная система ненулевых элементов в евклидовом пространстве E , $\{x_k\}_{k=1}^{k=m} \subset E$, то элементы $\{x_k\}_{k=1}^{k=m}$ – линейно независимы.
5. Докажите равенство параллелограмма $\|x + y\|^2 + \|x - y\|^2 = 2(\|x\|^2 + \|y\|^2)$ в гильбертовом пространстве.

6. Убедитесь, что для полной системы функций $\left\{1, \sin \frac{2p}{T} kt, \cos \frac{2p}{T} kt\right\}$ в $L_2[0; T]$ выражение для коэффициентов ряда Фурье, известное из курса математического анализа, есть частный случай формулы $l_k = \frac{\langle x, j_k \rangle}{\langle j_k, j_k \rangle}$.
7. Постройте графики функций Уолша $\{W_k(t)\}_{k=1}^{\infty}$ для $k = 8, \dots, 15$.
8. Разложите функцию $f(x) = \begin{cases} 1, & x \in [1/4; 1/2), \\ 2, & x \in [1/2; 3/4), \\ 0, & \text{иначе} \end{cases}$ в ряд Фурье по системам Хаара и Уолша. Проверьте выполнение равенства Парсеваля.

Семинар №2. Ряды Фурье. Частотное представление сигнала

Тригонометрические ряды Фурье

Тригонометрические ряды Фурье

Пусть $f(t) = f(t+T)$ – кусочно-гладкая на периоде T , тогда она может быть представлена в виде **ряда Фурье**:

$$\frac{f(t+0) + f(t-0)}{2} = \frac{a_0}{2} + \sum_{k=1}^{\infty} \left(a_k \cos \frac{2\pi k}{T} t + b_k \sin \frac{2\pi k}{T} t \right),$$

где

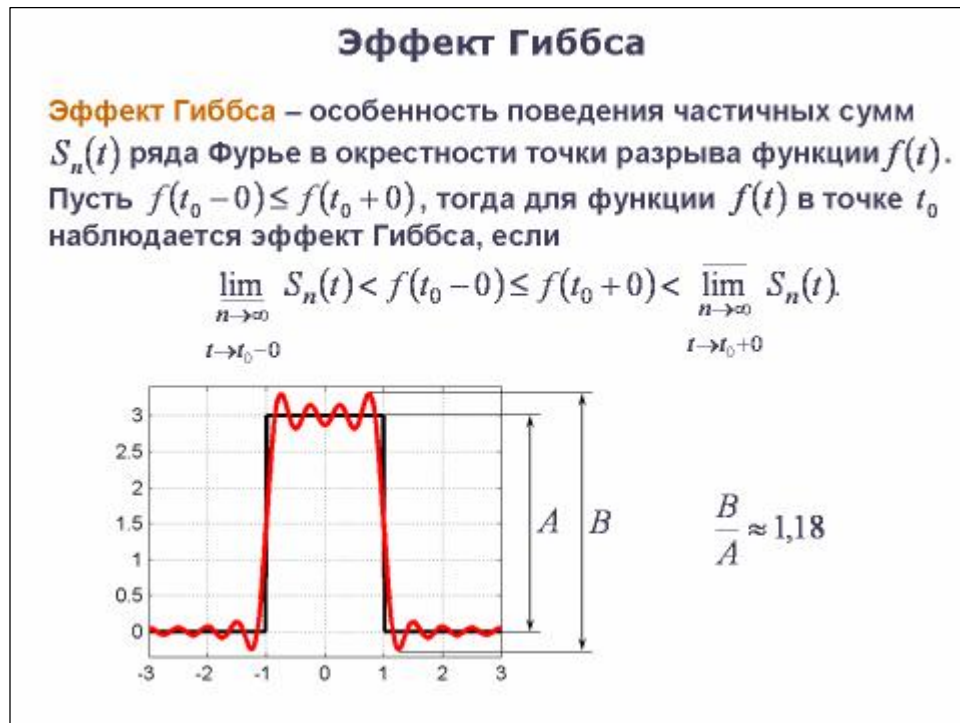
$$\left. \begin{aligned} a_k &= \frac{2}{T} \int_{T/2}^{T/2} f(t) \cos \left(\frac{2\pi k}{T} t \right) dt, k = 0, 1, \dots \\ b_k &= \frac{2}{T} \int_{T/2}^{T/2} f(t) \sin \left(\frac{2\pi k}{T} t \right) dt, k = 1, 2, \dots \end{aligned} \right\} - \text{коэффициенты Фурье}$$

n -я **частичная сумма** ряда Фурье:

$$S_n(t) = \frac{a_0}{2} + \sum_{k=1}^n \left(a_k \cos \frac{2\pi k}{T} t + b_k \sin \frac{2\pi k}{T} t \right).$$

Задание. С помощью программы нахождения коэффициентов Фурье task_1_2.m из предыдущего семинара, написать программу, приближающую прямоугольный импульс частичными суммами ряда Фурье.

Эффект Гиббса



Задание. Проверить выполнение приближенного равенства $\frac{B}{A} \approx 1,18$ для различного числа коэффициентов в частичной сумме.

Переход от периодического к непериодическому сигналу

Переход к непериодическому сигналу

Комплексная форма записи ряда Фурье:

$$\frac{f(t+0) + f(t-0)}{2} = \sum_{k=-\infty}^{\infty} c_k e^{i \frac{2\pi k}{T} t},$$

$$c_k = \frac{1}{T} \int_{-T/2}^{T/2} f(t) \exp\left(-i \frac{2\pi k}{T} t\right) dt, \quad k = 0, \pm 1, \pm 2, \dots$$

Обозначим $\nu_k = k/T$, $\Delta \nu_k = \nu_{k+1} - \nu_k = 1/T = \Delta \nu$, тогда

$$\sum_{k=-\infty}^{\infty} c_k e^{i \frac{2\pi k}{T} t} = \sum_{k=-\infty}^{\infty} \left(\int_{-T/2}^{T/2} f(u) \exp(-i 2\pi \nu_k u) du \right) e^{i 2\pi \nu_k t} \Delta \nu_k.$$

Переход к бесконечному периоду $T \rightarrow \infty$:

$$\lim_{T \rightarrow \infty} \sum_{k=-\infty}^{\infty} c_k e^{i \frac{2\pi k}{T} t} = \lim_{\Delta \nu \rightarrow 0} \sum_{k=-\infty}^{\infty} S(\nu_k) e^{i 2\pi \nu_k t} \Delta \nu_k = \int_{-\infty}^{\infty} S(\nu) e^{i 2\pi \nu t} d\nu.$$

Интеграл Фурье

Интеграл Фурье

Пусть $f(t)$ – кусочно-гладкая на любом конечном отрезке $t \in [a, b] \subset (-\infty; \infty)$ и $\exists \int_{-\infty}^{\infty} |f(t)| dt$, тогда она представима в виде **интеграла Фурье**: $f(t) = \int_{-\infty}^{\infty} S(\nu) e^{2\pi i \nu t} d\nu$, где $S(\nu) = \int_{-\infty}^{\infty} f(t) e^{-2\pi i \nu t} dt$ – **частотный спектр** (спектральная плотность, спектральная характеристика).

$S(\nu) = \int_{-\infty}^{\infty} f(t) e^{-2\pi i \nu t} dt$ – **прямое** преобразование Фурье

$f(t) = \int_{-\infty}^{\infty} S(\nu) e^{2\pi i \nu t} d\nu$ – **обратное** преобразование Фурье

$|S(\nu)|$ – **амплитудный спектр** сигнала

$\varphi(\nu) = -\arg S(\nu)$ – **фазовый спектр** сигнала

Свойства интеграла Фурье

Свойства интеграла Фурье

Некоторые **свойства преобразования Фурье**:

$$\forall f(t) \leftrightarrow S(\nu), \forall x(t) \leftrightarrow S_x(\nu), \forall y(t) \leftrightarrow S_y(\nu), \forall \alpha, \beta:$$

1. Сопряженная симметрия. $f(t) = \overline{f(t)} \leftrightarrow S(\nu) = \overline{S(-\nu)}$.

2. Линейность. $f(t) = \alpha x(t) + \beta y(t) \leftrightarrow S(\nu) = \alpha S_x(\nu) + \beta S_y(\nu)$.

3. Изменение масштаба. $\forall \alpha \neq 0: f(\alpha t) \leftrightarrow S_\alpha(\nu) = \frac{1}{\alpha} S(\nu/\alpha)$.

4. Задержка сигнала. $\forall t_0: f(t - t_0) \leftrightarrow e^{-2\pi i \nu t_0} S(\nu)$.

5. Сдвиг спектра. $f(t) e^{2\pi i \nu_0 t} \leftrightarrow S(\nu - \nu_0)$.

6. Свертка сигналов. $\int_{-\infty}^{\infty} x(t) y(u - t) dt \leftrightarrow S_x(\nu) \cdot S_y(\nu)$.

7. Произведение сигналов. $x(t) y(t) \leftrightarrow \int_{-\infty}^{\infty} S_x(u) \cdot S_y(\nu - u) du$.

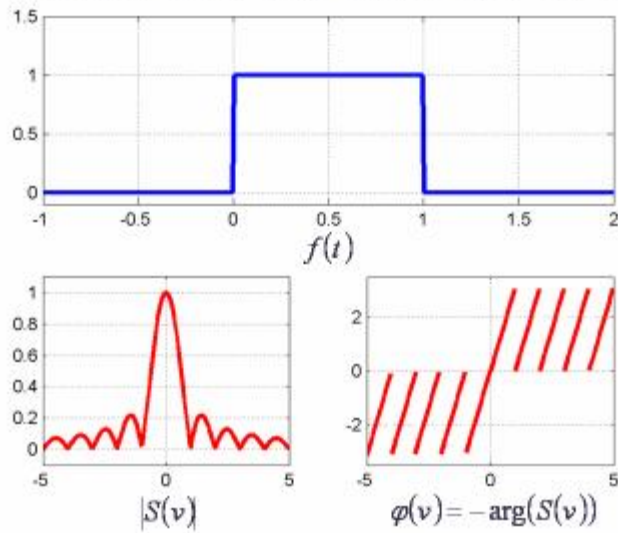
8. Равенство Парсеваля. **Энергия** сигнала

$$E = \int_{-\infty}^{\infty} |f(t)|^2 dt = \int_{-\infty}^{\infty} |S(\nu)|^2 d\nu.$$

Свойства интеграла Фурье

Сопряженная симметрия:

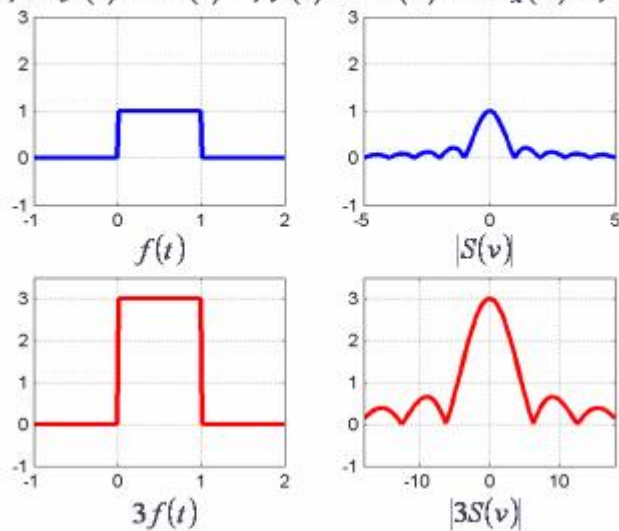
$$\forall f(t) \leftrightarrow S(\nu): \quad f(t) = \overline{f(t)} \leftrightarrow S(\nu) = \overline{S(-\nu)}$$



Свойства интеграла Фурье

Линейность: $\forall f(t) \leftrightarrow S(\nu), \forall x(t) \leftrightarrow S_x(\nu), \forall y(t) \leftrightarrow S_y(\nu)$

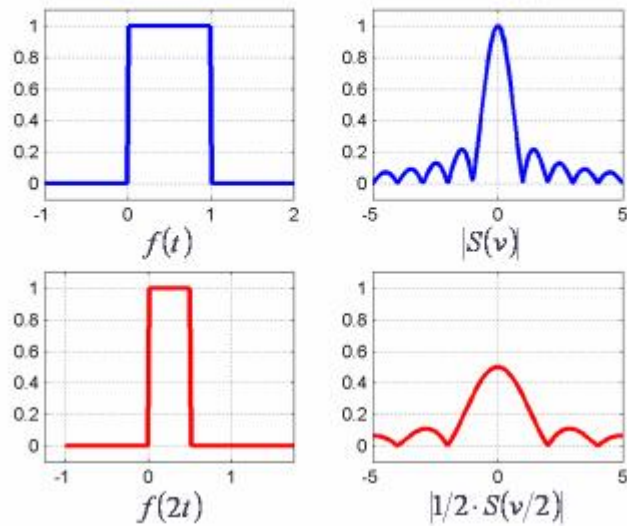
$$\forall \alpha, \beta: f(t) = \alpha x(t) + \beta y(t) \leftrightarrow S(\nu) = \alpha S_x(\nu) + \beta S_y(\nu)$$



Свойства интеграла Фурье

Изменение масштаба: $\forall f(t) \leftrightarrow S(\nu), \forall \alpha \neq 0:$

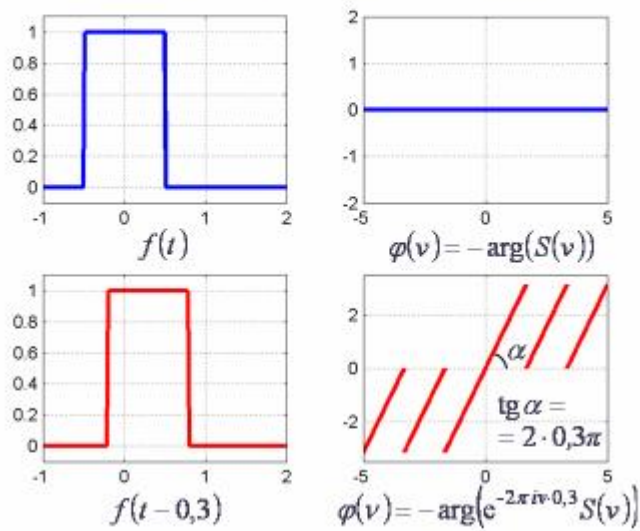
$$f(\alpha t) \leftrightarrow S_{\alpha}(\nu) = S(\nu/\alpha)/|\alpha|$$

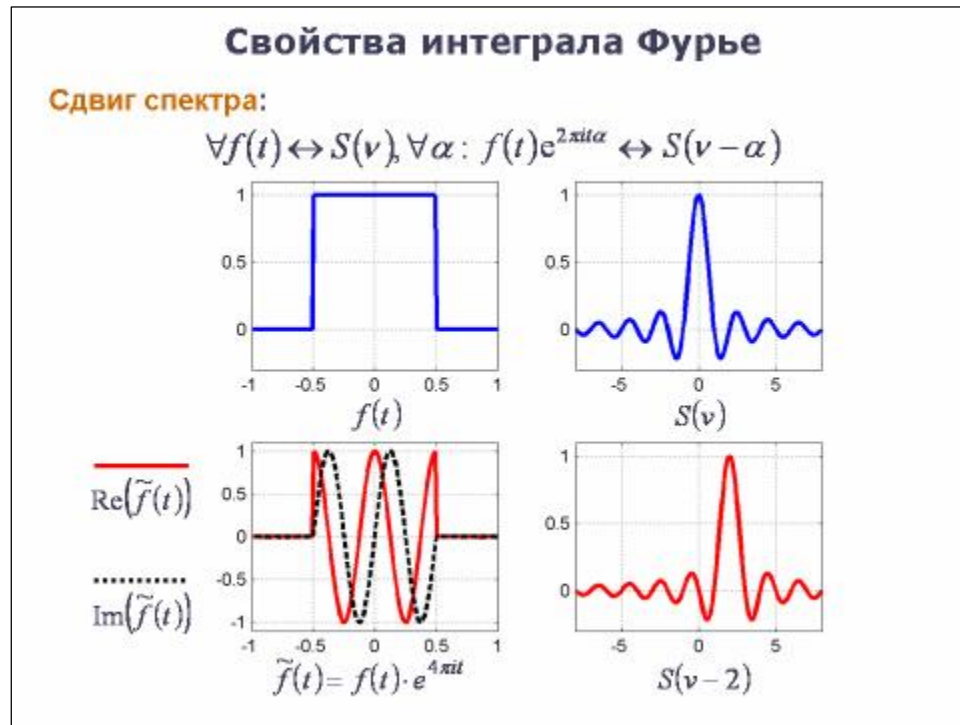


Свойства интеграла Фурье

Задержка сигнала:

$$\forall f(t) \leftrightarrow S(\nu), \forall t_0: f(t - t_0) \leftrightarrow e^{-2\pi i \nu t_0} S(\nu)$$





Задание. Найти амплитудный и фазовый спектры для функции

$$f(x) = \begin{cases} 1, & x \in [0; 0,5] \\ -1, & x \in [0,5; 1] \end{cases} \quad \text{используя то, что спектр прямоугольного импульса}$$

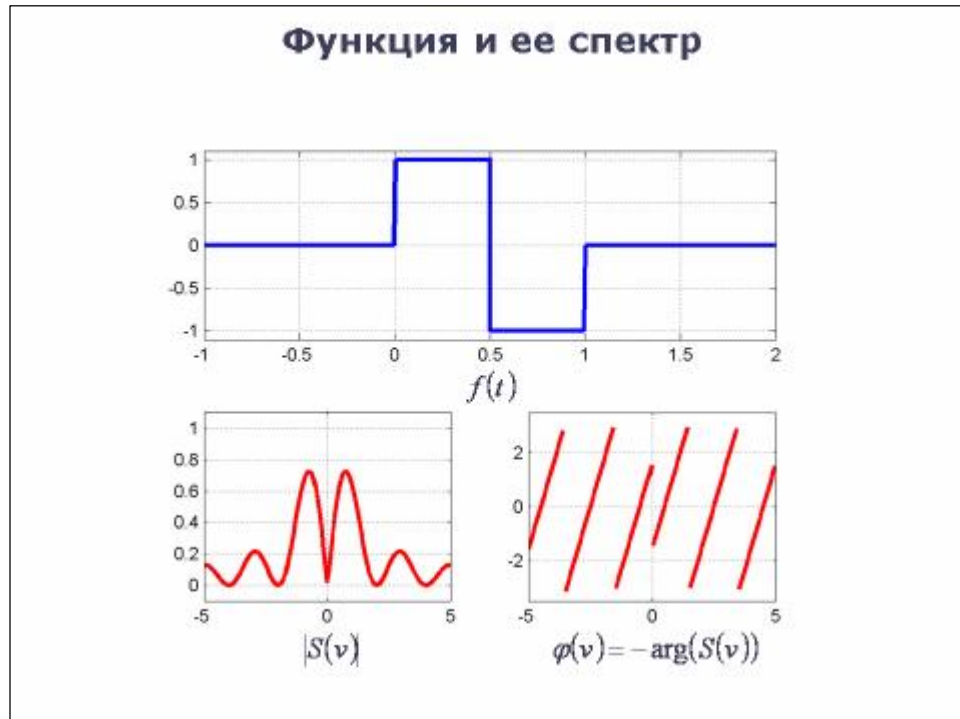
$$\Pi(x) = \begin{cases} 1, & x \in [-0,5; 0,5] \\ 0, & x \notin [-0,5; 0,5] \end{cases} \quad \text{имеет вид } S_{\Pi}(n) = \frac{\sin pn}{pn}.$$

$$\blacktriangleleft f(x) = \Pi\left(2x - \frac{1}{2}\right) - \Pi\left(2x - \frac{3}{2}\right), \text{ следовательно,}$$

$$\begin{aligned} S_f(\nu) &= \frac{1}{2} e^{-2\pi i \frac{\nu}{2} \cdot \frac{1}{2}} \frac{\sin(p\nu/2)}{p\nu/2} - \frac{1}{2} e^{-2\pi i \frac{\nu}{2} \cdot \frac{3}{2}} \frac{\sin(p\nu/2)}{p\nu/2} = \\ &= e^{-\pi i \frac{\nu}{2}} \frac{\sin(p\nu/2)}{p\nu} - e^{-\frac{3}{2}\pi i \nu} \frac{\sin(p\nu/2)}{p\nu} = e^{-\pi i \frac{\nu}{2}} \frac{\sin(p\nu/2)}{p\nu} (1 - e^{-\pi i \nu}) = \\ &= e^{-\pi i \frac{\nu}{2}} \frac{\sin(p\nu/2)}{p\nu} e^{\frac{\pi i \nu}{2}} \left(e^{\frac{\pi i \nu}{2}} - e^{-\frac{\pi i \nu}{2}} \right) = 2i \frac{e^{-\pi i \nu}}{p\nu} \left(\sin \frac{p\nu}{2} \right)^2 = \frac{2}{p\nu} \sin^2 \frac{p\nu}{2} \cdot e^{i\left(\frac{p}{2} - p\nu\right)} = \\ &= \frac{2 \sin^2 \frac{p\nu}{2}}{p\nu} \cdot e^{-ip\left(\nu - \frac{1}{2}\right)} = \frac{1 - \cos p\nu}{p\nu} \cdot e^{-ip\left(\nu - \frac{1}{2}\right)} = S_f(\nu). \end{aligned}$$

$$|S_f(\nu)| = A_f(\nu) = \frac{1 - \cos p\nu}{p|\nu|}, \quad j(\nu) = \begin{cases} -\arg \left(e^{-ip\left(\nu - \frac{1}{2}\right)} \right), & \nu > 0 \\ -\arg \left(-e^{-ip\left(\nu - \frac{3}{2}\right)} \right), & \nu < 0 \end{cases} \in [-p; p]. \blacktriangleright$$

Пример функции и ее спектра



Задание. С помощью программы task_2_1.m выполнить интегральное преобразование Фурье (в символьном виде) тестового сигнала, построить амплитудный и фазовый спектры.

```
% Построение спектра Фурье в символьном виде
function task_2_1
% Отрезок, на котором будет строиться график
xmin = -3;
xmax = 3;
syms t v w f
close all;
f = -heaviside(t-1-0.5) + heaviside(t-0.5);
Sw = fourier(f);
S = subs(Sw, w, 2*pi*v);
% ЧХ
subplot(2, 1, 1);
hold on
h = ezplot(действительная_часть_S, [xmin, xmax]);
set_preTTY(h, [xmin, xmax, -1, 1.5]);
h = ezplot(мнимая_часть_S, [xmin, xmax]);
hold off
set_preTTY(h, [xmin, xmax, -1, 1.5], 'r');
grid;
% АЧХ
subplot(2, 1, 2);
% Здесь нужно построить АЧХ
h = ezplot(АЧХ, [xmin, xmax]);
set_preTTY(h, [xmin, xmax, -1, 1.5]);
return;
```

```

function set_pretty(h, axis_xy, color)
if nargin < 3
    color = 'b';
end
grid;
set(h, 'LineWidth',2.5, 'Color', color);
title(''); xlabel('');
% xmin xmax ymin ymax
axis(axis_xy);

```

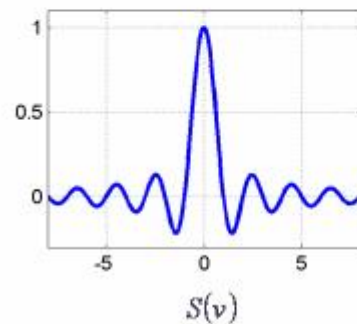
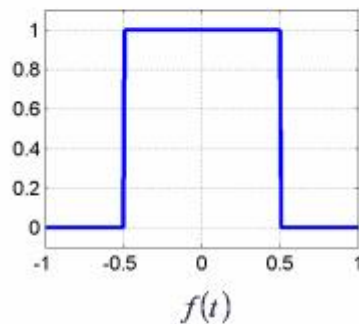
Свойство ограниченного носителя

Свойство ограниченного носителя

Носитель Ω функции $f(t)$: $\Omega = \text{supp } f(t) = \overline{\{t | f(t) \neq 0\}}$.

Функция $f(t)$ имеет **ограниченный носитель**, если
 $\exists [a, b]: \text{supp } f(t) \subset [a, b]$.

Никакая пара $f(t) \leftrightarrow S(v)$ не может **одновременно** иметь ограниченные носители в частотной и временной области!

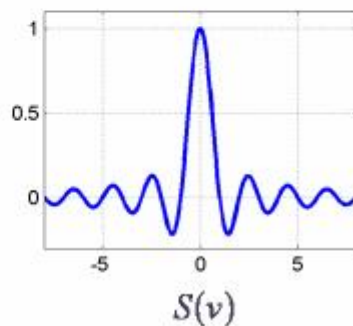
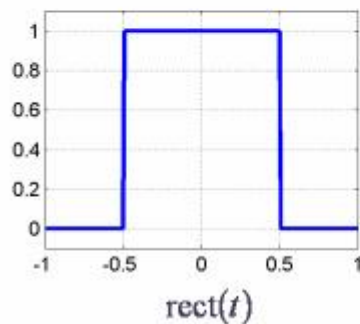


Примеры функций и их спектров

Примеры сигналов и спектров

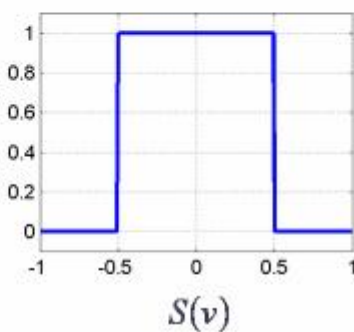
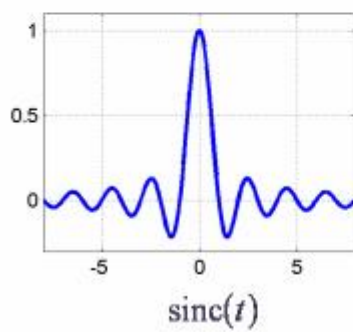
Прямоугольный импульс

$$\text{rect}(t) = \Pi(t) = \begin{cases} 0, & \text{если } |t| > 1/2 \\ 1/2, & \text{если } |t| = 1/2 \\ 1, & \text{если } |t| < 1/2 \end{cases} \longleftrightarrow S(\nu) = \frac{\sin(\pi \nu)}{\pi \nu}$$



Примеры сигналов и спектров

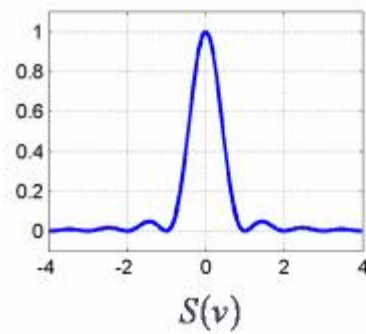
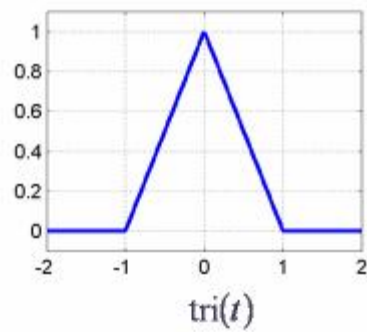
$$\text{sinc}(t) = \frac{\sin(\pi t)}{\pi t} \longleftrightarrow \text{rect}(\nu) = \begin{cases} 0, & \text{если } |\nu| > 1/2 \\ 1/2, & \text{если } |\nu| = 1/2 \\ 1, & \text{если } |\nu| < 1/2 \end{cases}$$



Примеры сигналов и спектров

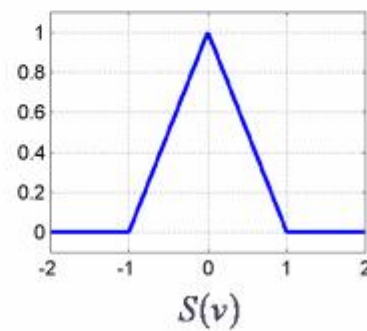
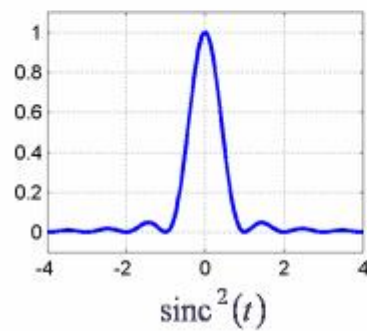
Треугольный импульс

$$\text{tri}(t) = \Lambda(t) = \begin{cases} 1 - |t|, & \text{если } |t| < 1 \\ 0, & \text{иначе} \end{cases} \longleftrightarrow \text{sinc}^2(v) = \frac{\sin^2(\pi v)}{\pi^2 v^2}$$



Примеры сигналов и спектров

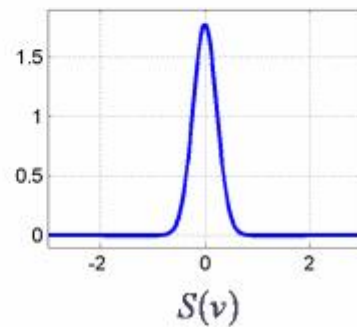
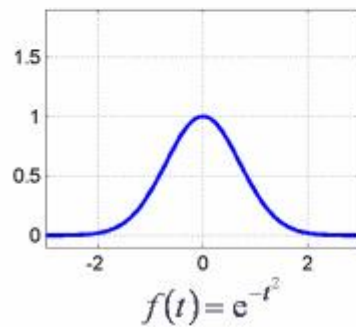
$$\text{sinc}^2(t) = \frac{\sin^2(\pi t)}{\pi^2 t^2} \longleftrightarrow \text{tri}(v) = \begin{cases} 1 - |v|, & \text{если } |v| < 1 \\ 0, & \text{иначе} \end{cases}$$



Примеры сигналов и спектров

Функция Гаусса

$$f(t) = e^{-t^2} \longleftrightarrow S(\nu) = \sqrt{\pi} f(\pi\nu) = \sqrt{\pi} e^{-(\pi\nu)^2}$$

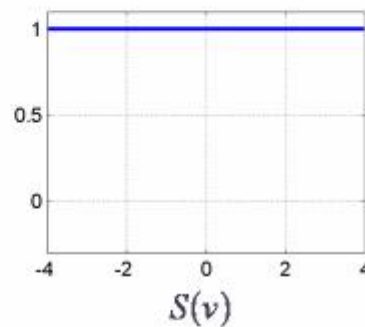
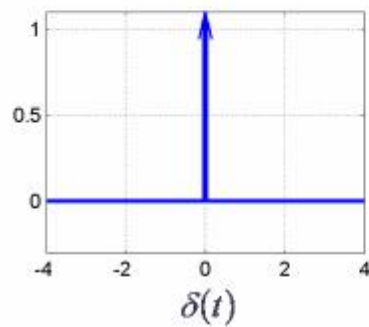


Примеры сигналов и спектров

Дельта-функция Дирака

Пусть $\delta_h(t) = \begin{cases} 1/h, & \text{при } t \in [0; h] \\ 0, & \text{иначе} \end{cases}, \quad \int_{-\infty}^{\infty} \delta_h(t) dt = \int_0^h \frac{dt}{h} = 1.$

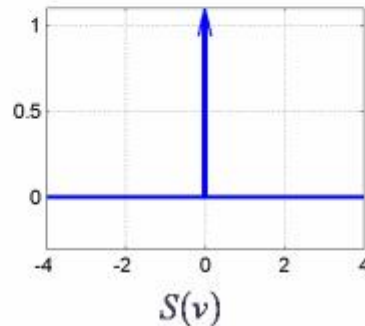
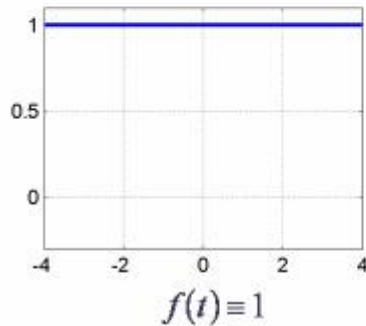
$$\delta(t) = \lim_{h \rightarrow 0} \delta_h(t) = \begin{cases} 0, & t \neq 0 \\ +\infty, & t = 0 \end{cases} \longleftrightarrow S(\nu) = \int_{-\infty}^{\infty} \delta(t) e^{-2\pi i \nu t} dt \equiv 1$$



Примеры сигналов и спектров

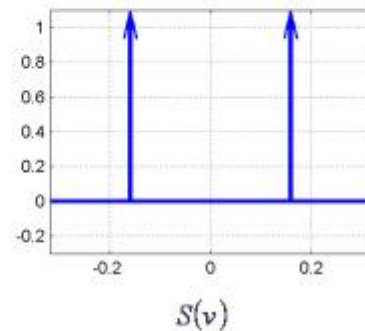
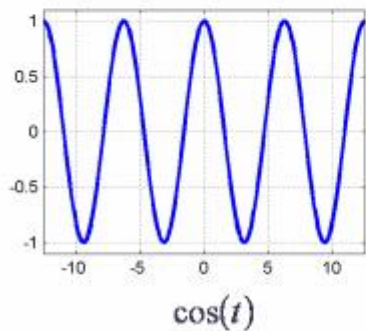
Константа

$$f(t) \equiv 1 \longleftrightarrow \delta(\nu)$$



Примеры сигналов и спектров

$$\cos(t) \longleftrightarrow S(\nu) = \frac{\delta\left(\nu - \frac{1}{2\pi}\right) + \delta\left(\nu + \frac{1}{2\pi}\right)}{2}$$



Домашнее задание

1. Найдите обобщенное преобразование Фурье функции $f(t) = \sin t$
2. Найдите обобщенное преобразование Фурье функции $f(t) = \cos t$.
3. Найдите амплитудный и фазовый спектры прямоугольного импульса

$$f(t) = \begin{cases} 1/\sqrt{T}, & t \in [0; T) \\ 0, & \text{иначе} \end{cases}$$

4. Постройте амплитудный спектр функции Хаара $h_5(t)$.

Семинар №3. Быстрое дискретное преобразование Фурье

Дискретное преобразование Фурье (ДПФ)

Быстрое преобразование Фурье

Дискретное преобразование Фурье (ДПФ):

$$y(k) = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x(j) e^{\frac{2\pi i}{N} kj}, \quad k = 0, \dots, N-1.$$

При $N = 2^n$ и $\omega_n = \exp\left(-\frac{2\pi i}{2^n}\right)$: $\tilde{y}(k) = \sqrt{N} y(k) = \sum_{j=0}^{2^n-1} x(j) \omega_n^{kj}$.

Пусть $X_0 = (x(0), x(2), \dots, x(2^n-2)) = (x_0(0), x_0(1), \dots, x_0(2^{n-1}-1))$

и $X_1 = (x(1), x(3), \dots, x(2^n-1)) = (x_1(0), x_1(1), \dots, x_1(2^{n-1}-1))$, тогда

$$\begin{aligned} \tilde{y}(k) &= \sum_{j=0}^{2^n-1} (x(2j) \omega_n^{k \cdot 2j} + x(2j+1) \omega_n^{k(2j+1)}) = \left| \begin{array}{l} \text{учтем, что} \\ \omega_n^2 = \omega_{n-1} \end{array} \right. = \\ &= \sum_{j=0}^{2^{n-1}-1} x_0(j) \omega_{n-1}^{kj} + \omega_n^k \sum_{j=0}^{2^{n-1}-1} x_1(j) \omega_{n-1}^{kj} = \tilde{y}_0(k) + \omega_n^k \tilde{y}_1(k) \end{aligned}$$

Задание. Определить вычислительную сложность алгоритма вычисления ДПФ.

◀ Вычисление каждого коэффициента $\tilde{y}(k)$ требует около 2^n комплексных сложений с умножениями, поэтому для реализации всего преобразования необходимо около $2^n \cdot 2^n = 2^{2n}$ комплексных сложений с умножениями, т.е. алгоритм вычисления имеет сложность $O(N^2)$. ▶

Идея *быстрого преобразования Фурье (БПФ)* заключается в замене одного ДПФ большой размерности несколькими ДПФ меньшей размерности. Для этого разобьем входной вектор X на четные X_0 и нечетные X_1 компоненты и после преобразований получим выражение для БПФ с *прореживанием по времени*.

Алгоритм БПФ с прореживанием по времени

БПФ с прореживанием по времени

Так как $\tilde{y}_0(k) = \tilde{y}_0(k + 2^{n-1})$, $\tilde{y}_1(k) = \tilde{y}_1(k + 2^{n-1})$ и

$\omega_n^{2^{n-1} + k} = \exp\left(-\frac{2\pi i}{2^n}(2^{n-1} + k)\right) = -\omega_n^k$, то имеем:

$$\begin{cases} \tilde{y}(k) = \tilde{y}_0(k) + \omega_n^k \tilde{y}_1(k) \\ \tilde{y}(k + 2^{n-1}) = \tilde{y}_0(k) - \omega_n^k \tilde{y}_1(k) \\ k = 0, 1, \dots, 2^{n-1} - 1 \end{cases}$$

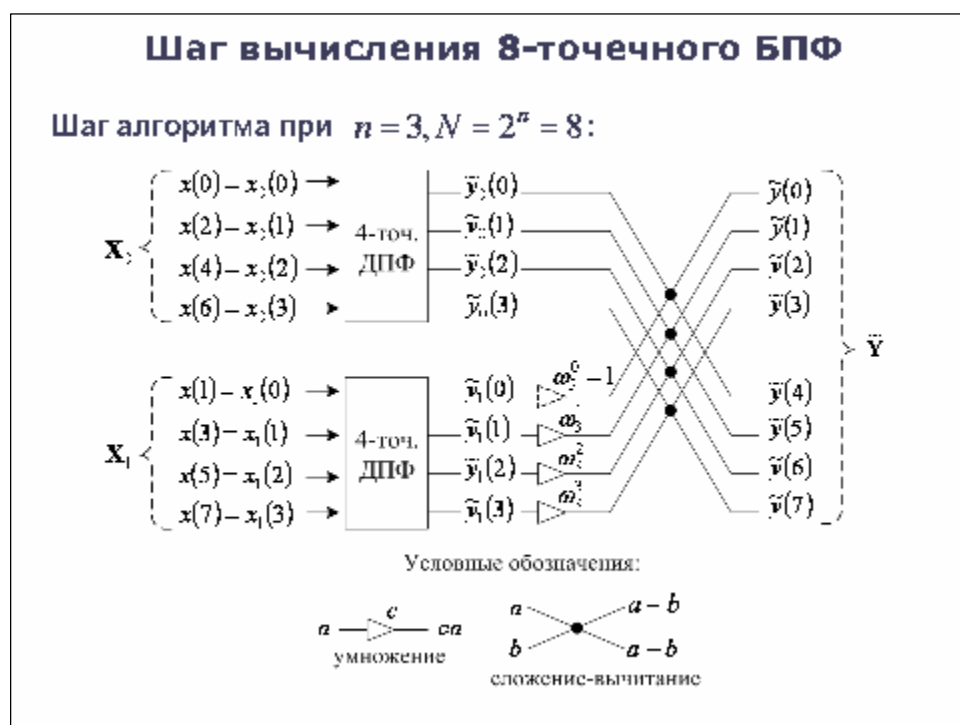
Алгоритм БПФ с прореживанием по времени:

рекуррентное вычисление ДПФ вектора $\tilde{Y}$ за n шагов с помощью сведения одного ДПФ к двум ДПФ меньшей размерности (обработка векторов X_0, X_1 , полученных из исходного вектора X «прореживанием» – выбором каждого второго отсчета).

Задание. Определить вычислительную сложность алгоритма вычисления БПФ с прореживанием по времени.

◀ При таком способе N -точечное ДПФ сводится к двум $\frac{N}{2}$ -точечным ДПФ, N сложениям и $\frac{N}{2}$ умножениям на ω_n^k . Эта же процедура применяется далее для замены двух $\frac{N}{2}$ -точечных ДПФ на четыре $\frac{N}{4}$ -точечных ДПФ, N сложений и $\frac{N}{2}$ умножений. Последовательное применение метода вычисления N -точечных ДПФ, $N = 2^n$, приводит к $n = \log_2 N$ шагам, на каждом из которых 2^i 2^{n-i} -точечных ДПФ сводятся к 2^{i+1} 2^{n-i-1} -точечным ДПФ и, дополнительно, N сложениям и $\frac{N}{2}$ умножениям. Следовательно, число комплексных умножений M и число комплексных сложений A , требуемых для вычисления N -точечного ДПФ, равны $M = \frac{N}{2} \log_2 N$ и $A = N \log_2 N$. ▶

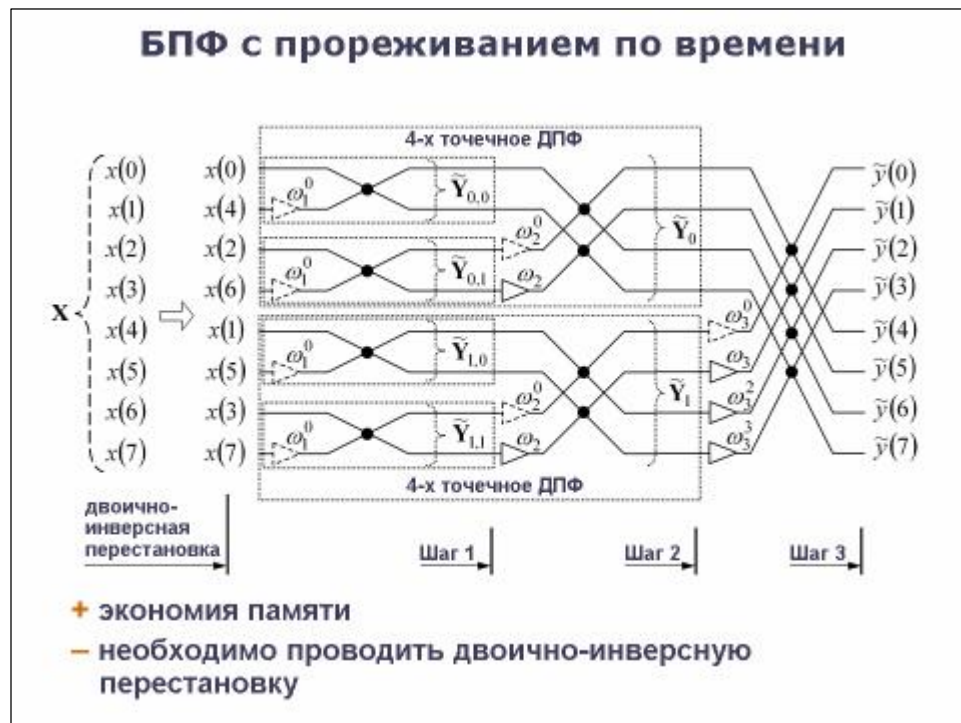
Вычисление 8-точечного ДПФ через два 2-точечных ДПФ



Вычисление 4-точечных ДПФ



Граф 8-точечного БПФ с прореживанием по времени



БПФ с прореживанием по времени без двоично-инверсных перестановок



Факторизация матрицы ДПФ

Факторизация матрицы ДПФ

ДПФ и ОДПФ в **матричном виде** ($N = 2^n$):

$$\begin{cases} Y = \frac{1}{\sqrt{N}} W X \\ X = \frac{1}{\sqrt{N}} \bar{W} Y \end{cases}, \text{ где } W = \left\{ w_{l,m} = \exp\left(-\frac{2\pi i}{N} lm\right) \right\}_{l,m=0}^{N-1}.$$

Факторизация матрицы ДПФ – представление в виде произведения слабозаполненных матриц:

$$W = W_n W_{n-1} \dots W_1,$$

матрица W_j соответствует j -му шагу алгоритма БПФ:

$$\sqrt{N} Y = \tilde{Y} = \underbrace{W_n}_{\text{шаг } n} \underbrace{W_{n-1}}_{\dots} \dots \underbrace{W_1}_{\text{шаг } 1} X$$

Задание. Представить матрицу ДПФ размерности $N=8$ в факторизованном виде, соответствующем алгоритму БПФ с прореживанием по времени с двоично-инверсными перестановками.

◀ Для $N=8=2^3$ получим три матрицы-сомножителя в разложении матрицы ДПФ: $Y = \frac{1}{\sqrt{8}} W_3 W_2 W_1 X$ и три шага алгоритма БПФ. Структура матрицы W_3

соответствует последнему шагу алгоритма БПФ: $\tilde{Y} = W_3 \begin{pmatrix} \tilde{Y}_0 \\ \tilde{Y}_1 \end{pmatrix}$, где $\tilde{Y} = \sqrt{8} Y$, а векторы $\tilde{Y}_0$ и $\tilde{Y}_1$ есть, соответственно, результаты 4-точечных «ненормированных» ДПФ, выполненных над чётными X_0 и нечетными X_1 компонентами вектора X :

$$\tilde{Y} = W_3 \begin{pmatrix} \tilde{Y}_0 \\ \tilde{Y}_1 \end{pmatrix} = W_3 \begin{pmatrix} \tilde{y}_0(0) \\ \tilde{y}_0(1) \\ \tilde{y}_0(2) \\ \tilde{y}_0(3) \\ \tilde{y}_1(0) \\ \tilde{y}_1(1) \\ \tilde{y}_1(2) \\ \tilde{y}_1(3) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & w_3 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & w_3^2 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & w_3^3 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -w_3 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & -w_3^2 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -w_3^3 \end{pmatrix} \begin{pmatrix} \tilde{y}_0(0) \\ \tilde{y}_0(1) \\ \tilde{y}_0(2) \\ \tilde{y}_0(3) \\ \tilde{y}_1(0) \\ \tilde{y}_1(1) \\ \tilde{y}_1(2) \\ \tilde{y}_1(3) \end{pmatrix}.$$

w_3

$$\tilde{\mathbf{Y}}_0 = \begin{pmatrix} \tilde{y}_0(0) \\ \tilde{y}_0(1) \\ \tilde{y}_0(2) \\ \tilde{y}_0(3) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & w_2 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -w_2 \end{pmatrix} \begin{pmatrix} \tilde{\mathbf{Y}}_{0,0} \\ \tilde{\mathbf{Y}}_{0,1} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & w_2 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -w_2 \end{pmatrix} \begin{pmatrix} \tilde{y}_{0,0}(0) \\ \tilde{y}_{0,0}(1) \\ \tilde{y}_{0,1}(0) \\ \tilde{y}_{0,1}(1) \end{pmatrix},$$

$$\tilde{\mathbf{Y}}_1 = \begin{pmatrix} \tilde{y}_1(0) \\ \tilde{y}_1(1) \\ \tilde{y}_1(2) \\ \tilde{y}_1(3) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & w_2 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -w_2 \end{pmatrix} \begin{pmatrix} \tilde{\mathbf{Y}}_{1,0} \\ \tilde{\mathbf{Y}}_{1,1} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & w_2 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -w_2 \end{pmatrix} \begin{pmatrix} \tilde{y}_{1,0}(0) \\ \tilde{y}_{1,0}(1) \\ \tilde{y}_{1,1}(0) \\ \tilde{y}_{1,1}(1) \end{pmatrix},$$

где $w_2 = \exp\left(-i\frac{p}{2}\right) = -i$. Отсюда

$$\begin{pmatrix} \tilde{\mathbf{Y}}_0 \\ \tilde{\mathbf{Y}}_1 \end{pmatrix} = \begin{pmatrix} \tilde{y}_0(0) \\ \tilde{y}_0(1) \\ \tilde{y}_0(2) \\ \tilde{y}_0(3) \\ \tilde{y}_1(0) \\ \tilde{y}_1(1) \\ \tilde{y}_1(2) \\ \tilde{y}_1(3) \end{pmatrix} = \begin{pmatrix} \boxed{1} & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & \boxed{1} & 0 & -i & 0 & 0 & 0 & 0 \\ 1 & 0 & \boxed{-1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & \boxed{i} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \boxed{1} & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & \boxed{1} & 0 & -i \\ 0 & 0 & 0 & 0 & 1 & 0 & \boxed{-1} & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & \boxed{i} \end{pmatrix} \begin{pmatrix} \tilde{y}_{0,0}(0) \\ \tilde{y}_{0,0}(1) \\ \tilde{y}_{0,1}(0) \\ \tilde{y}_{0,1}(1) \\ \tilde{y}_{1,0}(0) \\ \tilde{y}_{1,0}(1) \\ \tilde{y}_{1,1}(0) \\ \tilde{y}_{1,1}(1) \end{pmatrix}$$

$\underbrace{\quad}_{w_2}$

Для $\mathbf{W}_1$ имеем:

$$\mathbf{W}_1 = \begin{pmatrix} \boxed{1} & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & \boxed{-1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & \boxed{-1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \boxed{1} & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \boxed{-1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \boxed{1} & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & \boxed{-1} \end{pmatrix} \begin{pmatrix} x(0) \\ x(4) \\ x(2) \\ x(6) \\ x(1) \\ x(5) \\ x(3) \\ x(7) \end{pmatrix},$$

причем

$$\begin{pmatrix} x(0) \\ x(4) \\ x(2) \\ x(6) \\ x(1) \\ x(5) \\ x(3) \\ x(7) \end{pmatrix} = \mathbf{C} \cdot \begin{pmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \\ x(4) \\ x(5) \\ x(6) \\ x(7) \end{pmatrix}, \text{ где } \mathbf{C} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} -$$

матрица перестановок элементов, номера которых j и $\tilde{j}$ образуют двоично-инверсную пару, т.е. если $j = \sum_{k=0}^{n-1} j_k \cdot 2^k$, то $\tilde{j} = \sum_{k=0}^{n-1} j_{n-k-1} \cdot 2^k$, $j = 0, 1, \dots, 2^n - 1$.

Окончательно: $\mathbf{W} = \mathbf{W}_3 \mathbf{W}_2 \mathbf{W}_1 \mathbf{C}$, $\tilde{\mathbf{Y}} = \mathbf{W} \mathbf{X}$. ►

Алгоритм БПФ с прореживанием по частоте

БПФ с прореживанием по частоте

Ненормированное ДПФ: $\tilde{y}(k) = \sum_{j=0}^{2^n-1} x(j) \omega_n^{kj}, \omega_n = e^{\frac{2\pi i}{2^n}}$

$$\tilde{y}(k) = \sum_{j=0}^{2^{n-1}-1} x(j) e^{\frac{2\pi i}{2^n} kj} + \sum_{j=0}^{2^{n-1}-1} x(j+2^{n-1}) e^{\frac{2\pi i}{2^n} k(j+2^{n-1})}$$

С учетом того, что $e^{\frac{2\pi i}{2^n} 2^{n-1} k} = e^{\pi i k} = (-1)^k$ получим:

Алгоритм БПФ с прореживанием по частоте

{

$$\begin{cases} \tilde{y}(2k) = \sum_{j=0}^{2^{n-1}-1} (x(j) + x(j+2^{n-1})) \omega_{n-1}^{kj}, \\ \tilde{y}(2k+1) = \sum_{j=0}^{2^{n-1}-1} (x(j) - x(j+2^{n-1})) \omega_{n-1}^{kj} \omega_n^j, \\ k = 0, \dots, 2^{n-1} - 1. \end{cases}$$

Как и для алгоритма с прореживанием по времени, здесь также существует два варианта реализации алгоритма БПФ.

Граф 8-точечного БПФ с прореживанием по частоте



БПФ с прореживанием по частоте без перестановок



Задание. Представить матрицу ДПФ размерности $N=8$ в факторизованном виде, соответствующем алгоритму БПФ с прореживанием по частоте без двоично-инверсных перестановок.

$$\blacktriangleleft \tilde{\mathbf{Y}} = \underset{\mathbf{w}}{\mathbf{W}_3 \mathbf{W}_2 \mathbf{W}_1} \mathbf{X}.$$

$$\mathbf{W}_1 = \begin{pmatrix} \boxed{1} & 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \\ \boxed{1} & 0 & 0 & 0 & \boxed{-1} & 0 & 0 & 0 \\ 0 & \boxed{1} & 0 & 0 & 0 & \boxed{1} & 0 & 0 \\ 0 & \boxed{w_3} & 0 & 0 & 0 & \boxed{-w_3} & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 & \boxed{1} & 0 \\ 0 & 0 & \boxed{w_3^2} & 0 & 0 & 0 & \boxed{-w_3^2} & 0 \\ 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 & \boxed{1} \\ 0 & 0 & 0 & \boxed{w_3^3} & 0 & 0 & 0 & \boxed{-w_3^3} \end{pmatrix}$$

$$\mathbf{W}_2 = \begin{pmatrix} \boxed{1} & 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \\ \boxed{0} & 1 & 0 & 0 & \boxed{0} & 1 & 0 & 0 \\ \boxed{1} & 0 & 0 & 0 & \boxed{-1} & 0 & 0 & 0 \\ \boxed{0} & 1 & 0 & 0 & \boxed{0} & -1 & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 & \boxed{1} & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & \boxed{w_2} & 0 & 0 & 0 & \boxed{-w_2} & 0 \\ 0 & 0 & 0 & \boxed{w_2} & 0 & 0 & 0 & \boxed{-w_2} \end{pmatrix}$$

$$\mathbf{W}_3 = \begin{pmatrix} \boxed{1} & 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \\ \boxed{0} & 1 & 0 & 0 & \boxed{0} & 1 & 0 & 0 \\ \boxed{0} & 0 & 1 & 0 & \boxed{0} & 0 & 1 & 0 \\ \boxed{0} & 0 & 0 & 1 & \boxed{0} & 0 & 0 & 1 \\ \boxed{1} & 0 & 0 & 0 & \boxed{-1} & 0 & 0 & 0 \\ \boxed{0} & 1 & 0 & 0 & \boxed{0} & -1 & 0 & 0 \\ \boxed{0} & 0 & 1 & 0 & \boxed{0} & 0 & -1 & 0 \\ \boxed{0} & 0 & 0 & 1 & \boxed{0} & 0 & 0 & \boxed{-1} \end{pmatrix} \blacktriangleright.$$

Домашнее задание

1. Вычислите ДПФ сигнала $x_j = \sin\left(\frac{4p}{N} j\right)$ $j = 0, 1, \dots, N-1$.
2. Представьте матрицу ДПФ размерности $N=8$ в факторизованном виде, соответствующем алгоритму БПФ с прореживанием по времени без двоично-инверсных перестановок.
3. Представьте матрицу ДПФ размерности $N=8$ в факторизованном виде, соответствующем алгоритму БПФ с прореживанием по частоте с двоично-инверсными перестановками.
4. Допустим, в вашем распоряжении есть две программы, вычисляющие ДПФ последовательности $x(n)$ с $N = 2^n$ отсчетами. Программа A за N^2 с вычисляет ДПФ по определению. Программа B реализует алгоритм БПФ с прореживанием по времени и выполняется за $10N \log_2 N$ с. Найдите наименьшее значение N , при котором программа B работает быстрее программы A .
5. Пусть размерность векторов $\mathbf{x}$ и $\mathbf{y}$ равна $N = 2^n$. При каких значениях N применение БПФ приводит к сокращению числа выполняемых арифметических операций при вычислении свертки векторов $\mathbf{x} * \mathbf{y}$?

Семинар №4. Z-преобразование

Определение Z-преобразования

Z-преобразование

Функция $f(z)$ – **аналитическая** в области D , если она дифференцируема в этой области, и $f'(z)$ непрерывна в D .

Пусть $f(z)$ – аналитическая в области D , $g(z)$ – аналитическая в области G и $f(z)=g(z)$ в $D_1=D \cap G$, тогда

$$F(z) = \begin{cases} f(z), & z \in D \\ g(z), & z \in G \setminus D_1 \end{cases} \quad \text{– аналитическое продолжение } f(z) \text{ с области } D \text{ на область } G \setminus D_1$$

Z-преобразование числовой последовательности $\{x(k)\}_{k=0}^{\infty}$:

$$X(z) = Z\{x(n)\} = \begin{cases} \varphi(z) = \sum_{k=0}^{\infty} x(k)z^{-k}, & \text{при } |z| > R = \limsup_{n \rightarrow \infty} \sqrt[n]{|x(n)|} \\ \text{аналитическое продолжение } \varphi(z) & \\ \text{в круг } |z| \leq R \end{cases}$$

Задание. Найти Z-преобразование последовательности $x(n) = a^n$.

$$\blacktriangleleft X(z) = \sum_{k=0}^{\infty} x(k)z^{-k} = \sum_{k=0}^{\infty} \left(\frac{a}{z}\right)^k = \frac{1}{1 - \frac{a}{z}} = \frac{z}{z-a}, \text{ при } |z| > R = \limsup_{n \rightarrow \infty} \sqrt[n]{|a|^n} = |a|. \blacktriangleright$$

Задание. Найти Z-преобразование последовательности $x(n) = \sin bn$.

$$\begin{aligned} \blacktriangleleft \sin bn &= \frac{1}{2i}(e^{ibn} - e^{-ibn}). \quad X(z) = \frac{1}{2i} \left[\sum_{k=0}^{\infty} \frac{e^{ibk}}{z^k} - \sum_{k=0}^{\infty} \frac{e^{-ibk}}{z^k} \right] = \frac{1}{2i} \left[\frac{1}{1 - \frac{e^{ib}}{z}} - \frac{1}{1 - \frac{e^{-ib}}{z}} \right] = \\ &= \frac{1}{2i} \left[\frac{z}{z - e^{ib}} - \frac{z}{z - e^{-ib}} \right] = \frac{z}{2i} \frac{z - e^{-ib} - z + e^{ib}}{(z - e^{ib})(z - e^{-ib})} = \frac{z \sin b}{z^2 - 2z \cos b + 1}, \text{ при} \\ |z| > R &= \limsup_{n \rightarrow \infty} \sqrt[n]{|\sin bn|} = 1. \blacktriangleright \end{aligned}$$

Задание. Решить предыдущий пример в MATLAB в символьном виде.

$$\blacktriangleleft \text{syms } b \ n; \ y = \sin(b*n); \ Z = \text{ztrans}(y); \blacktriangleright$$

Свойства Z-преобразования

Свойства Z-преобразования

Некоторые свойства Z-преобразования:

$$\forall x(n) \leftrightarrow X(z), \forall y(n) \leftrightarrow Y(z), \forall M, N > 0, \forall \alpha, \beta:$$

1. Линейность. $\alpha x(n) + \beta y(n) \leftrightarrow \alpha X(z) + \beta Y(z)$.
2. Задержка. $x(n-N) \leftrightarrow z^{-N} X(z)$.
3. Опережение. $x(n+M) \leftrightarrow z^M \left(X(z) - \sum_{k=0}^{M-1} x(k) \cdot z^{-k} \right)$.
4. Умножение на a^n . $a^n x(n) \leftrightarrow X(z/a)$.
5. Умножение на n . $n x(n) \leftrightarrow -z \cdot dX(z)/dz$.
6. Свертка. $c(n) = \sum_{k=0}^n x(k)y(n-k) \leftrightarrow X(z)Y(z)$.
7. Обратное Z-преобразование.

$$x(n) = \frac{1}{2\pi i} \oint_{\gamma} X(z) z^{n-1} dz, \text{ где } \gamma - \text{произвольный замкнутый контур, внутри которого лежат все особые точки функции } X(z).$$

Задание. Найти Z-преобразование последовательности $x(n) = n^2 e^{an}$.

◀ $e^{an} \leftrightarrow \frac{z}{z-e^a}$. Используем 2 раза свойство 5:

$$n e^{an} \leftrightarrow -z \frac{dX(z)}{dz} = -z \frac{d\left(\frac{z}{z-e^a}\right)}{dz} = -z \frac{-e^a}{(z-e^a)^2} = \frac{ze^a}{(z-e^a)^2},$$

$$n^2 e^{an} \leftrightarrow -z \frac{d\left(\frac{ze^a}{(z-e^a)^2}\right)}{dz} = -z \frac{e^a(z-e^a)^2 - 2(z-e^a)ze^a}{(z-e^a)^4} = \frac{ze^a(z+e^a)}{(z-e^a)^3}. \blacktriangleright$$

Задание. Проверить правильность решения, используя символьные вычисления.

◀ `syms a n; y = n^2*exp(a*n); Z = ztrans(y);` ▶

Нахождение обратного Z-преобразования

Обратное Z-преобразование

$$x(n) = \frac{1}{2\pi i} \oint_{\gamma} X(z) z^{n-1} dz = \sum_{k=1}^N \text{res} [X(z) z^{n-1}, z_k],$$

где $\{z_k\}_{k=1}^N$ – особые точки функции $X(z)$, лежащие в γ .

Способы вычисления вычетов:

1. Если z_0 – полюс порядка m функции $f(z)$, то

$$\text{res}[f(z), z_0] = \frac{1}{(m-1)!} \cdot \lim_{z \rightarrow z_0} \frac{d^{m-1}}{dz^{m-1}} \left((z - z_0)^m f(z) \right).$$

$$\text{При } m=1: \text{res}[f(z), z_0] = \lim_{z \rightarrow z_0} (z - z_0) f(z).$$

2. Если в некоторой окрестности точки z_0 : $f(z) = \frac{u(z)}{v(z)}$

и $u(z_0) \neq 0$, $v(z_0) = 0$, $\frac{d}{dz} v(z_0) \neq 0$, тогда

$$\text{res}[f(z), z_0] = u(z_0) / v'(z_0).$$

Задание. Найти последовательность $x(n)$ по заданному Z-образу

$$X(z) = \frac{z}{(z^2 - 9)^2}.$$

◀ Имеем 2 полюса 2-го порядка:

$$x(n) = \frac{1}{2\pi i} \oint_{\gamma} X(z) z^{n-1} dz = \text{res} \left[\frac{z^n}{(z^2 - 9)^2}; 3 \right] + \text{res} \left[\frac{z^n}{(z^2 - 9)^2}; -3 \right].$$

$$\text{res} \left[\frac{z^n}{(z^2 - 9)^2}; 3 \right] = \lim_{z \rightarrow 3} \frac{d}{dz} \left[\frac{z^n}{(z+3)^2} \right] = \lim_{z \rightarrow 3} \left[\frac{nz^{n-1}}{(z+3)^2} - \frac{2z^n}{(z+3)^3} \right] = \frac{(n-1)3^{n-3}}{4},$$

$$\text{res} \left[\frac{z^n}{(z^2 - 9)^2}; 3 \right] = \lim_{z \rightarrow -3} \frac{d}{dz} \left[\frac{z^n}{(z-3)^2} \right] = -(-1)^n \frac{(n-1)3^{n-3}}{4}, \quad x(n) = \frac{3^{n-3}}{4} (n-1) (1 - (-1)^n). \quad \blacktriangleright$$

Задание. Найти последовательность с помощью средств символьных вычислений MATLAB.

◀ `syms z; Z = z/(z^2-9)^2; y = iztrans(Z); y = simple(y);` ▶

Последовательности и их Z-образы

Таблица Z-преобразований

Последовательность	Z-преобразование
$x(n) = \begin{cases} 1, n \geq 0 \\ 0, n < 0 \end{cases}$	$X(z) = \frac{z}{z-1}$
$x(n) = a^n$	$X(z) = z/(z-a)$
$x(n) = n$	$X(z) = z/(z-1)^2$
$x(n) = n^2$	$X(z) = z(z+1)/(z-1)^3$
$x(n) = \sin \beta n$	$X(z) = \frac{z \sin \beta}{z^2 - 2z \cos \beta + 1}$
$x(n) = \cos \beta n$	$X(z) = \frac{z(z - \cos \beta)}{z^2 - 2z \cos \beta + 1}$

Применение Z-преобразования для решения линейных разностных уравнений

Линейные разностные уравнения

Линейное разностное уравнение N -го порядка:

$$\sum_{k=0}^N a_k x(n+k) = f(n),$$

где a_k – константы, $f(n)$ – заданная последовательность, $x(n)$ – искомая последовательность

Начальные условия: $x(0) = x_0, x(1) = x_1, \dots, x(N-1) = x_{N-1}$.

Положим $a_N = 1$, тогда $x(N+n) = f(n) - \sum_{k=0}^{N-1} a_k x(n+k)$.

Использование Z-преобразования для решения разностных уравнений:

$$\sum_{k=0}^N a_k z^k \left(X(z) - \sum_{j=0}^{k-1} x(j) z^{-j} \right) = Z\{f(n)\}.$$

Задание. Решить разностное уравнение $x(n+2) - 5x(n+1) + 6x(n) = 0$ при начальных условиях $x(0) = 1, x(1) = 2$.

◀ По свойству 5 (опережение):

$$x(n+2) \leftrightarrow z^2 \left(X(z) - x(0) - \frac{x(1)}{z} \right) = X(z)z^2 - z^2 + 2z,$$

$$x(n+1) \leftrightarrow z(X(z) - x(0)) = X(z)z - z. \text{ Отсюда имеем:}$$

$$X(z)z^2 - z^2 + 2z - 5X(z)z + 5z + 6X(z) = 0, \quad X(z) = \frac{z^2 - 7z}{z^2 - 5z + 6} = \frac{z(z-7)}{(z-3)(z-2)}.$$

Находим обратное Z-преобразование:

$$x(n) = \lim_{z \rightarrow 2} \frac{z^n(z-7)}{z-3} + \lim_{z \rightarrow 3} \frac{z^n(z-7)}{z-2} = 5 \cdot 2^n - 4 \cdot 3^n \text{ или в MATLAB:}$$

`syms z; Z = z*(z-7)/((z-3)*(z-2)); y = iztrans(Z); ▶`

Домашнее задание

1. Найдите Z-преобразование последовательности $x(n) = \cos bn$.
2. Найдите Z-преобразование последовательности $x(n) = e^{an} \sin bn$.
3. Найдите последовательность $x(n)$ по Z-образу $X(z) = \frac{1}{(z-a)(z-b)}$.
4. Найдите последовательность $x(n)$ по Z-образу $X(z) = \frac{z}{z^4 + 1}$.
5. Решите разностное уравнение $x(n+2) - 3x(n+1) - 10x(n) = 0$ при $x(0) = 3$ и $x(1) = -1$.
6. Решите разностное уравнение $x(n+2) - \sqrt{3}x(n+1) + x(n) = 0$ при $x(0) = \frac{1}{2}$ и

$$x(1) = \frac{\sqrt{3}}{2}.$$

Семинар №5. Линейные дискретные фильтры

Определение линейного дискретного фильтра (ЛДФ)

Линейный дискретный фильтр

Линейный дискретный фильтр (ЛДФ, линейная дискретная система) – устройство, преобразующее входную последовательность $x(n)$ в выходную $y(n)$ по правилу:

$$\sum_{m=0}^M a_m y(n-m) = \sum_{k=0}^N b_k x(n-k), \quad \forall n \in \mathbb{Z}.$$

Положим $a_0 = 1$: $y(n) = -\sum_{m=1}^M a_m y(n-m) + \sum_{k=0}^N b_k x(n-k).$

Если $\exists m: a_m \neq 0$, то фильтр **рекурсивный**, в противном случае – **нерекурсивный**.

Импульсная характеристика $h(n)$ ЛДФ – отклик на единичное воздействие: $h(n) = y(n)_{x(n)=\delta(n)}$, $\delta(n) = \begin{cases} 1, & n=0 \\ 0, & n \neq 0 \end{cases}$

Если $\exists M: \forall m > M \ h(m) = 0$, то имеем **КИХ-фильтр**, иначе – **БИХ-фильтр**.

Задание. Найти реакцию фильтра, описываемого разностным уравнением $y(n) = x(n) + bx(n-1)$, на входное воздействие $x(n) = \tilde{d}(n)$. Каким является данный фильтр: рекурсивным/нерекурсивным, КИХ/БИХ?

◀ $y(0) = \tilde{d}(0) + b\tilde{d}(-1) = 1$, $y(1) = \tilde{d}(1) + b\tilde{d}(0) = b$, $\forall n > 1: y(n) = 0$. Имеем нерекурсивный КИХ-фильтр. ▶

Передаточная функция линейного дискретного фильтра

Передаточная функция

Передаточная функция ЛДФ – отношение Z-образов выходной и входной последовательностей:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^N b_k z^{-k}}{1 + \sum_{m=1}^M a_m z^{-m}}, \text{ при этом } H(z) = Z\{h(n)\}.$$

Способы нахождения отклика фильтра:

1. Через коэффициенты фильтра:

$$y(n) = -\sum_{m=1}^M a_m y(n-m) + \sum_{k=0}^N b_k x(n-k).$$

2. Через импульсную характеристику:

$$y(n) = \sum_{m=0}^n x(m)h(n-m) = \sum_{m=0}^n h(m)x(n-m).$$

3. Через передаточную функцию:

$$y(n) = Z^{-1}\{Y(z)\}, Y(z) = H(z)X(z).$$

Задание. Для ЛДФ $y(n) = -2y(n-1) - y(n-2) + x(n) + x(n-1)$ найти

- 1) передаточную функцию,
- 2) импульсную характеристику,
- 3) отклик фильтра на входное воздействие $x(n) = 3^n$, используя уравнение свертки и передаточную функцию.

◀ 1) Имеем $a_1 = 2, a_2 = 1, b_0 = 1, b_1 = 1$, следовательно:

$$H(z) = \frac{1 + z^{-1}}{1 + 2z^{-1} + z^{-2}} = \frac{z^2 + z}{z^2 + 2z + 1} = \frac{z(z+1)}{(z+1)^2} = \frac{z}{z+1}.$$

$$2) \text{ Для ИХ получаем: } h(n) = Z^{-1}\{H(z)\} = \text{res}\left[\frac{z^n}{z+1}, z = -1\right] = \lim_{z \rightarrow -1} (z^n) = (-1)^n.$$

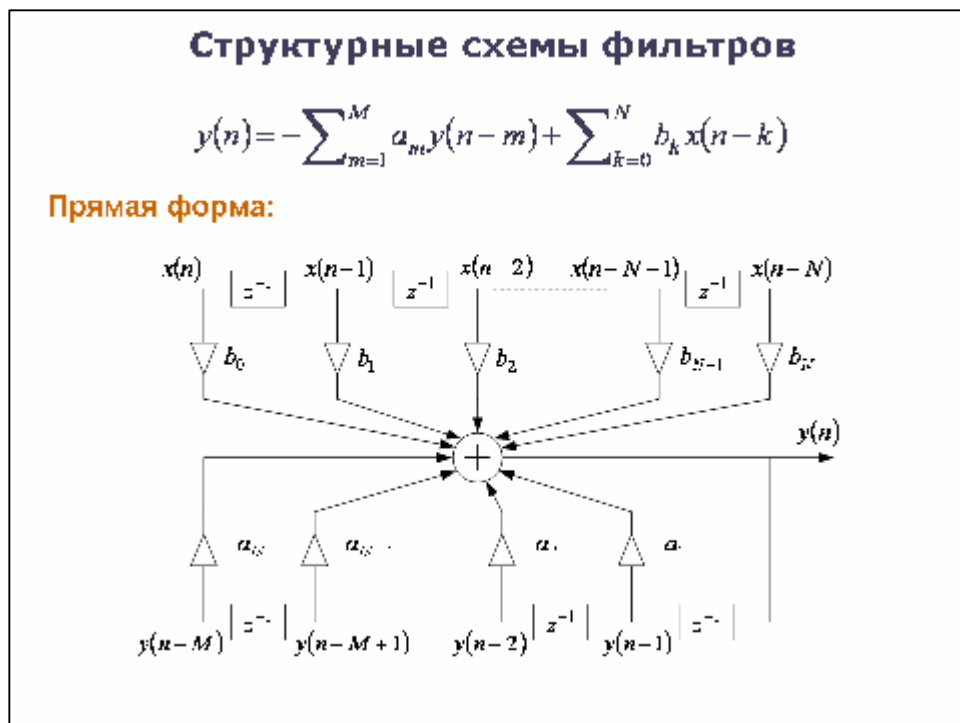
3) Используя формулу свертки, имеем:

$$\begin{aligned} y(n) &= \sum_{k=0}^n x(k)h(n-k) = \sum_{k=0}^n 3^k (-1)^{n-k} = (-1)^n \sum_{k=0}^n (-3)^k = (-1)^n \frac{1 - (-3)^{n+1}}{1 + 3} = \\ &= \frac{(-1)^n + (-1)^n (-1)^n 3^{n+1}}{4} = \frac{(-1)^n + 3^{n+1}}{4}. \end{aligned}$$

С помощью передаточной функции находим:

$$\begin{aligned} Y(z) &= H(z)X(z); X(z) = \frac{z}{z-3}; y(n) = Z^{-1}\left\{\frac{z}{(z+1)} \cdot \frac{z}{(z-3)}\right\} = \\ &= \lim_{z \rightarrow -1} \left(\frac{z^{n+1}}{(z-3)}\right) + \lim_{z \rightarrow 3} \left(\frac{z^{n+1}}{(z+1)}\right) = \frac{(-1)^{n+1}}{-4} + \frac{3^{n+1}}{4} = \frac{(-1)^n + 3^{n+1}}{4}. \blacktriangleright \end{aligned}$$

Структурные схемы фильтров: прямая форма

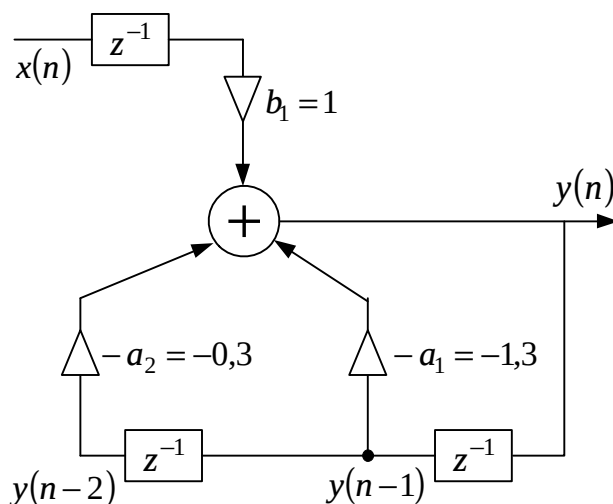


Задание. Изобразить структурную схему для фильтра, имеющего следующую передаточную характеристику: $H(z) = \frac{z^{-1}}{1 + 1,3z^{-1} + 0,3z^{-2}}$.

◀ Соответствующее разностное уравнение имеет вид:

$$y(n) = -1,3y(n-1) - 0,3y(n-2) + x(n-1), \quad b_0 = 1, a_1 = 1,3, a_2 = 0,3. \quad \text{Структурная}$$

схема в прямой форме изображена ниже. ▶

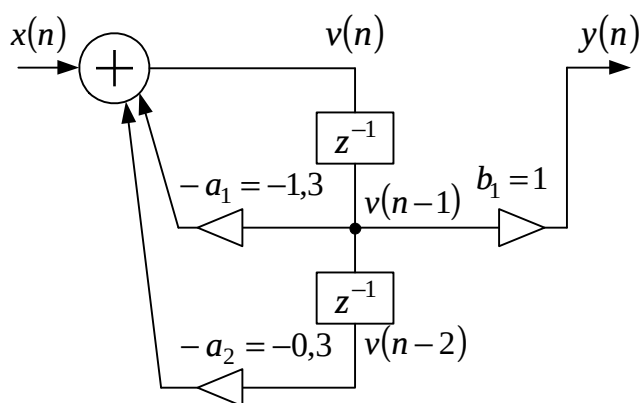


Структурные схемы фильтров: прямая каноническая форма

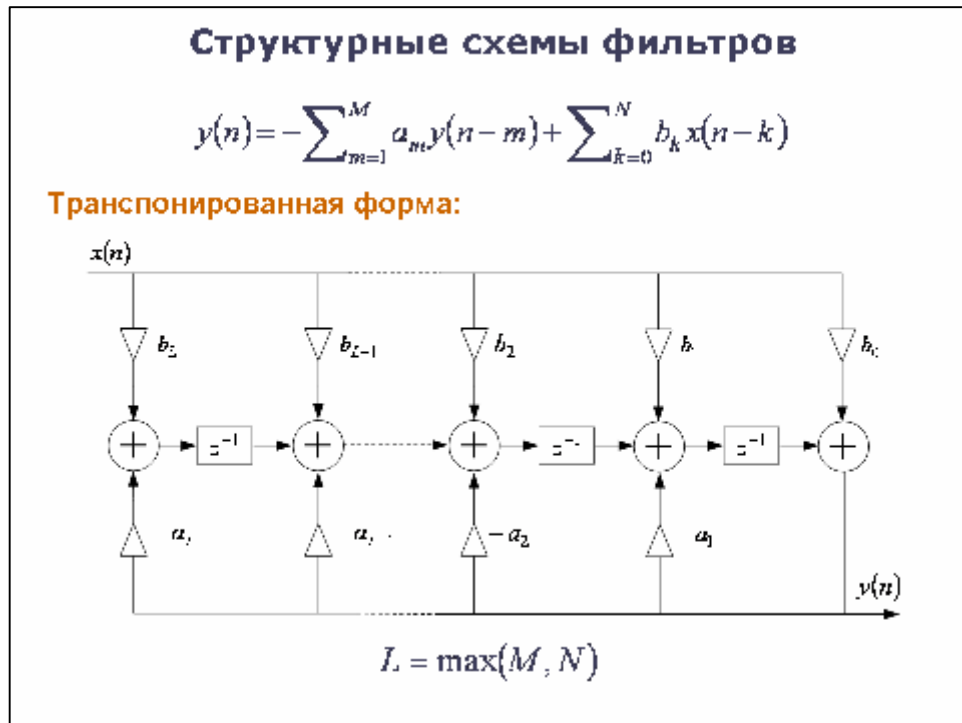


Задание. Представить структурную схему из предыдущего задания в прямой канонической форме.

◀ $b_0 = 1, a_1 = 1,3, a_2 = 0,3$. Структурная схема в прямой канонической форме:



Структурные схемы фильтров: транспонированная форма



Устойчивость ЛДФ

Устойчивость ЛДФ

ЛДФ называется **устойчивым**, если при любой входной последовательности $x(n): \forall n |x(n)| \leq C_1 < \infty$ выходная последовательность также является ограниченной: $\forall n |y(n)| \leq C_2 < \infty$.

Теорема. Необходимым и достаточным условием устойчивости ЛДФ является ограниченность суммы ряда

$$\sum_{n=0}^{\infty} h(n) \leq C < \infty.$$

Следствия:

1. КИХ-фильтры всегда устойчивы.

2. Необходимое условие устойчивости ЛДФ: $\lim_{n \rightarrow \infty} h(n) = 0$.

Теорема. Для того, чтобы ЛДФ был устойчив, необходимо и достаточно, чтобы все полюсы передаточной функции $H(z)$ лежали внутри единичного круга: $|z| < 1$.

Задание. Проверить устойчивость фильтра с импульсной характеристикой $h(n) = (1,5 - n)(-0,6)^n$.

◀ Проверим сходимость ряда $\sum_{n=0}^{\infty} |h(n)|$. Используем признак Даламбера:

$$\lim_{n \rightarrow \infty} \frac{|h(n+1)|}{|h(n)|} = \lim_{n \rightarrow \infty} \frac{|(1,5 - n - 1) \cdot 0,6^{n+1}|}{|(1,5 - n) \cdot 0,6^n|} = 0,6 \lim_{n \rightarrow \infty} \frac{|(0,5 - n)|}{|(1,5 - n)|} = 0,6 < 1 \quad - \quad \text{ряд сходится,}$$

следовательно, фильтр устойчив. ▶

Задание. Проверить устойчивость фильтра: $H(z) = \frac{z^{-1}}{1 - 1,3z^{-1} + 0,3z^{-2}}$.

$$\leftarrow H(z) = \frac{z^{-1}}{1 - 1,3z^{-1} + 0,3z^{-2}} = \frac{z}{z^2 - 1,3z + 0,3} = \frac{z}{(z - 0,3)(z - 1)}.$$
 Передаточная

функция имеет полюс первого порядка $z = 1$, который не лежит внутри единичного круга, следовательно, фильтр неустойчив. ▶

Частотная характеристика ЛДФ

Частотная характеристика ЛДФ

Частотная характеристика (ЧХ) ЛДФ – комплексная функция вещественной переменной ω :

$$K(\omega) = H(z)|_{z=e^{j\omega}} = H(e^{j\omega}) = \sum_{k=0}^{\infty} h(k) e^{j\omega k}$$

При $x(n) = e^{j\omega n}$, $n \geq 0$ имеем: $y(n) \xrightarrow{n \rightarrow \infty} x(n)K(\omega)$.

Частотная характеристика – комплексный коэффициент передачи гармонического колебания.

$K(\omega)$ – **амплитудно-частотная** характеристика

$\varphi(\omega) = -\arg(K(\omega))$ – **фазочастотная** характеристика

Отклик на гармоническое колебание:

$x(n) = A \cos(\omega n - \gamma) = \operatorname{Re}\{A e^{j\omega n}\}$, где $A = |c|^{-1/\gamma}$, тогда

$$y(n) = \operatorname{Re}\{A e^{j\omega n} |K(\omega)| e^{j\varphi(\omega)}\} = |A| \cdot |K(\omega)| \cos(\omega n - \gamma - \varphi(\omega)).$$

Задание: Для фильтра, заданного передаточной функцией $H(z) = \frac{c}{2} + cz^{-1} + \frac{c}{2}z^{-2}$, $c > 0$, найти АЧХ и ФЧХ, определить отклик $y(n)$ в установившемся режиме на входное воздействие $x(n) = \sin\left(\frac{p}{3}n - \frac{p}{6}\right)$.

$$\blacktriangleleft K(\omega) = e^{-j\omega} \left(\frac{c}{2} e^{j\omega} + c + \frac{c}{2} e^{-j\omega} \right) = e^{-j\omega} c(1 + \cos \omega). \quad |K(\omega)| = c(1 + \cos \omega) = 2c \cos^2 \frac{\omega}{2}.$$

На рисунке приведен график при $c=1$.

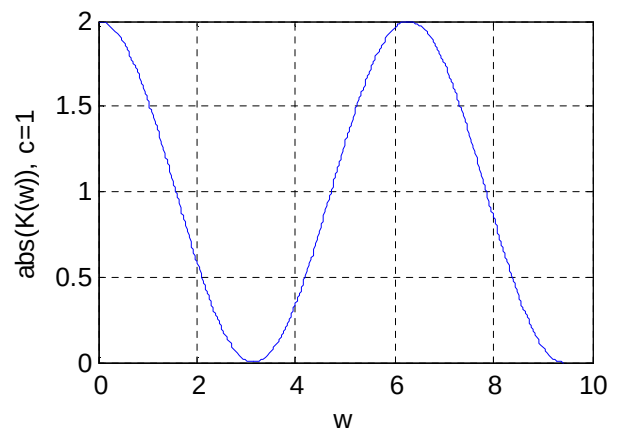
$j(\omega) = -\arg K(\omega) = \arg(e^{j\omega})$. (Учесть, что $j(\omega) \in (-p; p]$).

В установившемся режиме отклик фильтра на гармоническое входное

воздействие $x(n) = \operatorname{Im}\left\{ e^{j\left(\frac{p}{3}n - \frac{p}{6}\right)} \right\}$ с

циклической частотой $\omega = \frac{p}{3}$ находим

как



$$y(n) = \operatorname{Im} \left(K \left(\frac{p}{3} \right) e^{i \left(\frac{p}{3} n - \frac{p}{6} \right)} \right) = c \left(1 + \cos \frac{p}{3} \right) \sin \left(\frac{p}{3} n - \frac{p}{6} - \frac{p}{3} \right) =$$

$$= \frac{3}{2} c \sin \left(\frac{p}{3} n - \frac{p}{2} \right) = -\frac{3}{2} c \cos \left(\frac{p}{3} n \right) \blacktriangleright$$

Задание. Найти АЧХ и ФЧХ фильтра $y(n) = ay(n-1) + x(n)$.

$$\blacktriangleleft \quad b_0 = 1, \quad a_1 = -a, \quad H(z) = \frac{1}{1 - az^{-1}} = \frac{z}{z - a}. \quad K(w) = H(e^{-iw}) = \frac{1}{1 - ae^{-iw}}.$$

$$|K(w)| = \frac{1}{|1 - ae^{-iw}|} = \frac{1}{\sqrt{(1 - a \cos w)^2 + (a \sin w)^2}} = \frac{1}{\sqrt{1 + a^2 - 2a \cos w}},$$

$$j(w) = -\arg(K(w)) = \operatorname{arctg} \frac{a \sin w}{1 - a \cos w}. \blacktriangleright$$

Задание. Построить графики АЧХ и ФЧХ для фильтра из предыдущего задания при $a = \frac{1}{\sqrt{3}}$ и $a = -\frac{1}{\sqrt{3}}$. Полученные фильтры являются фильтрами верхних или нижних частот?

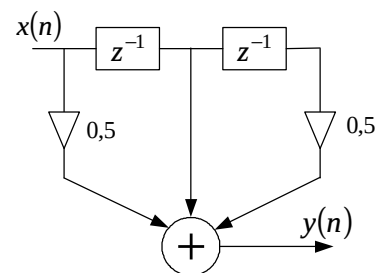
◀ Используем функцию freqz:

```
[H1, w1] = freqz([1], [1 -1/sqrt(3)]); % a = -1/sqrt(3)
[H2, w2] = freqz([1], [1 1/sqrt(3)]); % a = 1/sqrt(3)
plot(w1, abs(H1), 'r', w2, abs(H2), 'b'), grid;
legend('a = -1/sqrt(3)', 'a = 1/sqrt(3)');
```

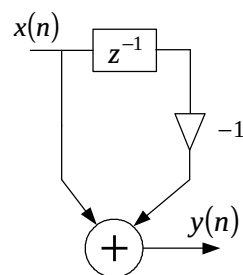
При $a = \frac{1}{\sqrt{3}}$ имеем фильтр верхних частот, при $a = -\frac{1}{\sqrt{3}}$ – нижних. ▶

Домашнее задание

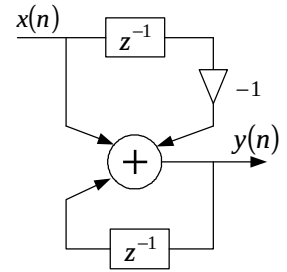
1. Найдите АЧХ и ФЧХ фильтра и постройте эскизы их графиков.



2. Найдите АЧХ и ФЧХ фильтра и постройте эскизы их графиков.



3. Найдите АЧХ и ФЧХ фильтра и постройте эскизы их графиков.



4. По заданной импульсной характеристике $h(n) = 3n + 1$ синтезируйте прямую, прямую каноническую и транспонированную формы фильтра.
5. По заданной импульсной характеристике $h(n) = 2(1 - n)$ синтезируйте прямую, прямую каноническую и транспонированную формы фильтра.
6. Найдите отклик фильтра с передаточной функцией $H(z) = \frac{1}{4z^2 - 4z + 1}$ на входное воздействие $x(n) = (-1)^n$.
7. Найдите отклик фильтра с передаточной функцией $H(z) = \frac{1}{(z-1)(z-0,5)}$ на входное воздействие $x(n) = 0,5^n$.
8. Определите, является ли устойчивым фильтр с импульсной характеристикой
- $$h(n) = \begin{cases} a^n, & n \geq 0, \\ 0, & n < 0. \end{cases}$$
9. Определите, является ли устойчивым фильтр с передаточной функцией

$$H(z) = \frac{1 + z^{-1}}{\left(1 - \frac{1}{2}z^{-1}\right)\left(1 + \frac{1}{4}z^{-1}\right)} \quad ? \text{ Найдите импульсную характеристику фильтра.}$$

Семинар №6. Источник дискретных сообщений без памяти. Энтропия. Эффективное кодирование

Понятие дискретного источника без памяти

Источник без памяти

Дискретный источник сообщений без памяти X :

- конечное число состояний $x_1, \dots, x_N$;
- заданы вероятности $p_k = P\{X = x_k\}$ перехода в состояния x_k ($k = 1, \dots, N$): $\sum_{k=1}^N p_k = 1$;
- случайные переходы источника X из одного состояния в другое не зависят от текущего и предыдущих состояний («нет памяти»).

Состояние	x_1	x_2	...	x_N
Вероятность	p_1	p_2	...	p_N

Пример источника без памяти:

Состояние	x_1	x_2	x_3	x_4
Вероятность	0,5	0,25	0,125	0,125
Код	0	10	110	111

Префиксный код –
короткие битовые слова не являются началом более длинных

Задание. На слайде приведен пример дискретного источника без памяти, имеющего 4 состояния $x_1, \mathbf{K}, x_4$, для которых заданы вероятности $p_1, \mathbf{K}, p_4$. Каждому состоянию поставлен в соответствие двоичный код. Определить среднее число бит R , необходимое для кодирования одного состояния (символа).

$$\blacktriangleleft R = \sum_{k=1}^4 p_k R_k = 0,5 \cdot 1 + 0,25 \cdot 2 + 0,125 \cdot 3 + 0,125 \cdot 3 = 1,75 \text{ (бит)}. \blacktriangleright$$

Энтропия дискретного источника

Энтропия

Энтропия – мера количества информации (априорной неопределённости); для дискретного источника сообщений без памяти:

$$H(X) = -\sum_{k=1}^N p_k \log p_k.$$

$H(X) = -\sum_{k=1}^N p_k \log_2 p_k$ – количество информации **в битах**.

Теорема Шэннона

При бесконечной длительности сообщение источника без памяти X с двоичной энтропией $H(X)$ можно закодировать так, что средние битовые затраты R , приходящиеся на один символ сообщения, $\forall \varepsilon > 0$ удовлетворяют неравенствам $H(X) \leq R < H(X) + \varepsilon$.

Идея алгоритмов эффективного кодирования – сделать одинаковыми вероятности появления в коде «0» и «1».

Задание. Определить двоичную энтропию для дискретного источника из предыдущего примера.

◀ Средние битовые затраты совпадают с двоичной энтропией:

$$H(X) = -\sum_{k=1}^4 p_k \log_2 p_k = -(0,5 \cdot (-1) + 0,25 \cdot (-2) + 0,125 \cdot (-3) + 0,125 \cdot (-3)) = 1,75$$

(бит). ►

Эффективное кодирование дискретного источника без памяти по методу Шэннона-Фано



1. Упорядочиваем таблицу символов (состояний источника) $\{x_1, \dots, x_N\}$ в порядке невозрастания вероятностей: $p_1 \geq \dots \geq p_N$. Полученная таблица является первым блоком символов для последующего разбиения.
2. Разбиваем текущий блок символов на две части (два подблока) так, чтобы вероятности попадания символа сообщения в подблоки были как можно ближе друг к другу.
3. Приписываем очередной бит в коды символов подблоков: «0» для первого подблока и «1» для второго подблока. Далее подблоки рассматриваем независимо друг от друга как новые блоки в разбиении таблицы символов.
4. Если в разбиении таблицы символов присутствует блок, содержащий более одного символа, то повторяем с этим блоком шаги 2 и 3, иначе заканчиваем процедуру построения двоичных кодов.

Задание. Построить эффективные коды для источника сообщений Z со следующим распределением вероятностей появления символов:

z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8
0,22	0,20	0,16	0,16	0,10	0,10	0,04	0,02

◀ Рассмотрим следующее разбиение таблицы состояний:

z_1 0,22	z_2 0,20	z_3 0,16	z_4 0,16	z_5 0,10	z_6 0,10	z_7 0,04	z_8 0,02
0			1				
00	01		10		11		
	010	011	100	101	110	111	
						1110	1111

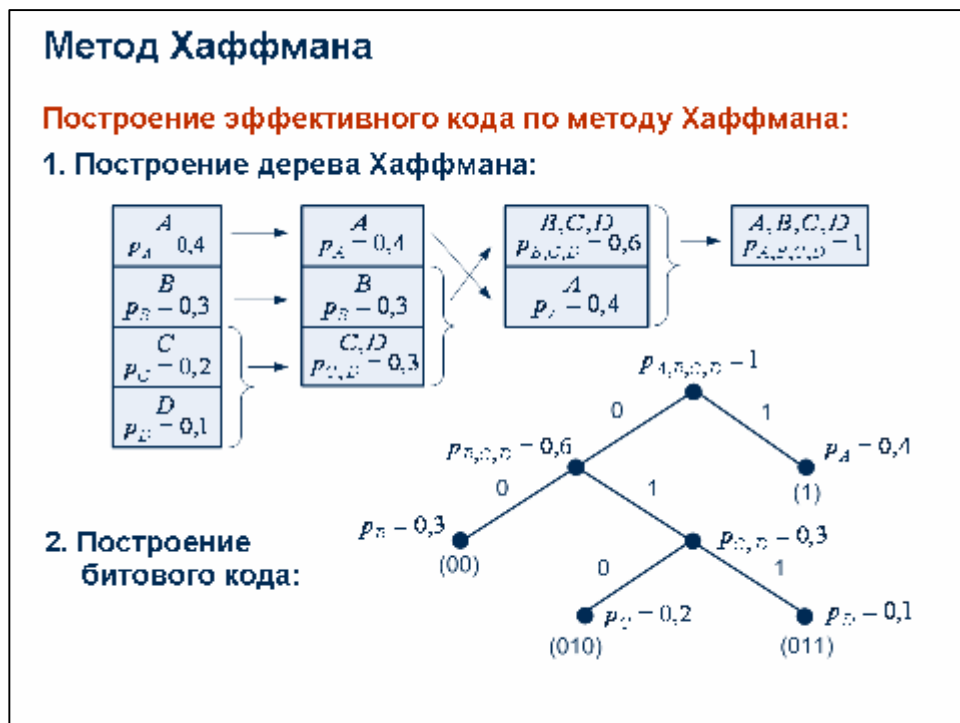
При этом средние битовые затраты $R = 2,84$ бита, а двоичная энтропия источника $H(Z) \approx 2,76$ бит. Таким образом, избыточность кода составит $R - H(Z) \approx 0,08$ бит.

Недостатком метода Шэннона-Фано является его *неоднозначность*. Так, в рассмотренном примере разбиение таблицы можно также выполнить следующим образом:

z_1 0,22	z_2 0,20	z_3 0,16	z_4 0,16	z_5 0,10	z_6 0,10	z_7 0,04	z_8 0,02
0		1					
00	01	10		11			
		100	101	110	111		
					1110	1111	
						11110	11111

При таком разбиении среднее количество бит, необходимое для кодирования символа, будет равно $R = 2,8$ бит. Избыточность кода $R - H(Z) \approx 0,04$ бит – вдвое меньше, чем в предыдущем случае. ►

Метод построения эффективных кодов по Хаффману



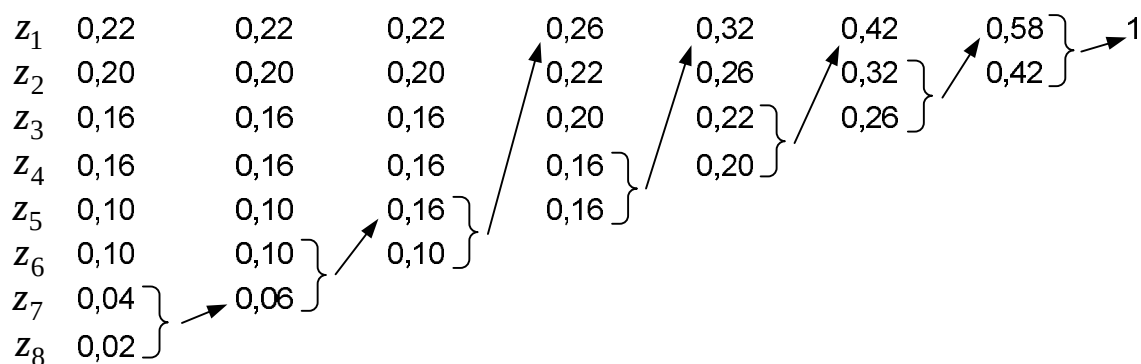
1. Построение дерева Хаффмана.
 - 1.1. Упорядочиваем текущую таблицу символов в порядке невозрастания вероятностей: $p_1 \geq \dots \geq p_N$.
 - 1.2. Два последних символа, имеющих наименьшие вероятности появления, объединяем в один новый символ, которому приписываем суммарную вероятность объединенных символов. Если в полученном алфавите имеется более одного символа (его вероятность равна 1), то переходим на шаг 1.1, иначе – на шаг 2.

Комментарий. Вершина дерева Хаффмана – это единственный символ окончательного алфавита, листья – символы исходного алфавита, прочие узлы – символы промежуточных алфавитов, полученные в результате слияния других символов.

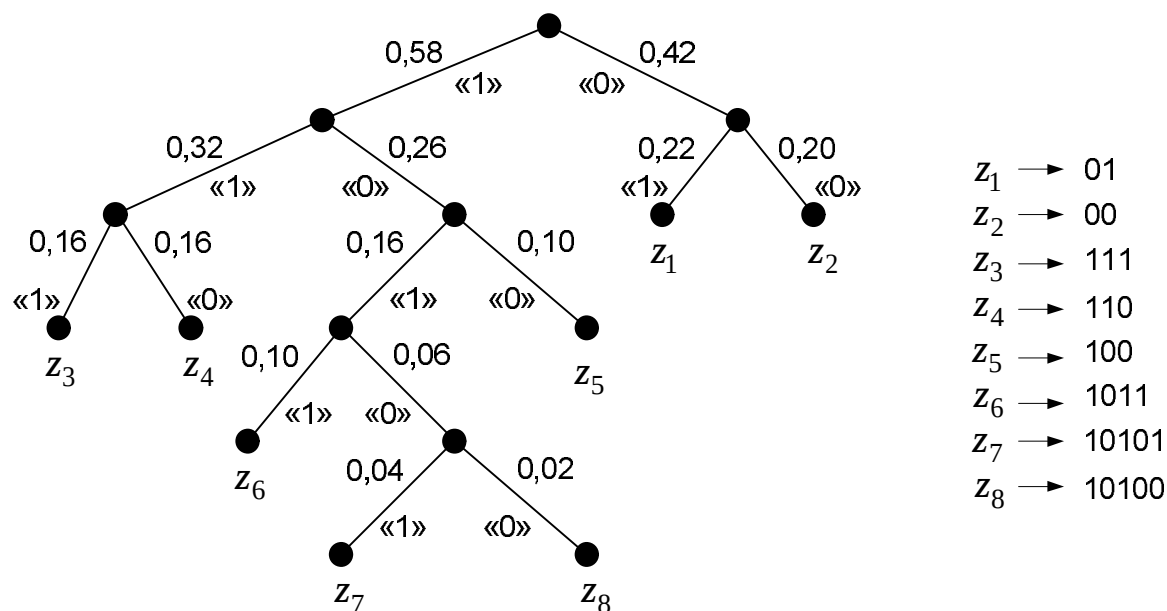
2. Построение битового кода. Из каждого узла дерева Хаффмана (за исключением листьев) строим по два ребра; приписываем одному ребру бит «0», другому – «1». Код каждого символа исходного алфавита получается в результате последовательного дописывания битов, соответствующих ребрам графа, которые необходимо пройти от вершины дерева до листа-символа.

Задание. Для источника из предыдущего примера построить код Хаффмана.

◀ Построение дерева Хаффмана:



Построение битового кода:



Среднее количество бит $R = 2,8$, избыточность $R - H(z) \approx 0,04$ бит. ►

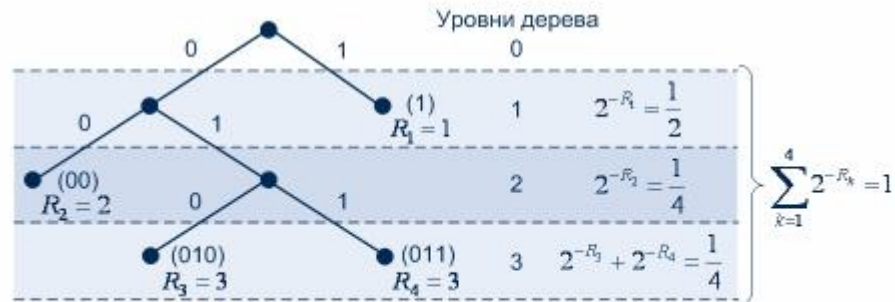
Замечание. Рассмотренные методы показывают, что префиксные коды источника можно представить в виде графа-дерева.

Неравенство Крафта

Неравенство Крафта

Лемма (неравенство Крафта)

Для существования однозначно декодируемого двоичного кода $\{(x_k, R_k)\}_{k=1}^N$, содержащего N кодовых слов с длинами $R_1, \dots, R_N$, необходимо и достаточно, чтобы выполнялось неравенство Крафта: $\sum_{k=1}^N 2^{-R_k} \leq 1$.



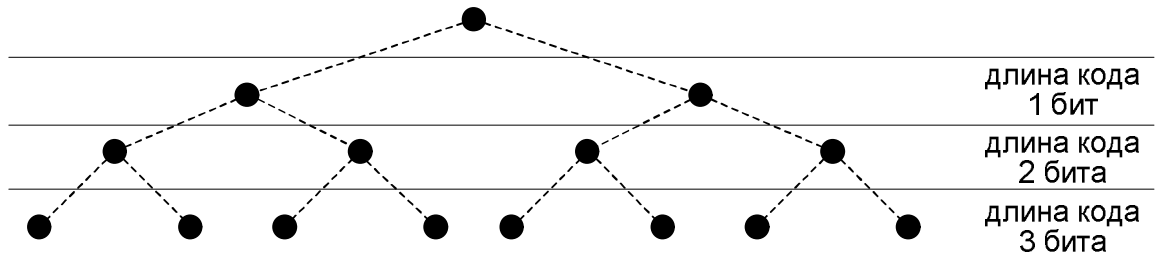
Задание. Проверить выполнение неравенства Крафта для кода из предыдущего примера.

$$\blacktriangleleft 2 \cdot 2^{-2} + 3 \cdot 2^{-3} + 2^{-4} + 2 \cdot 2^{-5} = 1. \blacktriangleright$$

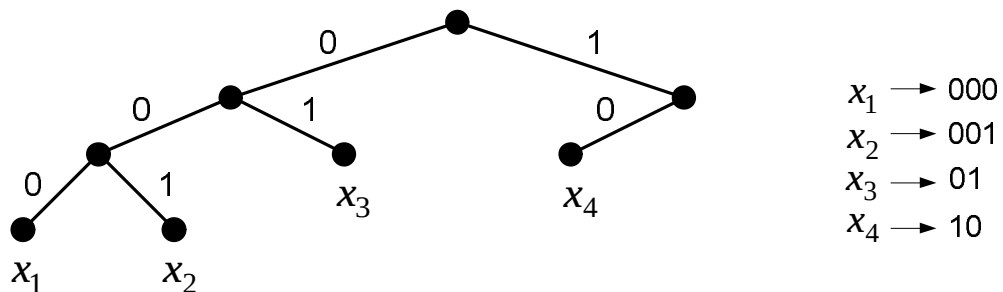
Задание. Для источника с 4-мя состояниями заданы длины кодовых слов: $R_1 = R_2 = 3$ (бит), $R_3 = R_4 = 2$ (бит). Проверить, выполняется ли неравенство Крафта, и если да, построить битовый код.

$$\blacktriangleleft 2 \cdot 2^{-2} + 2 \cdot 2^{-3} = \frac{1}{2} + \frac{1}{4} = \frac{3}{4} < 1 \quad - \quad \text{неравенство Крафта выполняется,}$$

следовательно, код существует. Построим код по каркасу (дереву). Так как максимальная длина кода по условию составляет 3 бита, дерево будет иметь 3 уровня (не считая 0-го):



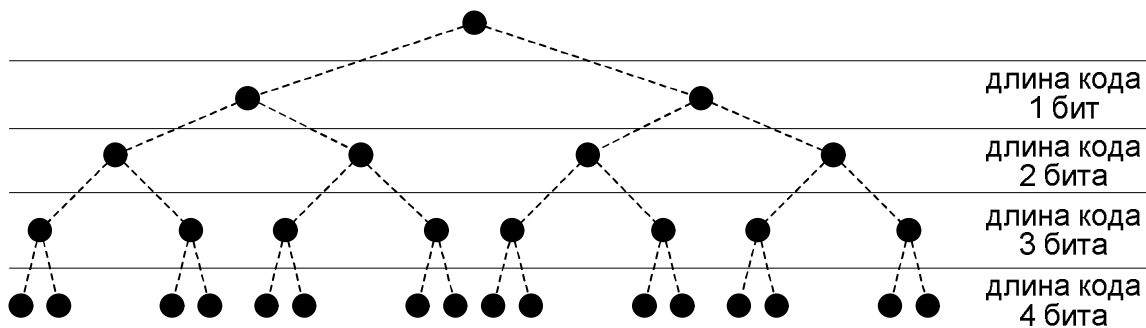
Будем заполнять граф слева направо, начиная с нижнего уровня. На нижнем уровне располагаются два символа: x_1 и x_2 . На следующем уровне свободные вершины занимаем символами x_3 и x_4 . В итоге получаем код:



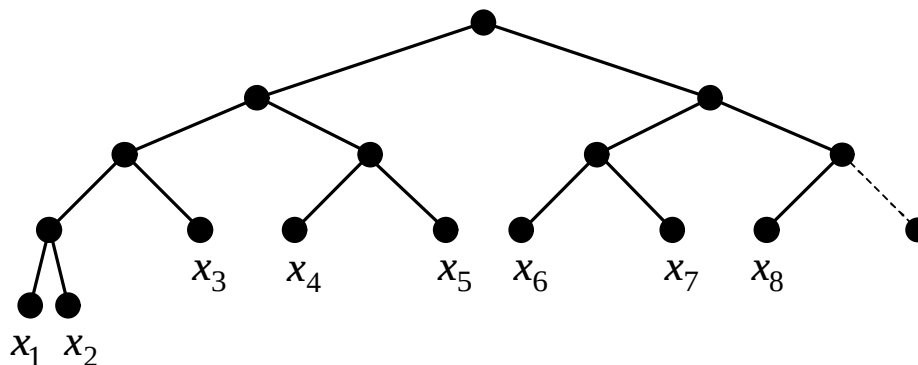
Видно, что данный код не является оптимальным, так как символ x_4 можно было бы закодировать, используя лишь один бит (1), вместо двух (10). В таком случае неравенство Крафта превратилось бы в равенство. ►

Задание. Построить префиксный код для источника с 9-ю состояниями, при заданных длинах кодовых слов: $R_1 = R_2 = 4$, $R_3 = R_4 = \dots = R_7 = R_8 = 3$, $R_9 = 2$.

◀ Как и в предыдущем примере, построим кодовое дерево. Так как максимальная длина кода по условию составляет 4 бита, потребуется дерево (каркас) с 4-мя уровнями (не считая 0-го):



На самом нижнем уровне находятся 2 символа x_1 и x_2 , которые имеют код длиной 4 бита. На следующем уровне занимаем слева направо свободные вершины символами $x_3, \dots, x_8$.



Теперь нужно перейти ещё на один уровень выше и найти свободную вершину для x_9 . Но такой вершины нет! Следовательно, код построить невозможно. Действительно, проверим выполнение неравенства Крафта:

$$2 \cdot 2^{-4} + 6 \cdot 2^{-3} + 1 \cdot 2^{-2} = 1,125 > 1. \blacktriangleright$$

Задание. Построить эффективный код для источника

z_1	z_2
0,9	0,1

◀ Энтропия источника $H = -0,1 \cdot \log_2 0,1 - 0,9 \cdot \log_2 0,9 \approx 0,47$ бит. При независимом кодировании символов средняя длина кода составляет 1 бит, что дает очень большую избыточность кода. Чтобы повысить эффективность кодирования, объединим символы в пары:

(z_1z_1)	(z_1z_2)	(z_2z_1)	(z_2z_2)
0,81	0,09	0,09	0,01

Построим код по методу Шэннона-Фано:

(z_1z_1)	(z_1z_2)	(z_2z_1)	(z_2z_2)
0,81	0,09	0,09	0,01
0	1		
	10	11	
		110	111

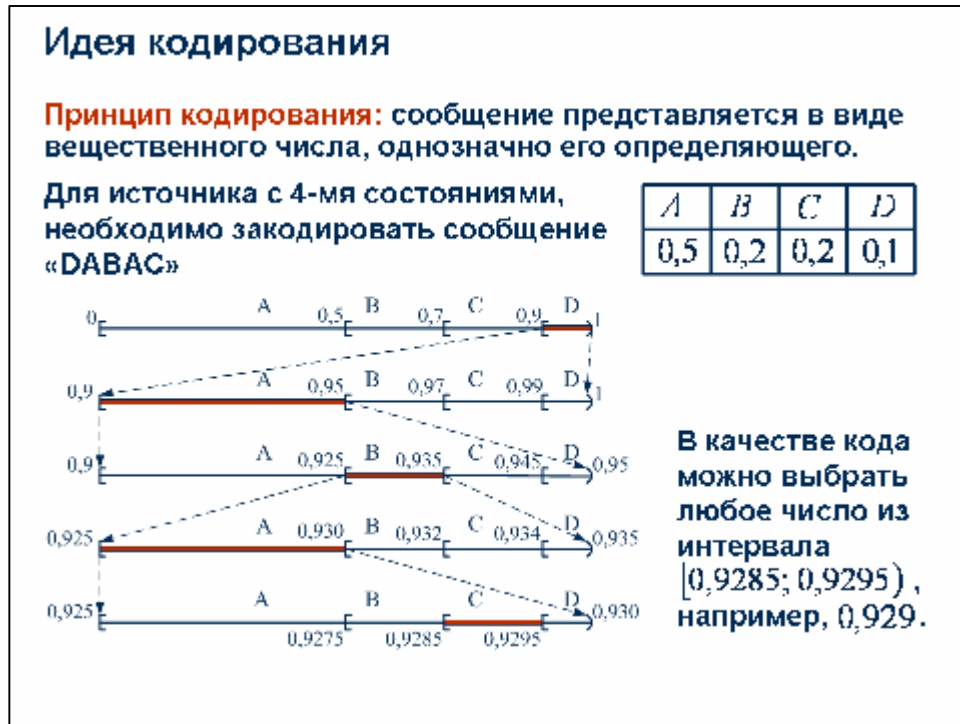
Получили среднюю длину кода для пары символов $2R = 0,81 \cdot 1 + 0,09 \cdot 2 + 0,09 \cdot 3 + 0,01 \cdot 3 = 1,29$ бит, при этом на один символ приходится $R = 0,645$ бит, т.е. объединение в двойки позволило существенно уменьшить избыточность кода. ►

Домашнее задание

1. Построить эффективный код для источника с равновероятными состояниями $p_1 = \dots = p_5 = 0,2$ по методам Шэннона-Фано и Хаффмана. Оценить избыточность полученного кода, т.е. сравнить средний битовый код с энтропией.
2. Для источника с 9-ю состояниями заданы длины кодовых слов в битах: $R_1 = R_2 = \dots = R_6 = 4$, $R_7 = 3$, $R_8 = R_9 = 2$. Проверить, выполняется ли неравенство Крафта, и если да, построить соответствующий битовый код.
3. Оценить эффективность кодирования символов источника с вероятностями $p_1 = 0,9$ и $p_2 = 0,1$, объединенных в тройки, по сравнению с рассмотренным выше кодированием пар.

Семинар №7. Арифметическое кодирование. Кодирование длин серий

Принцип арифметического кодирования



Метод арифметического кодирования был предложен в 70-х годах прошлого века и представляет собой альтернативу методам Шэннона-Фано и Хаффмана. Суть его состоит в том, что последовательности символов $\{x(1), x(2), \dots, x(M)\}$, созданных дискретным источником сообщений X с известным алфавитом и вероятностями появления символов $\{(x_k, p_k)\}_{k=1}^N$, ставится в соответствие некоторое число, однозначно задающее данную последовательность.

Алгоритм кодирования

Арифметическое кодирование

Задача – закодировать сообщение $\{x(1), \dots, x(M)\}$ источника с алфавитом $\{x_k\}_{k=1}^N$, для которого задано разбиение на вероятностные интервалы $\Lambda_k = [L_{x_k}; U_{x_k})$.

Шаг 1. Установить границы текущего интервала $L=0, U=1$ и счетчик символов $j=1$.

Шаг 2. Определить новые границы текущего интервала:

$$\begin{aligned} W &= U - L, \\ U &= L + W \times U_{x(j)}, \\ L &= L + W \times L_{x(j)}. \end{aligned}$$

Шаг 3. Увеличить счетчик символов $j = j + 1$. Если $j > M$, то перейти на шаг 4, иначе – на шаг 2.

Шаг 4. Выбрать число $B \in [L; U)$, содержащее наименьшее число цифр.

Задание. Для следующего источника

z_1	z_2
0,9	0,1

построить выходной битовый код для всех возможных пар символов, сравнить средние битовые затраты с энтропией. Проверить правильность кодирования, используя программу task1.m:

```
function task1
% арифметическое кодирование
P = [0.9 0.1]; % вероятности
S = [1 1]; % сообщение
L_cur = 0;
U_cur = 1;
L = cumsum([0 P(1:end-1)]);
U = cumsum(P);
for i = 1:length(S)
    W = U_cur - L_cur;
    U_cur = L_cur + W * U(S(i));
    L_cur = L_cur + W * L(S(i));
end
L_cur_bin = dec2bin(L_cur * 2^10)
U_cur_bin = dec2bin(U_cur * 2^10)
```

◀ Энтропия источника $H = -0,1 \cdot \log_2 0,1 - 0,9 \cdot \log_2 0,9 \approx 0,47$ бит.

Проведем кодирование пар символов. В результате получим следующие интервалы:

AA: $[0; 0,81) = [0,000...; 0,1100111101...)$, кодовое число – 0,1 (1 бит);

AB: $[0,81; 0,9) = [0,1100111101...; 0,1110011001...)$, кодовое число – 0,111 (3 бита);

BA: $[0,9; 0,99) = [0,1110011001...; 0,1111110101...)$, кодовое число – 0,1111 (4 бита);

BB: $[0,99; 1) = [0,1111110101...; 0,1111111111...)$, кодовое число – 0,1111111 (7 бит).

Определим средние битовые затраты:

$$R = \frac{1 \cdot 0,81 + 3 \cdot 0,09 + 4 \cdot 0,09 + 7 \cdot 0,01}{2} = 0,755. \text{ Получили значение больше энтропии,}$$

что не противоречит теории. ►

Выбор кодового числа

Выбор кодового числа

После окончания кодирования границы интервала L и U можно представить в виде

$$U = 0, b_1 b_2 \dots b_K 1 u_{K+2} u_{K+3} \dots_{(2)} = \sum_{k=1}^K b_k 2^{-k} + \frac{1}{2^{K+1}} + \sum_{k=K+2}^{\infty} u_k 2^{-k},$$

$$L = 0, b_1 b_2 \dots b_K 0 l_{K+2} l_{K+3} \dots_{(2)} = \sum_{k=1}^K b_k 2^{-k} + \sum_{k=K+2}^{\infty} l_k 2^{-k},$$

где $b_1, b_2, \dots, b_K$ – совпавшие биты в двоичном представлении чисел L и U .

Не все $u_k, k \geq K+2$ равны 0, и не все $l_k, k \geq K+2$ равны 1. Поэтому в качестве кодового числа можно использовать $B = b_1 b_2 \dots b_K 1$. Последний бит – всегда 1, следовательно, его не нужно хранить!

Из-за сближения в процессе кодирования верхней U и нижней L границ интервала, после окончания процедуры арифметического кодирования M символов первые биты в двоичной записи чисел $L = 0, l_1 l_2 \dots$ и $U = 0, u_1 u_2 \dots$ будут совпадать. Пусть количество первых совпавших разрядов $\{b_j\}$ равно K : $u_j = l_j = b_j, j = 1, \dots, K$. Тогда, поскольку $L < U$, первые несовпадающие биты: $l_{K+1} = 0, u_{K+1} = 1$, а сами числа L и U можно представить в виде, указанном на слайде.

Будем считать, что не все u_k равны 0 при $k \geq K+2$. Действительно, если $U = 0, b_1 b_2 \dots b_K 100\dots$, то можно записать $U = 0, b_1 b_2 \dots b_K 01111\dots$. Тогда в качестве

кодowego числа можно всегда выбирать $B = 0, b_1 b_2 \dots b_K 1$. Поскольку число B всегда заканчивается единичным битом, этот бит можно не хранить и передавать лишь $b_1 b_2 \dots b_K$ (декодер будет приписывать дополнительный единичный бит автоматически).

Принимая во внимание вышесказанное, определим средние битовые затраты для кодирования пары символов из предыдущего примера:

AA – кодовое число 0,1 (0 бит);

AB – кодовое число 0,111 (2 бита);

BA – кодовое число 0,1111 (3 бита);

BB – кодовое число 0,111111 (6 бит).

Следовательно, $R = \frac{0 \cdot 0,81 + 2 \cdot 0,09 + 3 \cdot 0,09 + 6 \cdot 0,01}{2} = 0,2550$. Получили, что

битовые затраты оказались меньше энтропии! Чтобы понять, как такое стало возможным, рассмотрим процесс декодирования сообщения по числу B .

Алгоритм декодирования

Алгоритм декодирования

Задача – декодировать сообщение из M символов.

Шаг 1. Ввести кодовое число B и установить счетчик символов $j = 0$.

Шаг 2. Найти интервал Δ_k , в который попадает число B .
Выдать символ $x(j) = x_k$.

Шаг 3. Привести текущий интервал $[L_{x_j}; U_{x_j})$ к $[0; 1)$:

$B' = (B - L_{x_j}) / (U_{x_j} - L_{x_j})$

Шаг 4. Увеличить счетчик символов $j = j + 1$. Если $j \leq M$, то перейти на шаг 2, иначе закончить работу.

Задание. Для источника со статистикой

z_1	z_2
0,9	0,1

декодировать сообщение из $M = 6$ символов по кодовому числу 0,111 (оно кодируется 2-мя битами), используя программу task2.m:

```

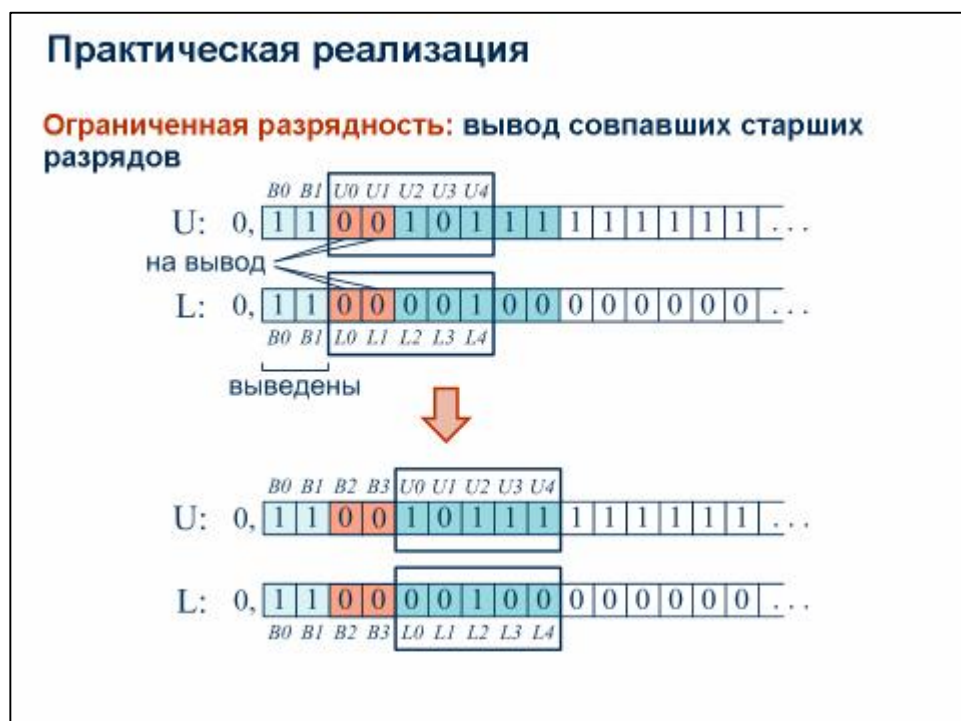
function task2
% арифметическое декодирование
P = [0.9 0.1]; % вероятности
M = 6; % количество символов в сообщении
B = 0.8750; % B = 0.111
L = cumsum([0 P(1:end-1)]);
U = cumsum(P);
S = zeros(1, M);
for i = 1:M
    S(i) = max(find(L <= B));
    B = (B - L(S(i))) / (U(S(i)) - L(S(i)));
end
S

```

◀ Чтобы не проводить вычисления вручную, воспользуемся программой task2.m. Для указанного кодового числа получим сообщение **ABAAAAB**. Но в предыдущем примере кодовое число 0,111 соответствовало последовательности **AB**! Более того, можно было бы задать $M > 6$, т.е. дальше продолжить декодирование (например, можно запустить программу для $M = 10$).

Таким образом, одного кодового числа недостаточно для того, чтобы однозначно определить сообщение – декодер просто не будет знать, когда ему остановиться. Поэтому на практике используют специальный символ «конец сообщения», или в качестве дополнительной информации передают длину сообщения M . При этом битовые затраты получаются близкими к энтропии, но не меньше её. ►

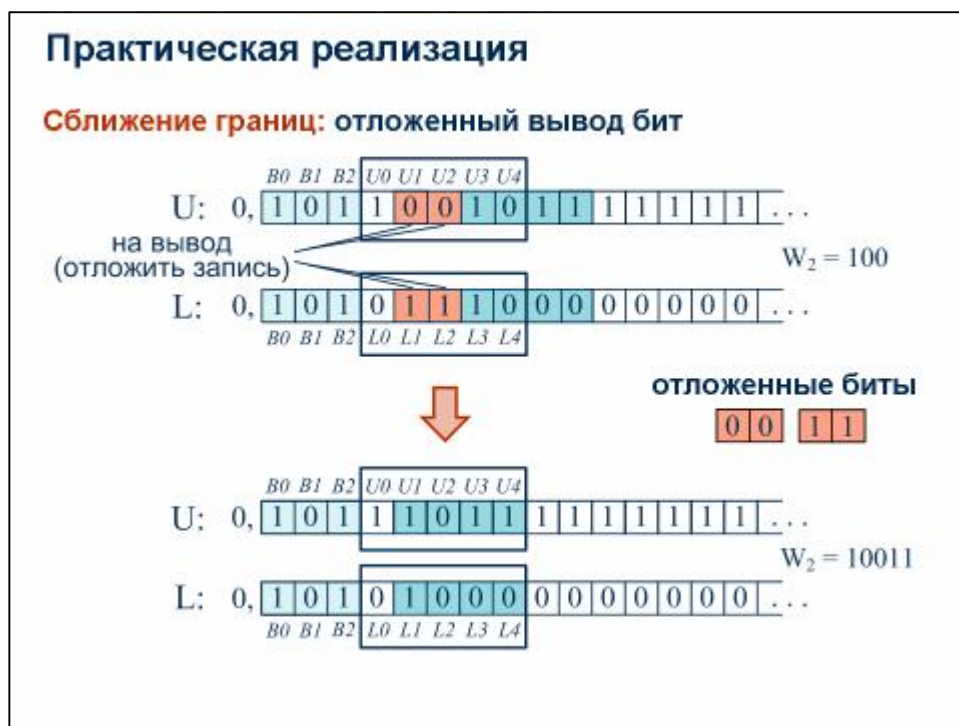
Проблемы при практической реализации арифметического кодера



Арифметический кодер реализуют с использованием чисел с фиксированной точкой, представляемых в памяти целыми числами, количество двоичных разрядов в которых может быть сколь угодно большим. При этом для представления значений L и R используются регистры конечной разрядности, которой хватает только для кодирования очень коротких сообщений.

В начале работы кодера нижняя граница интервала представляется в виде дроби с бесконечным числом нулей $L_0 = 0 = 0,0000\dots$, а верхняя – в виде дроби с бесконечным числом единиц $U_0 = 1 = 0,1111\dots$ (все вычисления проводятся в двоичной системе).

После того, как в процессе работы кодера несколько старших разрядов регистров U и L совпали, они уже не могут измениться и не участвуют в дальнейшей работе кодера. Следовательно, они могут быть выдвинуты из регистров и записаны в выходной поток, а в освободившиеся младшие разряды для L попадут нули, а для U – единицы.



Проблема возникает, если в процессе кодирования верхняя и нижняя границы интервала отличаются в старших разрядах, но длина интервала недопустимо мала. В этом случае слишком маленькая длина интервала может не позволить закодировать очередной символ, так как при выполнении целочисленной нормализации значения регистров U и L не изменятся и алгоритм заикнется. С этой проблемой можно бороться, отслеживая сближение верхней и нижней границы. Если длина интервала $W = U - L$ на очередной итерации стала недопустимо малой, то выполняется принудительное расширение интервала с помощью выдвигания из регистров одного или нескольких старших разрядов (за исключением самого старшего), ещё не готовых к выводу в выходной поток.

При этом сама запись выдвинутых разрядов в поток откладывается, так как их окончательные значения ещё неизвестны и станут известны только после того, как совпадут старшие разряды регистров, не участвовавшие в выдвигании.

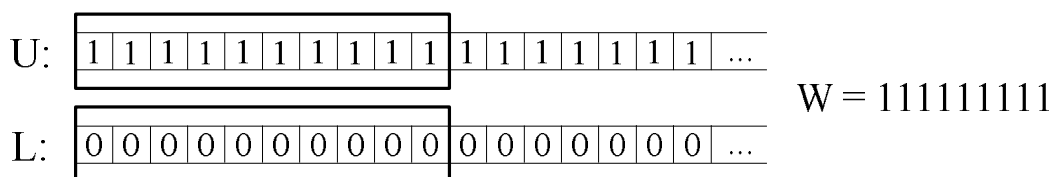
Задание. Для источника со статистикой

A	B	C	D
0,6	0,2	0,1	0,1

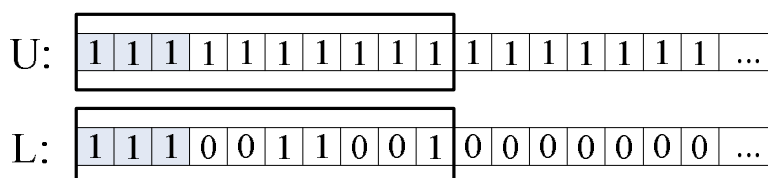
закодировать сообщение «DABAC», используя 10-разрядные регистры.

◀ Рассмотрим процесс кодирования.

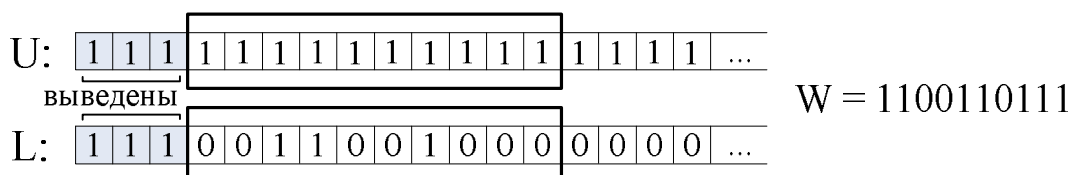
Начальные значения регистров:



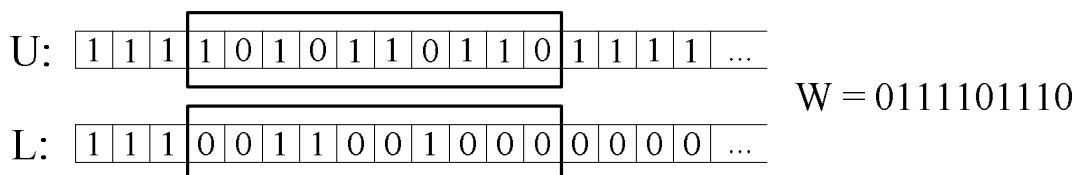
Кодируем символ «D»:



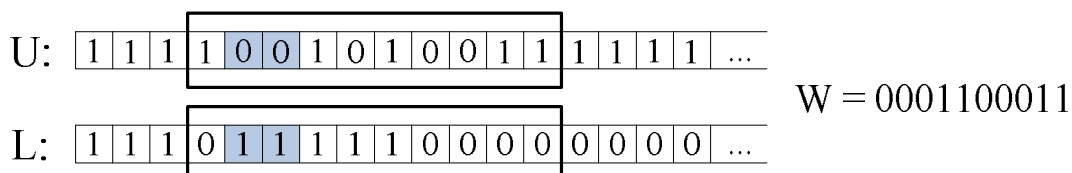
Совпали три старших разряда, их можно записать в выходной поток и выдвинуть из регистров. При этом получим:



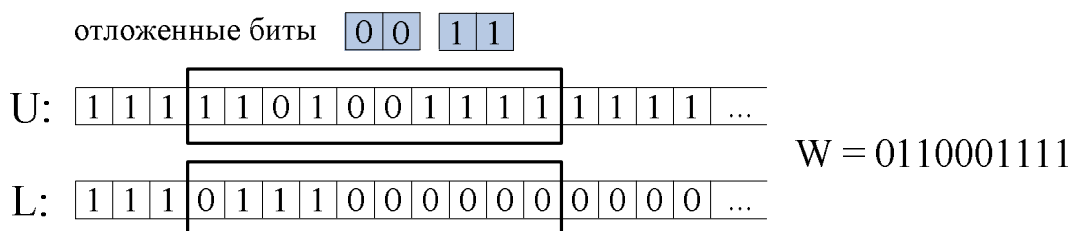
Кодируем символ «A»:



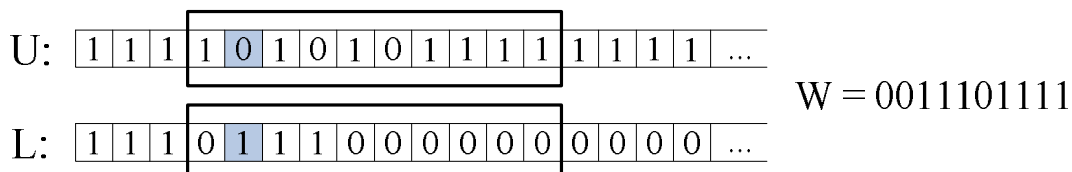
Кодируем символ «B»:



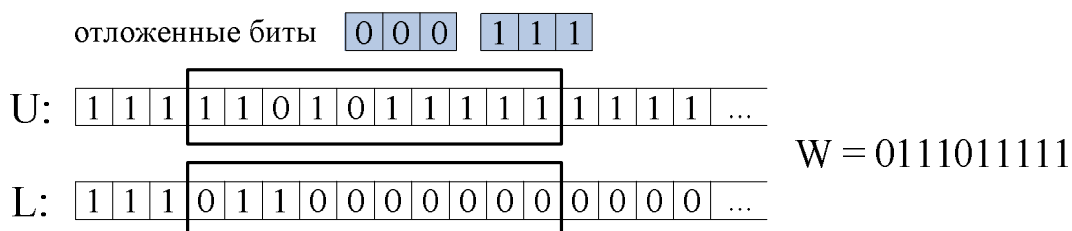
Для новых границ получили малую длину интервала, поэтому производим отложенный вывод двух бит, при этом границы изменятся следующим образом:



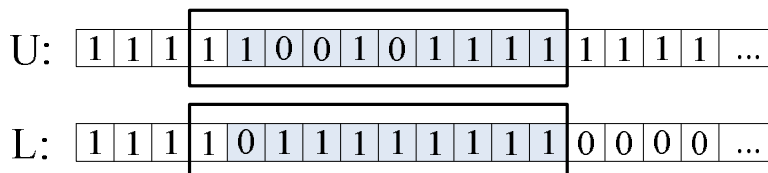
Кодируем символ «А»:



Границы опасно сблизились, поэтому откладываем один бит:



Кодируем символ «С»:

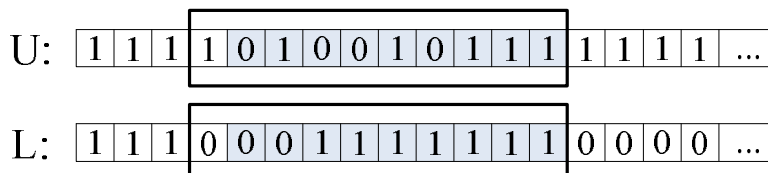


Старшие разряды совпали и равны 1, тогда отложенные биты равны 0, в итоге получаем следующие границы интервала:

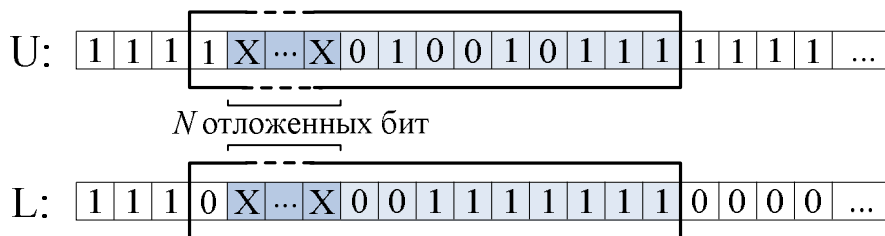


В качестве битового кода выбираем 11110001, что соответствует кодовому числу 0,94141. ►

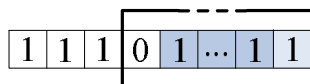
Замечание. В приведенном выше примере при кодировании последнего символа старшие биты совпали, что позволило определить, чему равны отложенные биты и вывести их в выходной поток. Рассмотрим случай, когда после завершения кодирования сообщения старшие биты в **U** и **L** не совпадают, причем некоторое число *N* бит уже было отложено и регистры имеют вид:



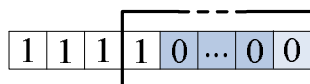
С учетом отложенных бит:



Если значение регистра $L < 0X...X1000000...$ (как в приведенном примере), то в качестве кодового можно выбрать следующее число:



Здесь старший разряд регистра равен 0, за ним следуют $N+1$ единичных бит. В случае, когда $L \geq 0X...X1000000...$, выбираем кодовое число, в котором старший бит равен 1, и к нему дописываются $N+1$ нулевых бит:



Кодирование длин серий

Кодирование длин серий

Кодирование длин серий (КДС) эффективно, когда одно из состояний источника имеет вероятность, близкую к 1 (энтропия источника мала).

Источник с двумя состояниями, $p > q$.

Рассматриваем $M = 2^b$ блоков:

z_1	z_2
p	$q = 1 - p$

Блок символов	Количество символов в блоке	Вероятность появления блока
$S_0 = z_2$	1	$1 - p$
$S_1 = z_1 z_2$	2	$p(1 - p)$
...	...	...
$S_{M-2} = z_1 z_1 \dots z_1 z_2$	$M - 1$	$p^{M-2}(1 - p)$
$S_{M-1} = z_1 z_1 \dots z_1 z_1$	$M - 1$	p^{M-1}

Код блока S_i – двоичное представление (b бит) его номера.

Код сообщения – последовательность кодов блоков.

Если энтропия источника сообщений мала (т.е. имеется состояние, в котором источник находится с вероятностью, близкой к единице), то вместо кодирования сообщений по Хаффману или Шэннону-Фано с объединением символов в блоки (было рассмотрено на предыдущем семинаре) можно использовать другой простой в реализации метод, который во многих случаях дает незначительную избыточность кода – метод кодирования длин серий (КДС). В этом методе последовательность символов сообщения также разбивается на блоки, но эти блоки

имеют переменное количество символов, а кодируются кодами фиксированной битовой длины b .

Рассмотрим метод КДС на примере двоичного источника и рассмотрим систему блоков, представленных на слайде. Каждый блок данной системы содержит на один символ z_1 больше предыдущего. Любое сообщение из последовательности символов, созданных источником Z , можно однозначно преобразовать в последовательность блоков из системы $\{S_i\}_{i=0}^{M-1}$. Битовый код сообщения будет представлять собой последовательно записанные (в двоичном представлении из b бит) номера соответствующих блоков. Фактически, кодируется длина серии символов z_1 , разделяющих в сообщении появления символа z_2 , что и дало название описанному методу. Реализация метода КДС весьма проста, так как для стороны кодера сводится к простому подсчету числа повторений символов, а для декодера – к формированию серий повторяющихся символов.

Задание. Для источника с двумя состояниями a и b и $M = 2^2$ закодировать сообщение $aaababbaaaab$.

◀ Запишем систему блоков (всего 4 штуки):

$$S_0 = b \rightarrow 00$$

$$S_1 = ab \rightarrow 01$$

$$S_2 = aab \rightarrow 10$$

$$S_3 = aaa \rightarrow 11$$

тогда для заданного сообщения получим:

$$aaa \ b \ ab \ b \ aaa \ ab \rightarrow 11 \ 00 \ 01 \ 00 \ 11 \ 01. \blacktriangleright$$

Домашнее задание

1. Для источника с 4-мя состояниями:

$$p(A) = 0,2; \ p(B) = 0,5; \ p(C) = 0,2; \ p(D) = 0,1,$$

закодировать и декодировать по полученному кодовому числу сообщение «CBAABD», используя регистры «бесконечной» разрядности.

2. Разобраться с отложенным выводом бит и закодировать сообщение «CBAABD» источника из предыдущего примера, используя 8-битные регистры.

3. Для источника с 2-мя состояниями и вероятностями $p = 0,8$ и $q = 0,2$ определить оптимальное количество блоков M и закодировать методом КДС сообщение $aabaaabaaaaabaaaaab$.

Семинар №8. Условная энтропия. Декоррелирующие преобразования. Дискретное косинусное преобразование

Условная энтропия

Условная энтропия

Ансамбль источников сообщения X и Y :

$$p(x_i, y_j) = p(x_i)p(y_j|x_i) = p(y_j)p(x_i|y_j), \text{ если при этом}$$

$\exists i, j: p(x_i, y_j) \neq p(x_i)p(y_j)$ то источники зависимы.

$$\text{Энтропия ансамбля: } H(X, Y) = H(X) + \sum_n p(x_n)H(Y|x_n),$$

$$\text{где } H(Y|x_n) = -\sum_m p(y_m|x_n) \log p(y_m|x_n) - \text{частная условная}$$

энтропия источника Y по отношению к состоянию x_n источника X .

$$H(X|Y) = \sum_n p(x_n)H(Y|x_n) - \text{условная энтропия источника } Y \text{ по отношению к источнику } X.$$

$$H(XY) \leq H(X), \quad H(X, Y) \leq H(X) + H(Y)$$

$$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$$

Задание. $H(X) = 5$ бит, $H(Y) = 10$ бит. В каких пределах изменяется $H(Y|X)$ при изменении $H(X|Y)$ в максимально возможных пределах?

◀ $H(X) + H(Y|X) = H(Y) + H(X|Y)$, откуда $H(Y|X) = H(Y) - H(X) + H(X|Y)$, следовательно, т.к. $0 \leq H(X|Y) \leq H(X) = 5$ бит и $H(Y) - H(X) = 5$ бит, имеем $5 = H(Y) - H(X) \leq H(Y|X) \leq H(Y) = 10$ бит. ▶

Источник сообщений с конечной памятью

Источник с памятью

Источник с конечной памятью: распределение вероятностей для текущего перехода источника в очередное состояние зависит от конечного числа M предыдущих состояний.

Стационарный источник – условные распределения не зависят от индекса m :

$$P(x_{k_0} | x_{k_1}, \dots, x_{k_M}) = P(X(m) = x_{k_0} | x(m-1) = x_{k_1}, \dots, x(m-M) = x_{k_M})$$

где $k_0, k_1, \dots, k_M \in \{1, \dots, N\}$ и $x_{k_j} \in \{x_1, \dots, x_N\}$.

Частная условная энтропия:

$$H(X | x_{k_1}, \dots, x_{k_M}) = - \sum_{k_0=1}^N p(x_{k_0} | x_{k_1}, \dots, x_{k_M}) \log p(x_{k_0} | x_{k_1}, \dots, x_{k_M})$$

Условная энтропия источника с памятью:

$$H_M(X) = \sum_{k_0=1}^N \dots \sum_{k_M=1}^N p(x_{k_1}, \dots, x_{k_M}) H(X | x_{k_1}, \dots, x_{k_M})$$

$X(m) \backslash X(m-1)$	x_0	x_1	x_2	$\sum_j p_{k,j}$
x_0	1/4	1/16	0	5/16
x_1	1/16	1/4	1/16	3/8
x_2	0	1/16	1/4	5/16
$\sum_k p_{k,j}$	5/16	3/8	5/16	1

Задание. Пусть последовательность символов сообщения $X(m)$ создается источником, для которого вероятности перехода в новое состояние зависят только от текущего состояния, а совместный закон распределения $p_{m_0, m_1} = P(X(m) = x_{m_0}, X(m-1) = x_{m_1})$ задан приведенной таблицей. Оценить минимальные битовые затраты R , которые необходимы для эффективного кодирования символа сообщения, если предыдущее состояние источника:

а) не учитывается,

б) учитывается.

◀ а) Если нет возможности учитывать предыдущее состояние источника, то эффективные коды строятся по безусловному

X	x_0	x_1	x_2
P	5/16	3/8	5/16
двоичный код	00	1	01

распределению вероятности. Используя метод Шэннона-Фано, получаем среднюю длину битового кода $R = \frac{5}{16} \cdot 2 + \frac{3}{8} \cdot 1 + \frac{5}{16} \cdot 2 = 1,625$ бит, при этом безусловная двоичная энтропия источника равна $H = -2 \cdot \frac{5}{16} \log_2 \frac{5}{16} - \frac{3}{8} \log_2 \frac{3}{8} \approx 1,579$ бит.

б) Возможны три варианта выбора статистической модели условного распределения вероятностей для символа $X(m)$ по реализации символа $X(m-1)$.

$X(m)$	x_0	x_1	x_2
$p(x_k x_0)$	4/5	1/5	0
двоичный код	1	0	—

$X(m-1) = x_0$. Тогда условное распределение для символа $X(m)$ задается следующей таблицей вероятностей (напомним, что условная вероятность находится по формуле $p(x|y) = p(x, y) / p(y)$).

Частная условная энтропия источника равна $H(X(m)|X(m-1) = x_0) = -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \approx 0,72$ бит, средние битовые затраты — $R = 1$ бит.

$X(m)$	x_0	x_1	x_2
$p(x_k x_1)$	1/6	2/3	1/6
двоичный код	00	1	01

$X(m-1) = x_1$. Частная условная энтропия равна $H(X(m)|X(m-1) = x_1) = -2 \cdot \frac{1}{6} \log_2 \frac{1}{6} - \frac{2}{3} \log_2 \frac{2}{3} \approx 1,25$ бит, средние битовые затраты $R = 2 \cdot \frac{1}{6} \cdot 2 + \frac{2}{3} = \frac{4}{3}$ бит.

$X(m)$	x_0	x_1	x_2
$p(x_k x_2)$	0	1/5	4/5
двоичный код	—	0	1

$X(m-1) = x_2$. Частная условная энтропия $H(X(m)|X(m-1) = x_2) = -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \approx 0,72$ бит, средние битовые затраты — $R = 1$ бит.

Условная энтропия источника с памятью:

$H_1(X) = H(X(m)|X(m-1)) = \sum_{k=0}^2 p(x_k) H(X(m)|X(m-1) = x_k) \approx 2 \cdot \frac{5}{16} \cdot 0,72 + \frac{3}{8} \cdot 1,25 \approx 0,92 < H(X)$ бит; средняя длина битового кода одного

символа: $R = 2 \cdot \frac{5}{16} \cdot 1 + \frac{3}{8} \cdot \frac{4}{3} = \frac{9}{8} = 1,125$ бит. ►

Дискретный марковский процесс

Дискретный марковский процесс

Стационарность (в широком смысле) дискретного процесса: отсчеты дискретного сигнала имеют одинаковые математические ожидания $M(X_k) = m_X$ и дисперсии $D(X_k) = \sigma_X^2$, а коэффициент корреляции $r(X_k, X_j)$ отсчетов X_k и X_j зависит только от расстояния между моментами наблюдения дискретного процесса, т.е. от величины $k - j$.

Дискретный марковский процесс порядка M (сложная цепь Маркова порядка M) – стационарный случайный процесс с дискретным временем и дискретными состояниями, условный закон распределения которого зависит только от M предыдущих значений $x(m-1), \dots, x(m-M)$.

Зависимость случайных величин

Зависимость случайных величин

Вектор данных $\mathbf{X} = (X_0, \dots, X_{N-1})^T$.

Вектор **математических ожиданий** $\mathbf{m}_x = (m_{X_0}, \dots, m_{X_{N-1}})^T$.

Ковариационная и корреляционная матрицы вектора $\mathbf{X}$:

$$\mathbf{K}_X = \begin{pmatrix} \sigma_{X_0}^2 & K_{0,1} & \dots & K_{0,N-1} \\ K_{1,0} & \sigma_{X_1}^2 & \dots & K_{1,N-1} \\ \dots & \dots & \dots & \dots \\ K_{N-1,0} & K_{N-1,1} & \dots & \sigma_{X_{N-1}}^2 \end{pmatrix}, \quad \mathbf{R}_X = \begin{pmatrix} 1 & \rho_{0,1} & \dots & \rho_{0,N-1} \\ \rho_{1,0} & 1 & \dots & \rho_{1,N-1} \\ \dots & \dots & \dots & \dots \\ \rho_{N-1,0} & \rho_{N-1,1} & \dots & 1 \end{pmatrix}.$$

где $K_{k,j} = K_{j,k} = \text{cov}(X_k, X_j) = M[(X_k - m_{X_k})(X_j - m_{X_j})]$,

$$\rho_{k,j} = \frac{\text{cov}(X_k, X_j)}{\sigma_{X_k} \cdot \sigma_{X_j}} \quad \text{и} \quad \forall k, j: |\rho_{k,j}| \leq 1.$$

Преобразование Карунена-Лозва

Преобразование Карунена-Лозва

Декоррелирующее преобразование Карунена-Лозва

вектора $\mathbf{X}$ в вектор $\mathbf{Y} = \mathbf{W}\mathbf{X}$, $\mathbf{W} = (\mathbf{r}_1^T \mathbf{r}_2^T \dots \mathbf{r}_N^T)^T$ – ортогональная матрица преобразования, где $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N$ – собственные векторы-столбцы матрицы $\mathbf{K}_X$.

$$\begin{pmatrix} \sigma_{X_0}^2 & K_{0,1} & \dots & K_{0,N-1} \\ K_{1,0} & \sigma_{X_1}^2 & \dots & K_{1,N-1} \\ \dots & \dots & \dots & \dots \\ K_{N-1,0} & K_{N-1,1} & \dots & \sigma_{X_{N-1}}^2 \end{pmatrix} \xrightarrow{\mathbf{W}\mathbf{K}_X\mathbf{W}^T} \begin{pmatrix} \sigma_{Y_0}^2 & 0 & \dots & 0 \\ 0 & \sigma_{Y_1}^2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & \sigma_{Y_{N-1}}^2 \end{pmatrix}$$

Компоненты вектора $\mathbf{Y} = (Y_0, Y_1, \dots, Y_{N-1})$ полностью декоррелированы.

Задание. Для ковариационной матрицы $\mathbf{K}_X = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$:

а) построить преобразование Карунена-Лозва,

б) убедиться, что ковариационная матрица $\mathbf{K}_Y = \mathbf{W}\mathbf{K}_X\mathbf{W}^T$ имеет диагональный вид.

◀ Характеристическое уравнение $\det(\mathbf{K}_X - l\mathbf{E}) = 0$, где l – собственные значения матрицы $\mathbf{K}_X$, $\det \begin{pmatrix} 1-l & r \\ r & 1-l \end{pmatrix} = (1-l)^2 - r^2 = 0$, откуда $l_1 = 1-r$, $l_2 = 1+r$. Найдем собственные векторы $\mathbf{r}$, удовлетворяющие соотношению

$\mathbf{K}_X \mathbf{r} = l \mathbf{r}$: $(\mathbf{K}_X - l\mathbf{E})\mathbf{r} = \mathbf{0}$, $\begin{pmatrix} 1-l & r \\ r & 1-l \end{pmatrix} \begin{pmatrix} r_1 \\ r_2 \end{pmatrix} = \mathbf{0}$. При $l = l_1 = 1-r$, имеем:

$\begin{pmatrix} r & r \\ r & r \end{pmatrix} \begin{pmatrix} r_1 \\ r_2 \end{pmatrix} = \mathbf{0}$, откуда $r_1 = -r_2$; при $l = l_2 = 1+r$ получим $\begin{pmatrix} -r & r \\ r & -r \end{pmatrix} \begin{pmatrix} r_1 \\ r_2 \end{pmatrix} = \mathbf{0}$,

следовательно, $r_1 = r_2$. Таким образом, собственными векторами матрицы $\mathbf{K}_X$ являются векторы вида $\mathbf{r} = (r, r)$ и $\mathbf{r} = (r, -r)$, где $r \in \mathbb{R}$, а матрица преобразования

имеет вид $\mathbf{W} = \begin{pmatrix} r & r \\ r & -r \end{pmatrix}$. Видим, что строки (столбцы) матрицы $\mathbf{W}$ взаимно

ортогональны. Потребуем, чтобы $\mathbf{W}^{-1} = \mathbf{W}^T$: $\mathbf{W} \cdot \mathbf{W}^T = \mathbf{E}$, т.е.

$\begin{pmatrix} r & r \\ r & -r \end{pmatrix} \begin{pmatrix} r & r \\ r & -r \end{pmatrix} = \begin{pmatrix} 2r^2 & 0 \\ 0 & 2r^2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, откуда получим $r = 1/\sqrt{2}$.

Окончательно матрица преобразования Карунена-Лоэва $\mathbf{W} = \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix}$.

Убедимся, что оно является декоррелирующим:

$$\mathbf{K}_Y = \mathbf{W} \mathbf{K}_X \mathbf{W}^T = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

используя MATLAB (`syms p, 1/2*[1 1; 1 -1]*[1 p; p 1]*[1 1; 1 -1]`),

находим $\mathbf{K}_Y = \begin{pmatrix} 1+r & 0 \\ 0 & 1-r \end{pmatrix}$. ►

Дискретное косинусное преобразование

ДКП

Марковский процесс 1-го порядка: $r(X_k, X_j) = \rho^{|k-j|}$.

Дискретное косинусное преобразование (ДКП):

$$y_k = \sqrt{2/N} c(k) \sum_{j=0}^{N-1} x_j \cos\left(\frac{\pi k}{N} (j+0,5)\right), \quad c(k) = \begin{cases} 1/\sqrt{2}, & k=0 \\ 1, & k \neq 0 \end{cases}$$

$k = 0, \dots, N-1$.

Обратное ДКП:

$$x_j = \sqrt{2/N} \sum_{k=0}^{N-1} c(k) y_k \cos\left(\frac{\pi k}{N} (j+0,5)\right), \quad c(k) = \begin{cases} 1/\sqrt{2}, & k=0 \\ 1, & k \neq 0 \end{cases}$$

$j = 0, \dots, N-1$.

Матрица преобразования:

$$\mathbf{W} = \left\{ w_{k,j} = \sqrt{2/N} c(k) \cos\left(\frac{\pi k}{N} \left(j + \frac{1}{2}\right)\right) \right\}_{k,j=0}^{N-1}, \quad \mathbf{W}^{-1} = \mathbf{W}^T.$$

Компоненты вектора ДКП, в отличие от амплитудного спектра ДПФ, не являются инвариантными относительно сдвига входного сигнала.

Задание. Убедиться в декоррелирующих свойствах ДКП, используя программу task1.m:

```
I = im2double(imread('boat.tif')); % load image
I = im2col(I, [1 32], 'distinct');
Kx = cov(I); % find covariation matrix
[W, D] = eig(Kx);
KLT = W' * Kx * W;
KLT(find(abs(KLT) < 10^-17)) = 10^-17;
I_dct = dct(I')'; % make DCT
DCT = cov(I_dct); % new covariation matrix
cmap = 'jet'
figure
subplot(221)
contourf(log10(abs(Kx)), 20, 'LineStyle', 'None'), ...
```

```

    colorbar, axis ij
colormap(cmap)
subplot(222)
contourf(log10(abs(KLT)),20,'LineStyle','None'), ...
    colorbar, axis ij
colormap(cmap)
subplot(223)
contourf(log10(abs(DCT)),20,'LineStyle','None'), ...
    colorbar, axis ij
colormap(cmap)

```

Домашнее задание

1. Построить матрицу ДКП для $N = 4$.
2. Найти ДКП векторов $\mathbf{x}_1 = (1, 0, 0, 0)$, $\mathbf{x}_2 = (0, 1, 0, 0)$, $\mathbf{x}_3 = (0, 0, 1, 0)$ и $\mathbf{x}_4 = (0, 0, 0, 1)$.
3. Используя функцию MATLAB `eig`, найти матрицы преобразования Карунена-

Лоэва для $\mathbf{K}_X = \begin{pmatrix} 1 & r & r^2 \\ r & 1 & r \\ r^2 & r & 1 \end{pmatrix}$, где $r = \{0,5; 0,9\}$, и вычислить соответствующие матрицы $\mathbf{K}_Y$.

4. Найти ковариационную матрицу $\mathbf{K}_Y$ вектора $\mathbf{Y}$, полученного в результате ДКП

вектора $\mathbf{X}$, имеющего ковариационную матрицу $\mathbf{K}_X = \begin{pmatrix} 1 & r & r^2 \\ r & 1 & r \\ r^2 & r & 1 \end{pmatrix}$, где $r = \{0,5; 0,9\}$.

Часть 2. Лабораторный практикум

Лабораторная работа №1. Дискретизация и квантование сигналов

Исследование эффектов дискретизации

Цель работы

Исследование влияния частоты дискретизации на спектр дискретных сигналов.

Теоретические сведения

Пусть $x(t)$ – непрерывная (или кусочно-непрерывная) функция, принимающая любые конечные значения. Сигналы, описываемые такими функциями, называются *аналоговыми* [1]. Аналоговыми сигналами $x(t)$ описывается большинство реальных физических процессов, причем интервал наблюдения обычно конечный: $t \in [a; b]$.

Если $t \in \{t_k\}$, то последовательность $\{x(t_k)\}$ называют *дискретным сигналом*. Рассмотрим *дискретизацию* аналогового сигнала с постоянным шагом Δt , то есть будем измерять аналоговый сигнал $x(t)$ через равные промежутки времени Δt , называемые *интервалом (периодом) дискретизации*. Тогда получим некоторую последовательность значений $x_k = x(k\Delta t)$ – *отсчётов* дискретного сигнала.

Величина $f_s = \frac{1}{\Delta t}$ называется *частотой дискретизации (sampling frequency)*. От ее выбора зависит возможность восстановления аналогового сигнала из дискретного без искажений. Согласно *теореме Котельникова (отсчётов)* [1], точное восстановление непрерывного сигнала, имеющего спектр ограниченной частотной полосы (т.е. $S(n) = 0$ при $|n| > F_{\max}$, $F_{\max} = \max_{n: S(n) \neq 0} |n|$), по его дискретным отсчётам

возможно только в том случае, когда частота дискретизации f_s удовлетворяет условию:

$$f_s \geq 2F_{\max}. \quad (1)$$

При несоблюдении этого условия возможно возникновение *эффекта наложения частот*, то есть в спектре дискретного сигнала могут появиться гармоники, которых, возможно, не было в исходном сигнале. Этот эффект приводит к необратимым искажениям в восстановленном аналоговом сигнале [1].

Аналогично одномерным ведут себя и двумерные сигналы. Эффект наложения частот хорошо заметен на цифровых изображениях (роль независимой переменной – аналога времени в одномерных сигналах – в этом случае играют две пространственные координаты) при их некорректном масштабировании.

Задание

1. Синтезировать сигнал $x(t)$, представляющий из себя сумму нескольких синусоид с разными частотами.
2. Определить допустимые значения частоты дискретизации f_s для сигнала $x(t)$.
3. Построить по отсчетам график исходного сигнала и его спектра при нескольких различных частотах дискретизации. Проиллюстрировать на примере сигнала $x(t)$ эффект наложения частот.
4. Воспроизвести сигнал $x(t)$ с различными частотами дискретизации.
5. Загрузить тестовое изображение. Уменьшить частоту дискретизации в 2, 3, 4 раза с помощью прореживания матрицы исходного изображения. Сравнить полученные результаты с результатом использования функции `imresize`.

Примечание

При выполнении работы в MATLAB можно использовать следующие функции: `fft`, `sound`, `imread`, `imshow`.

Контрольные вопросы

1. Что такое спектр кусочно-гладкой функции?
2. Дать определение амплитудного и фазового спектра.
3. Как связаны друг с другом дискретное и интегральное преобразования Фурье?
4. Сформулировать теорему Котельникова.
5. Что такое спектр дискретного сигнала?
6. Как связаны спектры аналогового сигнала $x(t)$ и соответствующего дискретного сигнала $\{x_k = x(k\Delta t)\}$?
7. Чему равен период спектра дискретного сигнала?
8. Схематично изобразить спектр дискретного сигнала с частотой дискретизации $f_s : f_s \geq 2F_{\max}, f_s < 2F_{\max}$.
9. В чем заключается эффект наложения частот? Привести пример эффекта наложения частот.

Исследование эффектов квантования

Цель работы

Исследование влияния параметров квантования на качество звуковых сигналов, изучение статистических аспектов квантования.

Теоретические сведения

Положим, что дискретный сигнал $\{x_k\}$, полученный из непрерывного сигнала $x(t)$, может принимать любые значения из диапазона $[x_{\min}; x_{\max}]$. Квантование сигнала заключается в замене каждого отсчёта x_k значением из некоторого конечного множества $\Omega = \{d_j\}_{j=0}^{N-1}$, где d_j – возможные уровни квантования, в соответствии с некоторым правилом $Q: x_k \xrightarrow{Q} \hat{x}_k \in \Omega$ [1]. Полученный сигнал $\{\hat{x}_k\}$ называется *цифровым*.

Разобьем отрезок $[x_{\min}, x_{\max}]$ на N в общем случае неравных частей (по числу уровней квантования) точками $\{t_j\}_{j=0}^N$, называемыми *порогами квантования*, где $t_0 = x_{\min}$ и $t_N = x_{\max}$. В этом случае правило квантования Q будет иметь следующий вид: если $x_k \in [t_j; t_{j+1})$, то принять $\hat{x}_k = d_j$, $j = 0, \dots, N-1$. Обычно $N = 2^n$, где n – число бит для представления одного отсчёта сигнала.

Отсчёты дискретного сигнала удобно рассматривать как реализацию некоторой случайной величины X непрерывного типа, при этом процесс квантования представляет собой процесс преобразования случайной величины непрерывного типа в случайную величину дискретного типа: $\hat{X} = Q(X)$. Выбор правила квантования Q определяется техническими возможностями реализации квантователя, а также наличием информации о законе распределения X .

Для оценки ошибки квантования используется либо величина

$$\overline{e^2} = M\left((X - \hat{X})^2\right), \quad (2)$$

где обозначение M используется для математического ожидания, а горизонтальная черта означает операцию усреднения; либо *отношение сигнал-шум (signal-to-noise ratio, SNR)*:

$$SNR = 20 \cdot \lg\left(\frac{A_{\text{signal}}}{A_{\text{noise}}}\right), \quad (3)$$

где A_{signal} и A_{noise} – среднеквадратичные значения (*root mean square, RMS*) сигнала и шума соответственно, а под шумом понимается сигнал ошибки $E = X - \hat{X}$. Вычисление среднеквадратичного значения производится по формуле

$$A_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} A_i^2}. \quad (4)$$

Отношение сигнал-шум выражается в децибелах (дБ). SNR тем выше, чем меньше амплитуда шума по отношению к амплитуде сигнала.

Равномерное квантование

Равномерное квантование удобно использовать в случае, когда о величине X известно лишь то, что она попадает в некоторый диапазон $X \in [x_{\min}; x_{\max}]$, либо необходимо реализовать простейший вариант квантователя [1].

При равномерном квантовании диапазон $[x_{\min}; x_{\max}]$ разбивается на N равных интервалов длины $q = \frac{x_{\max} - x_{\min}}{N}$: $\Delta_j = [x_{\min} + jq; x_{\min} + (j+1)q)$, $j = 0, \dots, N-1$. В качестве уровней квантования $\{d_j\}_{j=0}^{N-1}$ выбираются середины интервалов Δ_j : $d_j = x_{\min} + (j+0,5)q$. Правило квантования при этом имеет следующий вид: если $x \in \Delta_j$, то $Q(x) = d_j$.

Если интервал равномерного квантования q достаточно мал, можно считать, что ошибка квантования E подчиняется равномерному закону распределения на отрезке $\left[-\frac{q}{2}, \frac{q}{2}\right]$. В этом случае $M(E) = 0$, а дисперсия ошибки равна [1]

$$e^2 = D(E) = \frac{q^2}{12}. \quad (5)$$

Оптимальное квантование

В случае, когда известна функция плотности распределения вероятностей $f_X(x)$ случайной величины X , причем $f_X(x) > 0$ при $X \in [x_{\min}; x_{\max}]$ и $f_X(x) = 0$ при $X \notin [x_{\min}; x_{\max}]$, ошибка квантования (2) принимает вид [1]:

$$e^2 = \sum_{j=0}^{N-1} \int_{t_j}^{t_{j+1}} (x - d_j)^2 f_X(x) dx. \quad (6)$$

Для нахождения оптимального правила квантования необходимо решить задачу минимизации функции (6) по переменным $t_1, \dots, t_{N-1}$, $d_0, \dots, d_{N-1}$. Данная задача сводится к решению системы уравнений:

$$\begin{cases} t_j = \frac{d_{j-1} + d_j}{2}, \\ d_j = \frac{\int_{t_j}^{t_{j+1}} x f_X(x) dx}{\int_{t_j}^{t_{j+1}} f_X(x) dx}, \end{cases} \quad (7)$$
$$j = 0, \dots, N-1.$$

Решение системы (7), которое в общем случае находится численными методами, определяет квантователь Ллойда-Макса и дает минимальное значение ошибки (6).

Задание

1. Синтезировать случайный дискретный сигнал $\{x_k\}$, имеющий равномерное распределение отсчетов. Построить по отсчетам график полученного сигнала.
2. Провести равномерное квантование отсчетов сигнала $\{x_k\}$, используя от 1 до 8 бит на отсчет. Построить ступенчатые графики сигнала после квантования.
3. Экспериментально определить ошибку квантования (2). Сравнить полученные результаты с теоретической оценкой (5).
4. Вычислить SNR (3). Исследовать зависимость SNR от числа бит, выделяемого для хранения одного отсчета сигнала.
5. Синтезировать случайный дискретный сигнал $\{x_k\}$, имеющий нормальное распределение отсчетов. Построить по отсчетам график полученного сигнала.
6. По полученной выборке оценить параметры $\tilde{m}$ и $\tilde{s}$.
7. Определить параметры квантователя Ллойда-Макса: $t'_k = t_k \tilde{s} + \tilde{m}$, $d'_k = d_k \tilde{s} + \tilde{m}$, где $\{t_k\}$ и $\{d_k\}$ – параметры оптимального квантования для равномерного распределения с параметрами $m = 0$, $s = 1$ (см. приложение).
8. Выполнить оптимальное квантование сигнала $\{x_k\}$, используя от 1 до 4 бит на отсчет.
9. Вычислить выборочные значения ошибки (2) и SNR.
10. Выполнить равномерное квантование сигнала $\{x_k\}$ при числе бит на отсчет от 1 до 4. Сравнить результат с полученным в предыдущем пункте.

Контрольные вопросы

1. В чем заключается процесс квантования?
2. Какие величины используются для оценки ошибки квантования? От чего зависит ошибка квантования?
3. Что такое равномерное квантование? В каких случаях его применяют?
4. Что такое оптимальное квантование Ллойда-Макса?
5. Чем нужно руководствоваться при выборе метода квантования?

Приложение

Пороги и уровни оптимального квантования Ллойда-Макса для случайной величины со стандартизованным нормальным распределением:

1. При использовании 1 бита на отсчет:

$$t = [-\infty; 0; \infty],$$

$$d = [-0,7979; 0,7979].$$

2. При использовании 2 бит на отсчет:

$$t = [-\infty; -0,9816; 0; 0,9816; \infty],$$

$$d = [-1,5104; -0,4528; 0,4528; 1,5104].$$

3. При использовании 3 бит на отсчет:

$$t = [-\infty; -1,7479; -1,0500; 0; 1,0500; 1,7479; \infty],$$

$$d = [-2,1519; -1,3439; -0,7560; -0,2451; 0,2451; 0,7560; 1,3439; 2,1519].$$

4. При использовании 4 бит на отсчет:

$$t = [-\infty; -2,4008; -1,8435; -1,4371; -1,0993; -0,7995; -0,5224; -0,2582;$$

$$0; 0,2582; 0,5224; 0,7995; 1,0993; 1,4371; 1,8435; 2,4008; \infty],$$

$$d = [-2,7326; -2,0690; -1,6180; -1,2562; -0,9423; -0,6568; -0,3880;$$

$$-0,1284; 0,1284; 0,3880; 0,6568; 0,9423; 1,2562; 1,6180; 2,0690; 2,7326].$$

Литература

1. Умняшкин С.В. Теоретические основы цифровой обработки и представления сигналов.

Лабораторная работа №2. Быстрое преобразование Фурье. Дискретная свертка

Цель работы

Изучить алгоритм быстрого преобразования Фурье (БПФ) и убедиться в целесообразности использования быстрых алгоритмов ортогональных преобразований.

Теоретические сведения

Дискретным преобразованием Фурье (ДПФ) или дискретным спектром числового вектора $\mathbf{X} = (x_0, x_1, \dots, x_{N-1})$ называется вектор $\mathbf{Y} = (y_0, y_1, \dots, y_{N-1})$, компоненты которого определяются формулой [1]

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{-\frac{2\pi i}{N} kj}, \quad k = 0, \dots, N-1. \quad (8)$$

Вектор $\mathbf{X} = (x_0, x_1, \dots, x_{N-1})$ можно восстановить по дискретному спектру (8) при помощи обратного ДПФ (ОДПФ):

$$x_j = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} y_k e^{\frac{2\pi i}{N} kj}, \quad j = 0, \dots, N-1. \quad (9)$$

Вектор $\mathbf{X} = (x_0, x_1, \dots, x_{N-1})$ обычно представляет собой N последовательных отсчетов $x_j = x(j\Delta t)$ аналогового сигнала $x(t)$, тогда можно считать, что последовательность $\mathbf{Y} = (y_0, y_1, \dots, y_{N-1})$ с некоторой точностью представляет последовательные отсчеты $y_k = y(k\Delta n)$ в частотной области. Таким образом, ДПФ является аппроксимацией непрерывного преобразования Фурье [2, с. 75].

При нахождении ДПФ непосредственно по формуле (8) для вычисления каждого из N коэффициентов y_k требуется около N комплексных сложений с умножениями, следовательно, для реализации ДПФ по формуле (8) требуется около N^2 комплексных сложений с умножениями, т.е. алгоритм вычисления имеет сложность $O(N^2)$. Такой способ вычисления ДПФ не является эффективным и не представляет практического интереса.

БПФ с прореживанием по времени

Идея быстрого преобразования Фурье (БПФ) заключается в замене одного ДПФ большой размерности несколькими ДПФ меньшей размерности [2, с. 81].

Рассмотрим ДПФ размерности $N = 2^n$. Обозначив $x(k) = x_k$, $y(k) = y_k$, $w_n = e^{-\frac{2\pi i}{2^n}}$, запишем «ненормированное» ДПФ, $\tilde{y}(k) = \sqrt{N} y(k)$:

$$\tilde{y}(k) = \sum_{j=0}^{2^n-1} x(j)w_n^{kj}. \quad (10)$$

Разделим общую сумму на две: первая содержит слагаемые с четными индексами, вторая – с нечетными:

$$\tilde{y}(k) = \sum_{j=0}^{2^{n-1}-1} (x(2j)w_n^{k \cdot 2j} + x(2j+1)w_n^{k(2j+1)}) = \sum_{j=0}^{2^{n-1}-1} x_0(j)w_{n-1}^{kj} + w_n^k \sum_{j=0}^{2^{n-1}-1} x_1(j)w_{n-1}^{kj} = \tilde{y}_0(k) + w_n^k \tilde{y}_1(k),$$

$\mathbf{1} \quad \mathbf{4} \quad \mathbf{2} \quad \mathbf{4} \quad \mathbf{8}$ $\mathbf{1} \quad \mathbf{4} \quad \mathbf{2} \quad \mathbf{4} \quad \mathbf{8}$
 Обозначим $\tilde{y}_0(k)$ Обозначим $\tilde{y}_1(k)$

где $\mathbf{X}_0 = (x(0), x(2), \mathbf{K}, x(2^n - 2)) = (x_0(0), x_0(1), \mathbf{K}, x_0(2^{n-1} - 1))$ – вектор четных, а $\mathbf{X}_1 = (x(1), x(3), \mathbf{K}, x(2^n - 1)) = (x_1(0), x_1(1), \mathbf{K}, x_1(2^{n-1} - 1))$ – нечетных компонент исходного вектора $\mathbf{X}$; $\tilde{y}_0(k)$ – k -й элемент ДПФ (10) размерности 2^{n-1} вектора $\mathbf{X}_0$, а $\tilde{y}_1(k)$ – k -й элемент ДПФ (10) вектора $\mathbf{X}_1$. Учитывая, что

$\tilde{y}_0(k) = \tilde{y}_0(k + 2^{n-1})$, $\tilde{y}_1(k) = \tilde{y}_1(k + 2^{n-1})$, и $w_n^{2^{n-1}+k} = e^{-\frac{2\pi i}{2^n}(2^{n-1}+k)} = -w_n^k$, последнее выражение для $\tilde{y}(k)$ можно переписать в виде:

$$\begin{cases} \tilde{y}(k) = \tilde{y}_0(k) + w_n^k \tilde{y}_1(k), \\ \tilde{y}(k + 2^{n-1}) = \tilde{y}_0(k) - w_n^k \tilde{y}_1(k), \\ k = 0, 1, \mathbf{K}, 2^{n-1} - 1. \end{cases} \quad (11)$$

При помощи (11) можно выразить коэффициенты ДПФ (10) размерности 2^n через коэффициенты ДПФ размерности 2^{n-1} $\tilde{\mathbf{Y}}_0 = (\tilde{y}_0(0), \tilde{y}_0(1), \mathbf{K}, \tilde{y}_0(2^{n-1} - 1))$ и $\tilde{\mathbf{Y}}_1 = (\tilde{y}_1(0), \tilde{y}_1(1), \mathbf{K}, \tilde{y}_1(2^{n-1} - 1))$, которые получены из векторов $\mathbf{X}_0$ и $\mathbf{X}_1$ соответственно. Граф вычисления 8-точечного ДПФ по описанной выше схеме представлен на рис. 1.

При таком способе, называемом *прореживанием по времени*, N -точечное ДПФ сводится к двум $\frac{N}{2}$ -точечным ДПФ и, дополнительно, N сложениям и $\frac{N}{2}$ умножениям на w_n^k . Эта же процедура применяется далее для замены двух $\frac{N}{2}$ -

точечных ДПФ на четыре $\frac{N}{4}$ -точечных ДПФ, N сложений и $\frac{N}{2}$ умножений.

Последовательное применение метода вычисления N -точечных ДПФ, $N = 2^n$, приводит к $n = \log_2 N$ шагам, на каждом из которых 2^i 2^{n-i} -точечных ДПФ сводятся к 2^{i+1} 2^{n-i-1} -точечным ДПФ. Общее число комплексных умножений M и число комплексных сложений A , требуемых для вычисления N -точечного ДПФ, равны $M = \frac{N}{2} \log_2 N$ и $A = N \log_2 N$ [2, с. 82].

Дальнейшее использование идеи формулы (11) для ДПФ размерности $N = 8$ дает полный граф вычислений, который изображен на рис. 2.

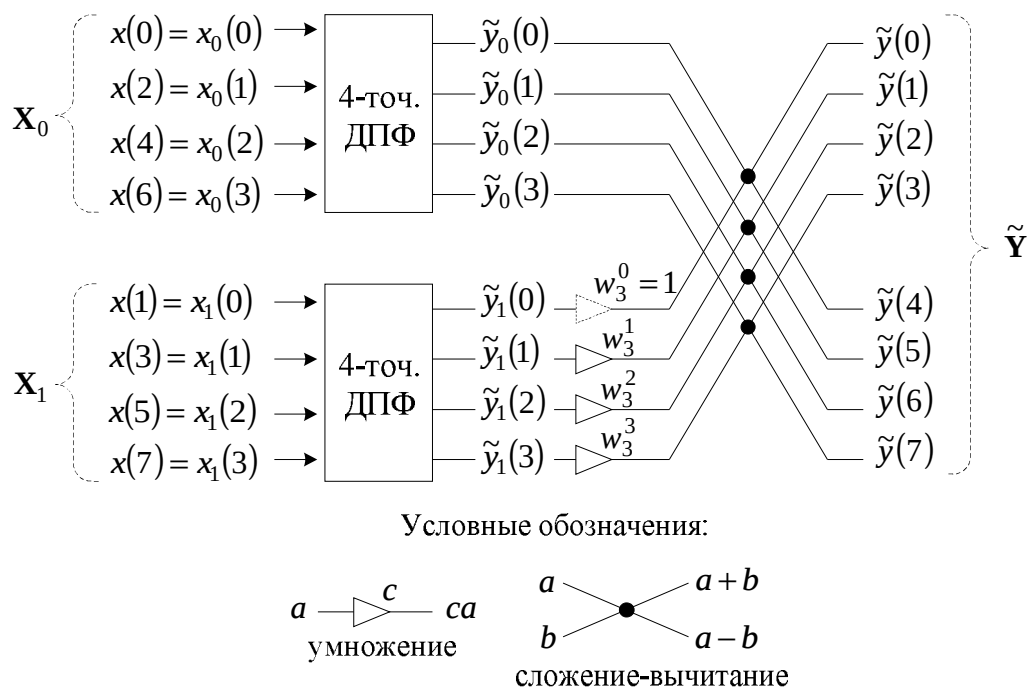


Рис. 1. Граф вычислений ДПФ, определяемый формулой (11), при $N=8$

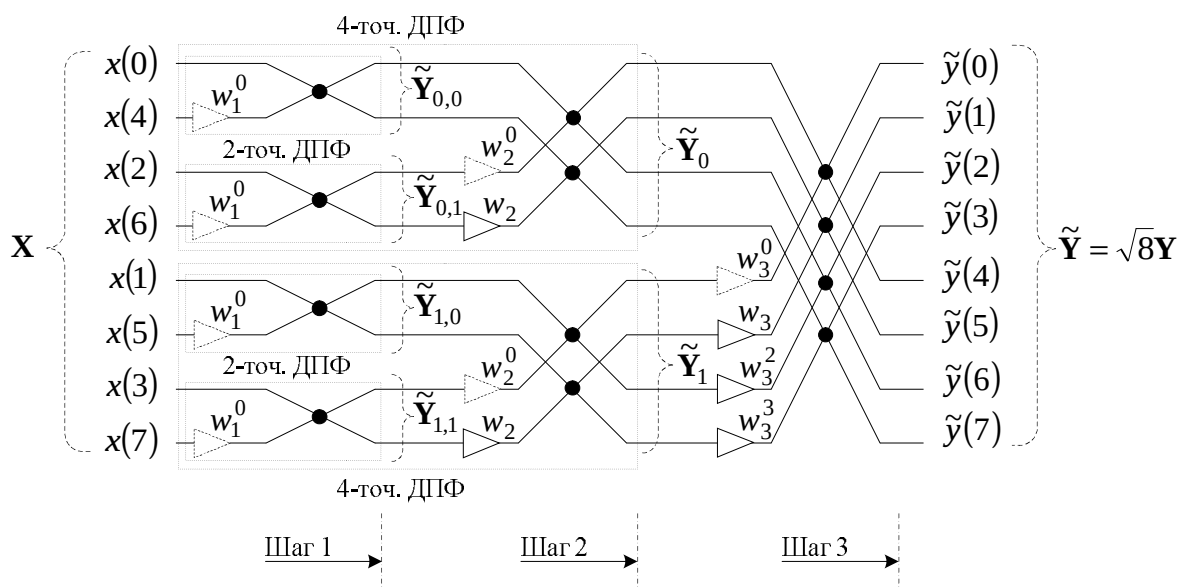


Рис. 2. Полный граф вычисления 8-точечного ДПФ с прореживанием по времени

Как видно из рис. 1 и 2, базовой операцией БПФ на j -ом шаге является так называемая «бабочка», см. рис. 3.

$\tilde{\mathbf{Y}} = \sqrt{8}\mathbf{Y}$, а векторы $\tilde{\mathbf{Y}}_0$ и $\tilde{\mathbf{Y}}_1$ есть, соответственно, результаты 4-точечных «ненормированных» ДПФ (10), выполненных над чётными $\mathbf{X}_0$ и нечетными $\mathbf{X}_1$ компонентами вектора $\mathbf{X}$. Граф рис. 1 соответствует следующей матричной операции умножения:

$$\tilde{\mathbf{Y}} = \mathbf{W}_3 \begin{pmatrix} \tilde{\mathbf{Y}}_0 \\ \tilde{\mathbf{Y}}_1 \end{pmatrix} = \mathbf{W}_3 \begin{pmatrix} \tilde{y}_0(0) \\ \tilde{y}_0(1) \\ \tilde{y}_0(2) \\ \tilde{y}_0(3) \\ \tilde{y}_1(0) \\ \tilde{y}_1(1) \\ \tilde{y}_1(2) \\ \tilde{y}_1(3) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & w_3 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & w_3^2 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & w_3^3 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -w_3 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & -w_3^2 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -w_3^3 \end{pmatrix} \begin{pmatrix} \tilde{y}_0(0) \\ \tilde{y}_0(1) \\ \tilde{y}_0(2) \\ \tilde{y}_0(3) \\ \tilde{y}_1(0) \\ \tilde{y}_1(1) \\ \tilde{y}_1(2) \\ \tilde{y}_1(3) \end{pmatrix},$$

$\mathbf{W}_3$

где $w_3 = \exp\left(-i\frac{p}{4}\right)$. В свою очередь (см. рис. 2 и (11)),

$$\tilde{\mathbf{Y}}_0 = \begin{pmatrix} \tilde{y}_0(0) \\ \tilde{y}_0(1) \\ \tilde{y}_0(2) \\ \tilde{y}_0(3) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & w_2 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -w_2 \end{pmatrix} \begin{pmatrix} \tilde{\mathbf{Y}}_{0,0} \\ \tilde{\mathbf{Y}}_{0,1} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & w_2 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -w_2 \end{pmatrix} \begin{pmatrix} \tilde{y}_{0,0}(0) \\ \tilde{y}_{0,0}(1) \\ \tilde{y}_{0,1}(0) \\ \tilde{y}_{0,1}(1) \end{pmatrix},$$

$$\tilde{\mathbf{Y}}_1 = \begin{pmatrix} \tilde{y}_1(0) \\ \tilde{y}_1(1) \\ \tilde{y}_1(2) \\ \tilde{y}_1(3) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & w_2 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -w_2 \end{pmatrix} \begin{pmatrix} \tilde{\mathbf{Y}}_{1,0} \\ \tilde{\mathbf{Y}}_{1,1} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & w_2 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -w_2 \end{pmatrix} \begin{pmatrix} \tilde{y}_{1,0}(0) \\ \tilde{y}_{1,0}(1) \\ \tilde{y}_{1,1}(0) \\ \tilde{y}_{1,1}(1) \end{pmatrix},$$

где $w_2 = \exp\left(-i\frac{p}{2}\right) = -i$. Отсюда для матрицы $\mathbf{W}_2$ получаем структуру:

$$\begin{pmatrix} \tilde{\mathbf{Y}}_0 \\ \tilde{\mathbf{Y}}_1 \end{pmatrix} = \begin{pmatrix} \tilde{y}_0(0) \\ \tilde{y}_0(1) \\ \tilde{y}_0(2) \\ \tilde{y}_0(3) \\ \tilde{y}_1(0) \\ \tilde{y}_1(1) \\ \tilde{y}_1(2) \\ \tilde{y}_1(3) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & -i & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & i & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & -i \\ 0 & 0 & 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & i \end{pmatrix} \begin{pmatrix} \tilde{y}_{0,0}(0) \\ \tilde{y}_{0,0}(1) \\ \tilde{y}_{0,1}(0) \\ \tilde{y}_{0,1}(1) \\ \tilde{y}_{1,0}(0) \\ \tilde{y}_{1,0}(1) \\ \tilde{y}_{1,1}(0) \\ \tilde{y}_{1,1}(1) \end{pmatrix},$$

$\mathbf{W}_2$

Аналогично, по графу рис. 2 получаем структуру матрицы $\mathbf{W}_1$:

$$\begin{pmatrix} \tilde{y}_{0,0}(0) \\ \tilde{y}_{0,0}(1) \\ \tilde{y}_{0,1}(0) \\ \tilde{y}_{0,1}(1) \\ \tilde{y}_{1,0}(0) \\ \tilde{y}_{1,0}(1) \\ \tilde{y}_{1,1}(0) \\ \tilde{y}_{1,1}(1) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \\ x(4) \\ x(5) \\ x(6) \\ x(7) \end{pmatrix}.$$

$\mathbf{W}_1$

Матрицу $\mathbf{W}_1$ можно представить в виде:

$$\mathbf{W}_1 = \begin{pmatrix} \boxed{1} & \boxed{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ \boxed{1} & \boxed{-1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & \boxed{1} & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & \boxed{-1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \boxed{1} & \boxed{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & \boxed{1} & \boxed{-1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \boxed{1} & \boxed{1} \\ 0 & 0 & 0 & 0 & 0 & 0 & \boxed{1} & \boxed{-1} \end{pmatrix} \cdot \mathbf{C},$$

где $\mathbf{C}$ – матрица перестановок элементов с индексами k, k' , образующих *двоично-инверсную* пару, т.е. если $k = \sum_{j=0}^{n-1} k_j \cdot 2^j$, то $k' = \sum_{j=0}^{n-1} k_{n-j-1} \cdot 2^j$, $k = 0, 1, \dots, 2^n - 1$.

Рассмотрим, к примеру, двоичное представление индексов от 0 до 7:

$$000_2 \ 001_2 \ 010_2 \ 011_2 \ 100_2 \ 101_2 \ 110_2 \ 111_2.$$

Если записать каждое число в обратном порядке, получим:

$$000_2 \ 100_2 \ 010_2 \ 110_2 \ 001_2 \ 101_2 \ 011_2 \ 111_2,$$

при этом в десятичной системе будем иметь нужный порядок индексов: 0 4 2 6 1 5 3 7. Для 8-точечного преобразования матрица перестановок $\mathbf{C}$ имеет вид:

$$\mathbf{C} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

Обобщая результаты примера на случай произвольной размерности $N = 2^n$, структуру матрицы ДПФ можно представить в виде:

$$\mathbf{W} = \mathbf{W}_n \cdot \dots \cdot \mathbf{W}_1 \cdot \mathbf{C},$$

где матрица $\mathbf{W}_k$ имеет следующий вид:

$$\mathbf{W}_k = \begin{pmatrix} \mathbf{B}_k^{(0)} \\ \mathbf{B}_k^{(1)} \\ \vdots \\ \mathbf{B}_k^{(2^{n-k}-1)} \end{pmatrix}, \quad k = 1, \dots, n,$$

а каждый блок $\mathbf{B}_k^{(j)}$ состоит из 2^k строк и может быть записан как

$$\mathbf{B}_k^{(j)} = \begin{pmatrix} \begin{array}{ccc|ccc|ccc|ccc} 0 & \mathbf{L} & 0 & 1 & 0 & 0 & \mathbf{L} & 0 & 1 & 0 & 0 & \mathbf{L} & 0 & 0 & \mathbf{L} & 0 \\ 0 & \mathbf{L} & 0 & 0 & 1 & 0 & \mathbf{L} & 0 & 0 & w_k & 0 & \mathbf{L} & 0 & 0 & \mathbf{L} & 0 \\ 0 & \mathbf{L} & 0 & 0 & 0 & 1 & \mathbf{L} & 0 & 0 & 0 & w_k^2 & \mathbf{L} & 0 & 0 & \mathbf{L} & 0 \\ 0 & \mathbf{L} & 0 & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} & \mathbf{M} & \mathbf{L} & \mathbf{M} \\ \mathbf{M} & \mathbf{L} & \mathbf{M} & 0 & 0 & \mathbf{L} & 0 & 1 & 0 & 0 & 0 & 0 & \mathbf{L} & w_k^{2^{k-1}-1} & \mathbf{M} & \mathbf{L} & \mathbf{M} \\ \mathbf{M} & \mathbf{L} & \mathbf{M} & 1 & 0 & 0 & \mathbf{L} & 0 & -1 & 0 & 0 & \mathbf{L} & 0 & \mathbf{M} & \mathbf{L} & \mathbf{M} \\ 0 & \mathbf{L} & 0 & 0 & 1 & 0 & \mathbf{L} & 0 & 0 & -w_k & 0 & \mathbf{L} & 0 & 0 & \mathbf{L} & 0 \\ 0 & \mathbf{L} & 0 & 0 & 0 & 1 & \mathbf{L} & 0 & 0 & 0 & -w_k^2 & \mathbf{L} & 0 & 0 & \mathbf{L} & 0 \\ 0 & \mathbf{L} & 0 & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} & \mathbf{M} & \mathbf{L} & \mathbf{M} \\ 0 & \mathbf{L} & 0 & 0 & 0 & \mathbf{L} & 0 & 1 & 0 & 0 & 0 & 0 & \mathbf{L} & -w_k^{2^{k-1}-1} & 0 & \mathbf{L} & 0 \end{array} \end{pmatrix} \begin{array}{l} \left. \begin{array}{l} \text{ } \end{array} \right\} 2^{k-1} \text{ строк} \\ \left. \begin{array}{l} \text{ } \end{array} \right\} 2^{k-1} \text{ строк} \end{array}$$

$\underbrace{j \cdot 2^k}_{\text{нулевых столбцов}} \quad \underbrace{2^{k-1}}_{\text{столбцов}} \quad \underbrace{2^{k-1}}_{\text{столбцов}} \quad \underbrace{2^n - (j+1) \cdot 2^k}_{\text{нулевых столбцов}}$

Таким образом, вычисление ДПФ по описанной выше схеме сводится к следующему:

1. Перестановка элементов входного вектора $\mathbf{X}$ с двоично-инверсными индексами.
2. Вычисление ДПФ с помощью тройного цикла:
 - а) цикл по матрицам $\mathbf{W}_k$, $k = 1, \dots, n$;
 - б) цикл по блокам $\mathbf{B}_k^{(j)}$ матрицы $\mathbf{W}_k$, $j = 0, \dots, 2^{n-k} - 1$;
 - в) цикл по строкам блока $\mathbf{B}_k^{(j)}$:

$$\begin{aligned} y(j \cdot 2^k + l) &= x(j \cdot 2^k + l) + w_k^l \cdot x(j \cdot 2^k + 2^{k-1} + l), \\ y(j \cdot 2^k + 2^{k-1} + l) &= x(j \cdot 2^k + l) - w_k^l \cdot x(j \cdot 2^k + 2^{k-1} + l), \\ l &= 0, \dots, 2^{k-1} - 1. \end{aligned}$$

Такой способ нахождения БПФ позволяет экономить память, т.к. получаемые после выполнения «бабочки» значения $y(j \cdot 2^k + l)$ и $y(j \cdot 2^k + 2^{k-1} + l)$ можно

сохранять на место элементов $x(j \cdot 2^k + l)$ и $x(j \cdot 2^k + 2^{k-1} + l)$, которые больше не участвуют в вычислениях. Следовательно, достаточно выделить память для хранения только одного вектора длиной N , при этом результирующий вектор $\tilde{\mathbf{Y}}$ будет находиться на месте входного вектора $\mathbf{X}$ (преобразование *in-place*). Эта схема вычисления часто используется в устройствах с ограниченным объемом оперативной памяти, например, в цифровых сигнальных процессорах.

Если перерисовать граф на рис. 1 в несколько другом виде (см. рис. 4), то по аналогии с рис. 2 для 8-точечного БПФ с прореживанием по времени получим полный граф вычислений, показанный на рис. 5. Здесь перед вычислением БПФ уже не требуется производить перестановку элементов входного вектора с двоично-инверсными номерами. Таким образом, сокращается время вычисления БПФ. Однако, при этом вектора $\mathbf{X}$ и $\tilde{\mathbf{Y}}$ должны располагаться в разных участках памяти.

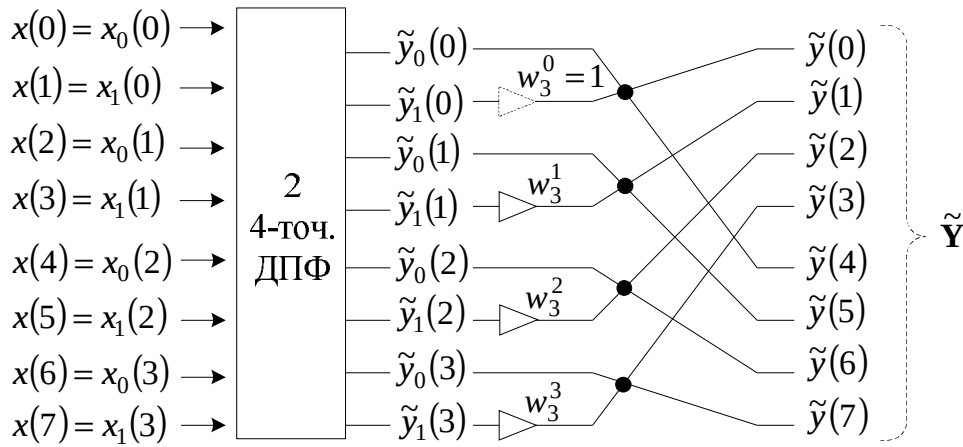


Рис. 4. Граф вычисления 8-точечного ДПФ

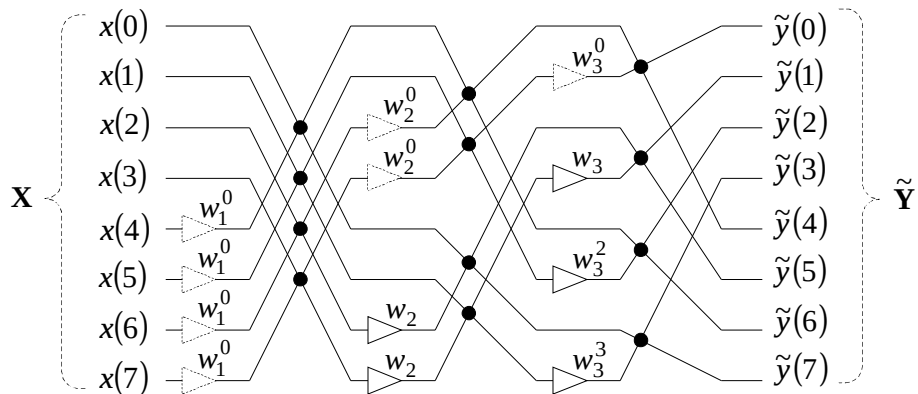


Рис. 5. Полный граф вычисления 8-точечного БПФ с прореживанием по времени

Рассмотрим факторизацию матрицы $\mathbf{W}$ 8-точечного ДПФ для рассмотренной выше схемы. Из графа на рис. 4 получаем:

$$\mathbf{W}_3 = \begin{pmatrix} \boxed{1 & 1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1 & w_3} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \boxed{1 & w_3^2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \boxed{1 & w_3^3} \\ \boxed{1 & -1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1 & -w_3} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \boxed{1 & -w_3^2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \boxed{1 & -w_3^3} \end{pmatrix},$$

далее (см. рис. 5):

$$\mathbf{W}_2 = \begin{pmatrix} \boxed{1 & 0 & 1 & 0} & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \boxed{1 & 0 & w_2 & 0} \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & w_2 \\ \boxed{1 & 0 & -1 & 0} & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \boxed{1 & 0 & -w_2 & 0} \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & -w_2 \end{pmatrix},$$

$$\mathbf{W}_1 = \begin{pmatrix} \boxed{1 & 0 & 0 & 0 & 1 & 0 & 0 & 0} \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ \boxed{1 & 0 & 0 & 0 & -1 & 0 & 0 & 0} \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 \end{pmatrix}.$$

В общем случае $\mathbf{W} = \mathbf{W}_n \cdot \dots \cdot \mathbf{W}_1$, где матрица $\mathbf{W}_k$ имеет вид:

$$\mathbf{W}_k = \begin{pmatrix} \mathbf{B}_k^{(0)}(+), \\ \mathbf{B}_k^{(1)}(+), \\ \mathbb{M} \\ \mathbf{B}_k^{(2^{k-1}-1)}(+), \\ \hline \mathbf{B}_k^{(0)}(-), \\ \mathbf{B}_k^{(1)}(-), \\ \mathbb{M} \\ \mathbf{B}_k^{(2^{k-1}-1)}(-) \end{pmatrix}, \quad k = 1, \dots, n.$$

Каждый блок $\mathbf{B}_k^{(j)}$ может быть записан следующим образом:

$$\mathbf{B}_k^{(j)}(\pm) = \left\{ \begin{array}{cccc|cccc|cccc|cccc} 0 & 0 & 0 & & 1 & 0 & 0 & \mathbf{L} & 0 & \pm w_k^j & 0 & 0 & \mathbf{L} & 0 & 0 & 0 & 0 & 0 \\ 0 & \mathbf{L} & 0 & & 0 & 1 & 0 & \mathbf{L} & 0 & 0 & \pm w_k^j & 0 & \mathbf{L} & 0 & 0 & \mathbf{L} & 0 & 0 \\ 0 & \mathbf{L} & 0 & & 0 & 0 & 1 & \mathbf{L} & 0 & 0 & 0 & \pm w_k^j & \mathbf{L} & 0 & 0 & \mathbf{L} & 0 & 0 \\ \mathbf{M} & \mathbf{L} & \mathbf{M} & & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} & \mathbf{M} & \mathbf{L} & \mathbf{M} & \mathbf{M} \\ 0 & 0 & 0 & & 0 & 0 & 0 & \mathbf{L} & 1 & 0 & 0 & 0 & \mathbf{L} & \pm w_k^j & 0 & 0 & 0 & 0 \end{array} \right\} \left. \begin{array}{l} \\ \\ \\ \\ \end{array} \right\} 2^{n-k} \text{ строк}$$

$j \cdot 2^{n-k+1}$ нулевых столбцов 2^{n-k} столбцов 2^{n-k} столбцов $2^n - (j+1) \cdot 2^{n-k+1}$ нулевых столбцов

БПФ с прореживанием по частоте

Другая форма алгоритма БПФ получается разбиением входной последовательности $\mathbf{X}$ на две части, x_j и $x_{j+N/2}$, из $\frac{N}{2}$ членов, соответствующих $\frac{N}{2}$ первым и $\frac{N}{2}$ последним отсчетам. Этот метод называется *прореживанием по частоте* [2, с.82]:

$$\tilde{y}(k) = \sum_{j=0}^{2^{n-1}-1} x(j) e^{-\frac{2\pi i}{2^n} kj} + \sum_{j=0}^{2^{n-1}-1} x(j+2^{n-1}) e^{-\frac{2\pi i}{2^n} k(j+2^{n-1})}.$$

Из второй суммы можно выделить множитель $e^{-\frac{2\pi i}{2^n} 2^{n-1} k} = e^{-\pi i k} = (-1)^k$, при этом получим:

$$\left\{ \begin{array}{l} \tilde{y}(2k) = \sum_{j=0}^{2^{n-1}-1} (x(j) + x(j+2^{n-1})) w_{n-1}^{kj}, \\ \tilde{y}(2k+1) = \sum_{j=0}^{2^{n-1}-1} (x(j) - x(j+2^{n-1})) w_{n-1}^{kj} w_n^j, \\ k = 0, \dots, 2^{n-1} - 1. \end{array} \right. \quad (13)$$

Фигурирующие здесь суммы представляют собой ДПФ суммы и разности половин исходной последовательности, при этом разность перед вычислением ДПФ умножается на комплексные экспоненты w_n^j .

Как и в случае прореживания по времени, этот метод может использоваться рекурсивно для вычисления N -точечного ДПФ за $n = \log_2 N$ шагов, на каждом из которых 2^i 2^{n-i} -точечных ДПФ сводятся к 2^{i+1} 2^{n-i-1} -точечным ДПФ и, дополнительно, N сложениям и $\frac{N}{2}$ умножениям. Следовательно, метод прореживания по частоте требует такого же числа операций, как и метод прореживания по времени.

Структура матрицы ДПФ при факторизованном представлении для алгоритма с прореживанием по частоте имеет следующий вид (при наличии двоично-инверсных перестановок):

$$\mathbf{W} = (\mathbf{W}_n \cdot \dots \cdot \mathbf{W}_1 \cdot \mathbf{C})^T = \mathbf{C}^T \cdot \mathbf{W}_1^T \cdot \dots \cdot \mathbf{W}_n^T = \mathbf{C} \cdot \mathbf{W}_1^T \cdot \dots \cdot \mathbf{W}_n^T,$$

где $\mathbf{W}_k$ – k -я матрица факторизованного представления в алгоритме с прореживанием по времени,

$$\mathbf{W}_k^T = \left((\mathbf{B}_k^{(0)})^T, \dots, (\mathbf{B}_k^{(2^{n-k}-1)})^T \right),$$

$$(\mathbf{B}_k^{(j)})^T = \left(\begin{array}{cc|cc|cc|cc|cc} 0 & 0 & 0 & \mathbf{L} & 0 & 0 & 0 & 0 & \mathbf{L} & 0 \\ \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} \\ 0 & 0 & 0 & \mathbf{L} & 0 & 0 & 0 & 0 & \mathbf{L} & 0 \\ \hline 1 & 0 & 0 & \mathbf{L} & 0 & 1 & 0 & 0 & \mathbf{L} & 0 \\ 0 & 1 & 0 & \mathbf{L} & 0 & 0 & 1 & 0 & \mathbf{L} & 0 \\ 0 & 0 & 1 & \mathbf{L} & 0 & 0 & 0 & 1 & \mathbf{L} & 0 \\ \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} \\ 0 & 0 & 0 & \mathbf{L} & 1 & 0 & 0 & 0 & \mathbf{L} & 1 \\ \hline 1 & 0 & 0 & \mathbf{L} & 0 & -1 & 0 & 0 & \mathbf{L} & 0 \\ 0 & w_k & 0 & \mathbf{L} & 0 & 0 & -w_k & 0 & \mathbf{L} & 0 \\ 0 & 0 & w_k^2 & \mathbf{L} & 0 & 0 & 0 & -w_k^2 & \mathbf{L} & 0 \\ \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{O} & \mathbf{M} \\ 0 & 0 & 0 & \mathbf{L} & w_k^{2^{k-1}-1} & 0 & 0 & 0 & \mathbf{L} & -w_k^{2^{k-1}-1} \\ \hline 0 & 0 & 0 & \mathbf{L} & 0 & 0 & 0 & 0 & \mathbf{L} & 0 \\ \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} & \mathbf{M} \\ 0 & 0 & 0 & \mathbf{L} & 0 & 0 & 0 & 0 & \mathbf{L} & 0 \end{array} \right) \begin{array}{l} \left. \begin{array}{c} j \cdot 2^k \\ \text{нулевых} \\ \text{строк} \end{array} \right\} \\ \left. \begin{array}{c} 2^{k-1} \\ \text{строк} \end{array} \right\} \\ \left. \begin{array}{c} 2^{k-1} \\ \text{строк} \end{array} \right\} \\ \left. \begin{array}{c} 2^n - (j+1) \cdot 2^k \\ \text{нулевых} \\ \text{строк} \end{array} \right\} \end{array}$$

$$\underbrace{\hspace{10em}}_{\substack{2^{k-1} \\ \text{столбцов}}} \quad \underbrace{\hspace{10em}}_{\substack{2^{k-1} \\ \text{столбцов}}}$$

При вычислении БПФ без перестановок (см. выше), факторизованное представление матрицы ДПФ записывается аналогично.

Обратное ДПФ

Все проведённые рассуждения для прямого ДПФ (10) можно в точности повторить для обратного ДПФ, вместо $w_j = \exp\left(-\frac{2\pi i}{2^j}\right)$ положив $w_j = \exp\left(\frac{2\pi i}{2^j}\right)$, верными останутся и аналитическое выражение (11), и примеры графов вычислений, приведённые на рис. 2 и 5. Кроме того, умножение на матрицу $\overline{\mathbf{W}}$

можно реализовать, используя вычислительную процедуру реализации ДПФ $\tilde{Y} = \mathbf{W}X$ (см. [1]).

Дискретная свертка

Наряду с ДПФ, одной из стандартных и наиболее распространенных вычислительных процедур в цифровой обработке сигналов является *дискретная свертка*. С ее помощью вычисляются корреляционная и автокорреляционная функции, производится цифровая фильтрация сигналов. Многие модели сигнальных процессоров (DSP) специально оптимизируются для эффективного выполнения свертки.

Дискретной сверткой бесконечных последовательностей $x(n)$ и $y(n)$ называется последовательность $u(n)$, обозначаемая $u(n) = x(n) * y(n)$, элементы которой находятся по формуле:

$$u(n) = \sum_{k=-\infty}^{\infty} x(k)y(n-k). \quad (14)$$

При этом имеет место равенство

$$u(n) = \sum_{k=-\infty}^{\infty} x(k)y(n-k) = \sum_{k=0}^n x(k)y(n-k) = \sum_{k=0}^n y(k)x(n-k) = \sum_{k=-\infty}^{\infty} y(k)x(n-k),$$

так как считаем, что при $n < 0$ $x(n) = y(n) = 0$ и, следовательно, $u(n) = 0$.

Вычисление свертки с помощью БПФ

Пусть M – длина последовательности $x(n)$, т.е. $x(n) = 0$ при $n \geq M$, а L – длина последовательности $y(n)$: $y(n) = 0$, $n \geq L$. Дискретная свертка этих

последовательностей $y(n) = \sum_{m=0}^n x(m)y(n-m) = \sum_{m=0}^n x(m)y(n-m)$ будет иметь длину

$M + L - 1$, так как для $n \geq M + L - 1$ приведенная формула свертки дает $u(n) = 0$, а для $n = M + L - 2$ в общем случае $y(M + L - 2) = x(M - 1)y(L - 1) \neq 0$.

Использование БПФ для вычисления свертки основано на том, что ДПФ свертки последовательностей есть покомпонентное произведение ДПФ соответствующих последовательностей [1]. Рассмотрим процедуру нахождения свертки с помощью БПФ.

Добавлением нулевых отсчетов сформируем векторы одинаковой размерности $2N \geq \max(2L, 2M)$ (обычно $N = 2^n$):

$$X = \begin{pmatrix} x(0) & x(1) & \dots & x(M-1) & 0 & 0 & \dots & 0 \\ \mathbf{1} & \mathbf{4} & \mathbf{4} & \mathbf{4} & \mathbf{4} & \mathbf{2} & \mathbf{4} & \mathbf{4} & \mathbf{4} & \mathbf{3} \end{pmatrix}_{2N}, \quad Y = \begin{pmatrix} y(0) & y(1) & \dots & y(L-1) & 0 & 0 & \dots & 0 \\ \mathbf{1} & \mathbf{4} & \mathbf{4} & \mathbf{4} & \mathbf{4} & \mathbf{2} & \mathbf{4} & \mathbf{4} & \mathbf{4} & \mathbf{3} \end{pmatrix}_{2N}$$

Затем над этими векторами выполним следующие действия:

1. БПФ: $X \rightarrow \hat{X}$ и $Y \rightarrow \hat{Y}$.
2. Покомпонентное перемножение полученных дискретных спектров:

$$\hat{U} = \sqrt{2N}(\hat{x}_0\hat{y}_0, \dots, \hat{x}_{2N-1}\hat{y}_{2N-1}).$$
3. Обратное БПФ: $\hat{U} \rightarrow U$.

В полученном векторе размерности $2N$ первые $M+L-1$ компонент представляют собой свертку $u(n)$ последовательностей $x(n)$ и $y(n)$, а остальные компоненты – нулевые:

$$U = \left(\underbrace{u(0), u(1), \dots, u(L-1)}_L, \underbrace{u(L), u(L+1), \dots, u(L+M-1)}_{M-1}, 0, 0, \dots, 0 \right)$$

Использование описанной процедуры в вычислительном плане может быть более эффективно, чем непосредственная реализация формулы свертки (14).

Задание

1. Реализовать на C или C++ алгоритмы непосредственного вычисления ДПФ и ОДПФ по формулам (8) и (9) для комплексного входного сигнала с двойной точностью (double). Входные данные загружать из текстового файла (разделитель – пробел), сгенерированного, например, в MATLAB.
2. Реализовать на C или C++ алгоритмы прямого и обратного БПФ для комплексного входного сигнала длиной 2^n , n – любое натуральное число:
 - а) с прореживанием по времени и двоично-инверсными перестановками (вариант 1);
 - б) с прореживанием по времени без двоично-инверсных перестановок (вариант 2);
 - в) с прореживанием по частоте и двоично-инверсными перестановками (вариант 3);
 - г) с прореживанием по частоте без двоично-инверсных перестановок (вариант 4);
3. Убедиться в корректности работы алгоритмов:
 - а) проверить выполнение равенства $X = \text{ОДПФ}(\text{ДПФ}(X))$;
 - б) сравнить результаты работы реализованного алгоритма, например, с результатами процедуры `fft`, встроенной в MATLAB.
4. Проанализировать зависимость времени выполнения БПФ и непосредственного вычисления ДПФ от длины N преобразования.

5. Реализовать на С или С++ процедуру прямого вычисления свертки двух последовательностей по формуле (14). Входные данные загружать из текстового файла (разделитель – пробел), сгенерированного, например, в MATLAB.
6. Реализовать процедуру нахождения дискретной свертки, основанную на БПФ. При вычислении БПФ использовать результаты п. 2 задания.
7. Убедиться в корректности работы процедуры из п. 6 задания, сравнив полученные результаты с результатами работы встроенной функций MATLAB conv.
8. Сравнить производительность алгоритмов вычисления свертки по определению (14) и с помощью БПФ в двух случаях: когда размер одной из последовательностей фиксирован, и когда меняются длины обеих последовательностей.

Контрольные вопросы

1. Дать определение ДПФ и ОДПФ.
2. Перечислить основные свойства ДПФ.
3. Что такое матрица ДПФ $\mathbf{W}$? Доказать унитарность матрицы ДПФ.
4. Что такое факторизация матрицы ДПФ?
5. Как выглядит матрица $\mathbf{W}_3$ для 8-точечного преобразования?
6. Как можно дополнительно ускорить рассмотренные алгоритмы БПФ?
7. Что такое дискретная свертка последовательностей?
8. Доказать, что ДПФ свертки последовательностей есть покомпонентное произведение ДПФ этих последовательностей.

Литература

1. Умняшкин С.В. Теоретические основы цифровой обработки и представления сигналов.
2. Нуссбаумер Г. Быстрое преобразование Фурье и алгоритмы вычисления свертки: Пер. с. Англ. – М.: Радио и связь, 1985. – 248 с., ил.
3. Ефимов А.В., Лесин В.В. Методические разработки по курсу «Теория алгоритмов и вычислительные методы». – М.: МИЭТ, 1982.

Лабораторная работа №3. Линейные дискретные фильтры. Синтез КИХ-фильтров по заданной частотной характеристике

Цель работы

Ознакомиться с основными понятиями и методами дискретной фильтрации. Изучить метод построения линейных дискретных фильтров по заданной частотной характеристике.

Теоретические сведения

Линейные дискретные фильтры

Линейным дискретным фильтром (ЛДФ) или линейной дискретной системой называется устройство, которое преобразует входную последовательность $x(n)$ в выходную $y(n)$ по правилу, определяемому для любого целого значения n следующим разностным уравнением:

$$\sum_{m=0}^M a_m y(n-m) = \sum_{k=0}^N b_k x(n-k), \quad (15)$$

где a_m, b_k – константы (вообще говоря, комплексные). При этом $y(n)$ называют реакцией или откликом фильтра на входное воздействие $x(n)$.

Положив $a_0 = 1$ (при необходимости все коэффициенты разностного уравнения (15) делятся на a_0), уравнение (15) можно переписать в другом виде:

$$y(n) = -\sum_{m=1}^M a_m y(n-m) + \sum_{k=0}^N b_k x(n-k), \quad (16)$$

что задает в виде рекуррентной формулы некоторую расчётную процедуру нахождения очередного элемента выходной последовательности по предыдущим элементам входной и выходной последовательностей.

Если в уравнении (15) или (16) хотя бы один коэффициент $a_m \neq 0$, то фильтр называется *рекурсивным*, в противном случае – *нерекурсивным*.

Реакция (отклик) ЛДФ на единичное воздействие $x(n) = \tilde{d}(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$ называется *импульсной характеристикой* (ИХ) ЛДФ, обозначаемой $h(n)$: $h(n) = y(n)|_{x(n)=\tilde{d}(n)}$.

Фильтры с конечной ИХ ($\exists M : \forall m > M \ h(m) = 0$) называют *КИХ-фильтрами* (*FIR filters*), с бесконечной – *БИХ-фильтрами* (*IIR filters*).

Зная ИХ фильтра $h(n)$, можно найти реакцию фильтра $y(n)$ на произвольное входное воздействие $x(n)$, представив входной сигнал в виде $x(n) = \sum_{m=0}^{\infty} x(m) \tilde{d}(n-m)$. В силу стационарности (инвариантности во времени

свойств фильтра) реакция фильтра на сигнал $\tilde{d}(n-m)$ ($n=0,1,\dots$) представляет собой последовательность $h(n-m)$ ($n=0,1,\dots$). Тогда в силу линейности фильтра реакция на сигнал $x(n) = \sum_{m=0}^{\infty} x(m) \tilde{d}(n-m)$ есть $y(n) = \sum_{m=0}^{\infty} x(m) h(n-m)$. Таким образом, по известной ИХ отклик фильтра находится по формуле дискретной свёртки:

$$y(n) = \sum_{m=0}^{\infty} x(m) h(n-m) = \sum_{m=0}^n x(m) h(n-m) = \sum_{m=0}^n h(m) x(n-m). \quad (17)$$

Набор отсчетов импульсной характеристики, как и набор коэффициентов a_m , b_k , полностью определяет фильтр.

Нерекурсивные фильтры всегда являются фильтрами с КИХ. Чтобы синтезировать КИХ-фильтр по заданной ИХ $h(n) = \{h(0), h(1), \dots, h(N), 0, 0, \dots\}$, нужно выбрать коэффициенты $\{b_k = h(k)\}_{k=0}^N$.

Передаточной функцией ЛДФ называется отношение Z -образов выходной и входной последовательностей:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^N b_k z^{-k}}{1 + \sum_{m=1}^M a_m z^{-m}}. \quad (18)$$

Передаточная функция не зависит от входного воздействия и является Z -преобразованием импульсной характеристики $h(n)$ фильтра: $H(z) = Z\{h(n)\}$.

Порядком фильтра называется максимальное из чисел N и M .

Частотной характеристикой (ЧХ) ЛДФ называется комплексная функция вещественной переменной ω : $K(\omega) = H(e^{i\omega})$, которая получается при подстановке в передающую функцию аргумента $z = e^{i\omega}$. Физический смысл частотной характеристики – комплексный коэффициент передачи гармонических колебаний: $K(\omega)$ характеризует амплитуду и фазу (в установившемся режиме) на выходе ЛДФ при поданном на вход гармоническом колебании с циклической частотой ω и единичной амплитудой. Если входное воздействие представляет собой отсчёты комплексного гармонического сигнала, а именно, $x(n) = e^{i\omega n}$ ($n > 0$), то по формуле свёртки получим следующий отклик ЛДФ:

$$y(n) = \sum_{k=0}^n h(k) e^{j\omega(n-k)} = e^{j\omega n} \sum_{k=0}^n h(k) e^{-j\omega k} \xrightarrow{n \rightarrow \infty} K(\omega) e^{j\omega n}. \quad (19)$$

То есть в установившемся режиме (при больших значениях n) на выходе ЛДФ получим гармоническое колебание той же частоты, но с другой амплитудой и фазой, изменение которых определяется ЧХ: $\frac{y(n)}{x(n)} = K(\omega)$. ЧХ может быть записана в показательной форме: $K(\omega) = |K(\omega)| e^{-ij(\omega)}$, тогда изменение амплитуды гармонического колебания на выходе ЛДФ в установившемся режиме характеризуется модулем частотной характеристики или *амплитудно-частотной характеристикой (АЧХ)* $|K(\omega)|$, а фазы – *фазочастотной характеристикой (ФЧХ)* $j(\omega) = -\arg(K(\omega))$.

Учитывая, что $h(n) = 0$ при $n < 0$ и переходя к линейной частоте $\omega = \frac{p}{2}$, ЧХ можно записать в виде: $K_f(p) = \sum_{k=-\infty}^{\infty} h(k) e^{-2\pi i n k}$. ЧХ $K_f(p)$ представляет собой частотный спектр дискретного сигнала – импульсной характеристики $h(n)$ для единичного интервала дискретизации $\Delta t = 1$. В частотной области частота сигнала $F_{\max} = \frac{1}{2\Delta t}$ соответствует нормированной частоте $\omega = \frac{1}{2}$ ($\omega = p$), а период ЧХ (спектра ИХ), в соответствии со свойством периодичности спектра дискретного сигнала в частотной области, будет единичным для $K_f(p)$ и равным $2p$ для $K(\omega)$.

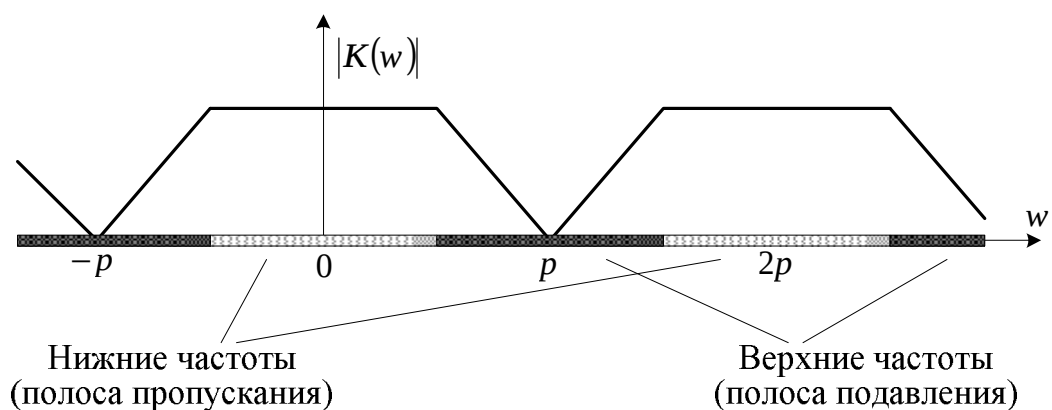


Рис. 6. График АЧХ фильтра нижних частот

На рис. 6 представлен схематический график для нормированной АЧХ $|K(\omega)|$, соответствующей фильтру нижних частот.

Синтез КИХ-фильтров

На практике часто возникает необходимость поиска таких параметров фильтра $\{a_n\}$, $\{b_n\}$, которые с заданной точностью обеспечивают требуемую частотную характеристику (ЧХ). Соответствующая задача *синтеза фильтра* должна решаться так, чтобы в структуре фильтра можно было получить минимальное число элементов задержки и умножителей. Обычно это требует использования рекурсивных схем, однако методика синтеза рекурсивных фильтров более сложна и обычно заключается в преобразовании непрерывного фильтра в дискретный, обладающий необходимыми спецификациями (подробнее см., например, [3]).

Рассмотрим синтез фильтра по заданной АЧХ $|K_0(w)|$ на примере нерекурсивного *симметричного* фильтра, параметры которого $\{b_n = h(n)\}_{n=0}^{2N}$ (см. [1]) таковы, что $b_{N-k} = b_{N+k}$, $k = 0, \dots, N$. Порядок данного фильтра будет равен $2N$. Для ЧХ симметричного КИХ-фильтра получим:

$$\begin{aligned} K(w) &= \sum_{k=0}^{2N} b_k e^{-i w k} = e^{-i w N} \sum_{k=-N}^N b_{N+k} e^{-i w k} = \\ &= e^{-i w N} \left(b_N + \sum_{k=1}^N (b_{N+k} e^{-i w k} + b_{N-k} e^{i w k}) \right), \end{aligned}$$

тогда окончательно

$$K(w) = e^{-i w N} \left(b_N + 2 \sum_{k=1}^N b_{N+k} \cos w k \right). \quad (20)$$

Коэффициенты фильтра $\{b_{N+k}\}_{k=0}^N$, должны быть такими, чтобы ЧХ синтезированного фильтра $K(w)$ хорошо аппроксимировала требуемую ЧХ $K_0(w)$. Поскольку для фильтров с вещественными коэффициентами $|K(w)| = |K(-w)|$, то с учетом 2π -периодичности ЧХ для определения АЧХ достаточно задать последнюю на отрезке $w \in [0, \pi]$.

Для нахождения коэффициентов фильтра $\{b_{N+k}\}_{k=0}^N$ приравняем значения требуемой АЧХ $|K_0(w)|$ к значениям АЧХ $|K(w)|$ синтезируемого фильтра в некоторых точках $w_j \in [0, \pi]$, $j = 0, \dots, N$. Искомые коэффициенты $\{b_{N+k}\}_{k=0}^N$ могут быть найдены (см. (20)) из решения следующей системы линейных уравнений:

$$\begin{aligned} b_N + 2 \sum_{k=1}^N b_{N+k} \cos w_j k &= |K_0(w_j)|, \\ j &= 0, \dots, N. \end{aligned} \quad (21)$$

Тогда полученное из формулы (20) соотношение

$$|K(w)| = \left| b_N + 2 \sum_{k=1}^N b_{N+k} \cos wk \right| \quad (22)$$

можно рассматривать как интерполяционную формулу для требуемой АЧХ $|K_0(w)|$, точки w_j в (21) задают узлы интерполяции. Вопрос оптимального расположения узлов представляет собой отдельную задачу. Простейшим (очевидно, не лучшим) решением этой задачи является равномерное расположение точек $w_j \in [0; p]$: $w_j = p(j+0,5)/(N+1)$, $j = 0, \dots, N$. В этом случае из (21) получаем систему:

$$b_N + 2 \sum_{k=1}^N b_{N+k} \cos \frac{pk(j+0,5)}{N+1} = \left| K_0 \left(\frac{p(j+0,5)}{N+1} \right) \right|, \\ j = 0, \dots, N,$$

решение которой находится как

$$b_{N+k} = \frac{1}{N+1} \sum_{j=0}^N \left| K_0 \left(\frac{p(j+0,5)}{N+1} \right) \right| \cos \frac{pk(j+0,5)}{N+1}, \\ k = 0, \dots, N. \quad (23)$$

При аппроксимации разрывной функции конечной тригонометрической суммой вида (20) всегда возникает «всплеск» вблизи точек разрыва (*эффект Гиббса*, см. рис. 7).

Для лучшего приближения тригонометрическими рядами аппроксимируемая функция должна быть как можно более гладкой, т.е. быть непрерывной и по возможности иметь непрерывные производные (все) до как можно более высокого порядка. На практике при задании требований к АЧХ полосового фильтра проектировщик, формулируя ограничения для неравномерности полосы пропускания e_0 и для полосы подавления e_1 , должен определить допустимый «переходный» участок частот, который разделяет полосы пропускания и подавления, см. рис. 8.

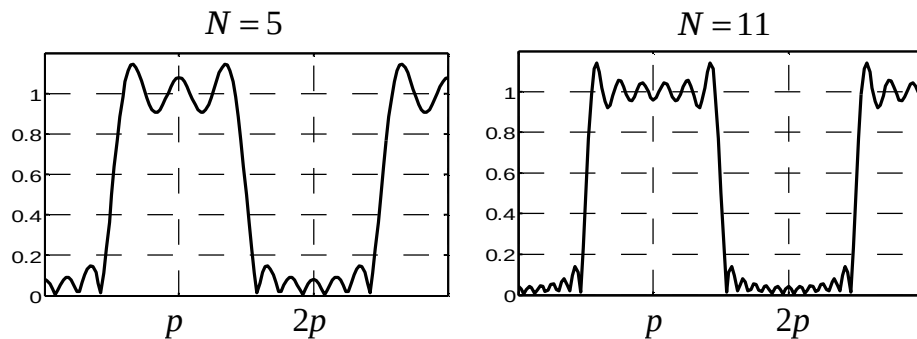


Рис. 7. АЧХ фильтров порядков $2N=10$ и $2N=22$, синтезированных по заданной

$$\text{идеальной АЧХ } |K_0(w)| = \begin{cases} 1, & w \in [p/2; 3p/2] \\ 0, & w \in [0; p/2) \cup [3p/2; 2p) \end{cases}$$

Требуемая АЧХ синтезируемого фильтра задается уже в виде некоторого допустимого «коридора» для области возможных значений $|K(w)|$. Тогда можно подобрать более гладкую функцию $|K_0(w)|$ для последующей аппроксимации функцией $|K(w)|$, которая определяется формулой (20). Например, для узкополосного фильтра с полосой пропускания $w \in [w_0 - d; w_0 + d;]$ (где d – мало) удобно выбирать $|K_0(w)| = \exp(-c(w - w_0)^2)$. Обычно требуется обеспечить наилучшее равномерное приближение АЧХ: $\max_{w \in [0; p]} \|K(w) - |K_0(w)|\| \rightarrow \min$.

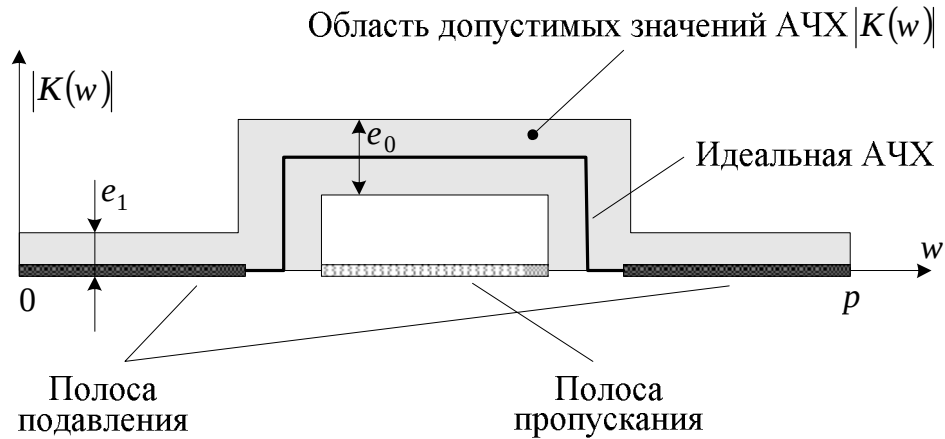


Рис. 8. Задание ограничений для АЧХ

Однако, конечная цель синтеза фильтра состоит не в поиске приемлемой функции $|K_0(w)|$, а в том, чтобы найти такие коэффициенты фильтра (минимально возможного порядка) $\{b_k\}$, чтобы реальная АЧХ $|K(w)|$ не выходила за пределы заданного «коридора». Один из классических подходов к решению данной задачи состоит в следующем [3]. Задавшись некоторым порядком фильтра, равномерно расположим в частотной области отсчеты $w_j \in [0; p]$, $j = 0, \dots, N$. Для тех отсчетов, которые попали в полосу подавления, приравняем значение частотной характеристики к нулю; для отсчетов, попавших в полосу пропускания, приравняем АЧХ к константе – требуемому коэффициенту передачи. Значения АЧХ для тех отсчетов, которые попали в «переходную» область, считаем свободными переменными, которые выбираем так, чтобы в области пропускания и в области подавления реальная АЧХ была наиболее близка к требуемой. Если точность полученной аппроксимации оказалась избыточной, рассматриваем данную задачу для фильтра меньшего порядка, а если обеспечить требуемую аппроксимацию не удалось, то повышаем порядок синтезируемого фильтра. Выбор значений АЧХ в «переходной» области сводится к оптимизационной задаче, решаемой численными методами.

Рассмотрим возможность приближения идеальной АЧХ для примера на рис. 7 при отсутствии ограничений на значения $|K_0(w)|$ в двух ближайших к частоте среза ($w = p/2$) точках интерполяции. Положим $\left|K_0\left(\frac{p(j+0,5)}{N+1}\right)\right| = 0$ (полоса подавления) для $j = 0, \dots, \frac{N-3}{2}$ и $\left|K_0\left(\frac{p(j+0,5)}{N+1}\right)\right| = 1$ (полоса пропускания) для $j = \frac{N+3}{2}, \dots, N$. Если выбрать значения АЧХ в точках $w_{\frac{N-1}{2}} = \frac{p}{2} \left(\frac{N}{N+1}\right)$ и $w_{\frac{N+1}{2}} = \frac{p}{2} \left(\frac{N+2}{N+1}\right)$ равными соответственно $\left|K_0\left(w_{\frac{N-1}{2}}\right)\right| = 0,16$ и $\left|K_0\left(w_{\frac{N+1}{2}}\right)\right| = 0,84$, то для значений $N = 5, N = 11$ получим АЧХ $|K(w)|$, графики которых приведены на рис. 9.

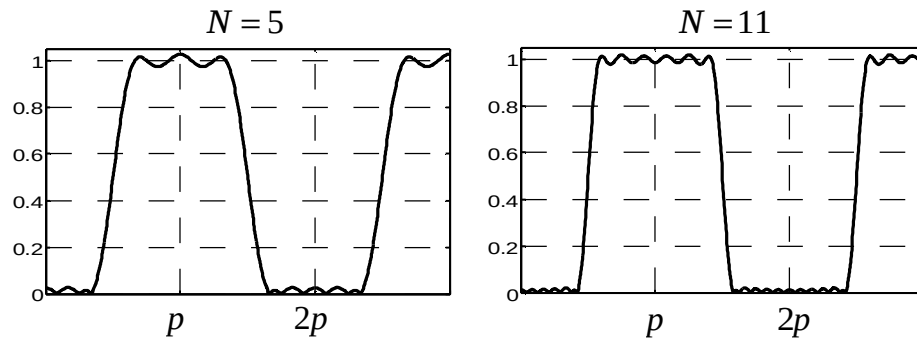


Рис. 9. АЧХ фильтров порядков $2N = 10$ и $2N = 22$

Находя параметры нерекурсивного фильтра решением системы (21), в области полосы пропускания фильтра получаем (см. (20)), что $K(w) = e^{-iwn} |K(w)|$, т.е. ФЧХ $j(w) = wn + 2pt$, где целое число t выбирается из условия $j(w) \in (-p; p]$. ФЧХ вида $j(w) = aw + 2pt$, ($a \geq 0, t \in \mathbb{Z}$) называется *линейной*. Такая ФЧХ гарантирует, что гармонические колебания $x(n) = A \cos(\omega n - g)$ любой частоты w при прохождении через фильтр получают одинаковую временную задержку a . На выходе системы получаем отклик:

$$y(n) = A |K(w)| \cos(\omega n - g - a\omega - 2pt) = |K(w)| x(n - a). \quad (24)$$

Различия наблюдаются только в амплитуде колебаний, изменение которой определяется АЧХ. Свойство линейности ФЧХ очень часто является необходимым требованием, предъявляемым к синтезируемому фильтру.

Задание

1. Синтезировать нерекурсивный симметричный фильтр порядка $2N$ с заданной в таблице вариантов полосой пропускания (см. (22) и (23)). Построить

амплитудно-частотную и фазочастотную характеристики. Сравнить полученную АЧХ с идеальной.

2. Синтезировать нерекурсивный симметричный фильтр порядка $2N$ с заданной в таблице вариантов полосой пропускания, используя процедуру `firpm` пакета MATLAB (для версий MATLAB 6.5 и ниже следует использовать процедуру `remez`). Построить амплитудно-частотную и фазочастотную характеристики. Сравнить полученную АЧХ с идеальной.
3. Реализовать в MATLAB функцию синтеза фильтра (т.е. нахождения коэффициентов $\{b_k\}_{k=0}^{2N}$), которая обеспечивает наилучшее равномерное приближение заданной АЧХ и принимает в качестве аргументов:
 - а) положение полосы пропускания проектируемого фильтра на оси ω ,
 - б) положение полосы подавления проектируемого фильтра на оси ω ,
 - в) параметр N .

С помощью реализованной функции синтезировать фильтр того же порядка, что и в п. 1. Сравнить полученную АЧХ с идеальной и с АЧХ, найденной в п. 1. Для решения задачи оптимизации можно использовать встроенные функции MATLAB, например, `fmincon`.

4. Проверить, удовлетворяет ли фильтр, синтезированный в п. 3, требованиям для неравномерности полос пропускания и подавления при заданных параметрах ϵ_0 и ϵ_1 . Определить минимальный порядок фильтра ($2N$), удовлетворяющего данным требованиям.
5. С помощью синтезированного в п. 3 фильтра обработать сигналы $x(t) = \sin \omega t$ для указанных в таблице вариантов значений ω . Определить задержку a гармонического колебания на выходе фильтра (см. (24)).
6. С помощью синтезированного в п. 3 фильтра провести фильтрацию тестового изображения. Для этого профильтровать последовательно каждую строку, затем каждый столбец изображения. Объяснить полученный результат.

В заданиях 1-4 амплитуду на графиках АЧХ необходимо также выражать в децибелах, т.е. помимо графика зависимости $|K(\omega)|$ нужно приводить график

$20 \lg \left| \frac{K(\omega)}{C} \right|$, где C – коэффициент усиления в полосе пропускания (в рассмотренных выше примерах $C=1$). Для этого значения АЧХ в точках, где $|K(\omega)| \leq 10^{-7}$, принять равными 10^{-7} .

Варианты заданий

№ варианта	N	Полоса пропускания	Полоса подавления	e_0	e_1	w
1	5	$[0; 0,6p)$	$[0,8p; p)$	0,025	0,015	$\{0,3p; 0,5p\}$
2	7	$[0,5p; p)$	$[0; 0,4p)$	0,055	0,020	$\{0,6p; 0,9p\}$
3	6	$[0; 0,55p)$	$[0,7p; p)$	0,020	0,060	$\{0,2p; 0,45p\}$
4	8	$[0,35p; p)$	$[0; 0,25p)$	0,015	0,030	$\{0,5p; 1,2p\}$

Контрольные вопросы

1. Дать определение ЛДФ.
2. Каковы основные характеристики линейных дискретных систем?
3. Показать взаимосвязь передаточной и импульсной характеристик.
4. Каков физический смысл АЧХ и ФЧХ?
5. Сформулировать задачу синтеза ЛДФ.
6. Сформулировать задачу оптимизации, возникающую при синтезе фильтра по описанному выше методу.

Литература

1. Умняшкин С.В. Теоретические основы цифровой обработки и представления сигналов.
2. А. Оппенгейм, Р. Шафер. Цифровая обработка сигналов. – М.: Техносфера, 2006. – 856 с.
3. Рабинер Л., Голд Б. Теория и применение цифровой обработки сигналов. – М: Мир, 1978. – 848 с.

Лабораторная работа №4. Словарные методы сжатия данных. Алгоритм LZW

Цель работы

Ознакомится со словарными методами кодирования данных, изучить алгоритм LZW и исследовать особенности его реализации.

Теоретические сведения

Алгоритмы сжатия данных

Существует множество алгоритмов сжатия данных без потерь, часть которых является открытыми, а другая часть – коммерческими алгоритмами. Открытые алгоритмы обычно рассматривают только саму идею, не останавливаясь на проблемах ее реализации в виде программы, или реализованы в виде «демонстрационной» (не очень эффективной) программы. Коммерческие алгоритмы, как правило, включают в себя не только идею алгоритма сжатия, но и ее эффективную реализацию в виде программы (например, такие программы как ZIP, ARJ, RAR, ACE и др.).

Алгоритмы сжатия данных без потерь можно разделить на две группы:

1. *Алгоритмы частотного анализа* – подсчет частоты различных символов в данных и преобразование кодов символов с соответствии с их частотой.
2. *Алгоритмы корреляционного анализа* – поиск корреляций (в простейшем случае точных повторов) между различными участками данных и замена коррелирующих данных на код(ы), позволяющая восстановить данные на основе предшествующих данных.

В первой группе можно выделить следующие алгоритмы сжатия данных:

1. *Метод Хаффмана* – замена кода равной длины для символов на коды неравной длины в соответствии с частотой появления символов, коды минимальной длины присваиваются наиболее часто встречающимся символам. Если частоты появления символов являются степенью двойки (2^n), то этот метод достигает теоретической энтропийной границы эффективности сжатия для методов такого типа.
2. *Метод Шеннона-Фано* – сходен с методом Хаффмана, но использует другой алгоритм построения кодов и не всегда дает оптимальные коды.
3. *Арифметическое кодирование* – метод, позволяющий получать близкие к минимально возможным коды для данных, где частоты появления символов не являются степенью двойки (2^n). Его суть состоит в том, что

последовательности символов ставится в соответствие некоторое число, однозначно задающее данную последовательность.

Для второй группы можно назвать следующие алгоритмы:

1. *Метод кодирования длин серий (run-length encoding, RLE)* – заменяет цепочки повторяющихся символов на код символа и число повторов. Это пример элементарного и очень понятного метода сжатия, но, к сожалению, он не обладает достаточной эффективностью.
2. *Методы Зива-Лемпела (LZ)* – это группа алгоритмов сжатия, объединенная общей идеей: поиск повторов фрагментов текста в данных и замена повторов ссылкой (кодом) на первое (или предыдущее) вхождение этого фрагмента в данные. Отличаются друг от друга методом поиска фрагментов и методом построения ссылок (кодов).

Универсальные программные архиваторы, как правило, используют алгоритмы и частотного, и корреляционного анализа.

Словарные методы сжатия

Входную последовательность символов можно рассматривать как последовательность *фраз*, содержащих произвольное количество символов. Идея *словарных методов* состоит в замене фраз исходной последовательности на индексы фраз из некоторого словаря, известного и кодеру, и декодеру. При декодировании осуществляется обратная замена индекса на соответствующую ему фразу словаря.

Словарь – это набор таких фраз, которые, как предполагается, будут встречаться во входной последовательности. Индексы фраз должны быть построены таким образом, чтобы в среднем их представление занимало меньше места, чем требуют замещаемые строки.

Уменьшение размера возможно в первую очередь за счет того, что обычно в сжимаемых данных встречается лишь небольшая часть всех возможных строк длины n , поэтому для представления индекса фразы требуется, как правило, меньшее число битов, чем для представления исходной строки. Кроме того, если существуют заслуживающие доверия гипотезы о частоте использования тех или иных фраз, либо проводился какой-то частотный анализ обрабатываемых данных, то можно назначить более вероятным фразам коды меньшей длины. Обычно же просто предполагается, что короткие фразы используются чаще длинных. Поэтому в большинстве случаев индексы строятся таким образом, чтобы длина индекса короткой фразы была меньше длины индекса длинной фразы.

Классические алгоритмы Зива-Лемпела

Алгоритмы *словарного сжатия Зива-Лемпела* появились во второй половине 1970-х годов. Это были так называемые алгоритмы LZ77 и LZ78, разработанные совместно Зивом (Ziv) и Лемпелом (Lempel). В дальнейшем первоначальные схемы подвергались множественным изменениям, в результате чего на сегодняшний день существуют десятки достаточно самостоятельных алгоритмов и бесчисленное количество модификаций.

LZ77 и LZ78 являются универсальными алгоритмами сжатия, в которых словарь формируется на основании уже обработанной части входного потока, т.е. адаптивно. Принципиальным отличием является лишь способ формирования фраз. Словарные алгоритмы Зива-Лемпела разделяют на два семейства – алгоритмы типа LZ77 и алгоритмы типа LZ78. Иногда также говорят о словарных методах LZ1 и LZ2. Методы данного семейства являются одними из самых популярных среди всех методов сжатия данных. Кроме того, практически все реально используемые словарные алгоритмы относятся к семейству Зива-Лемпела.

Метод LZ77

Этот словарный алгоритм сжатия является самым старым среди методов LZ. Описание было опубликовано в 1977 году, но сам алгоритм разработан не позднее 1975 года. Алгоритм LZ77 является «родоначальником» целого семейства словарных схем – так называемых алгоритмов со *скользящим словарем*, или *скользящим окном*. В LZ77 в качестве словаря используется блок уже закодированной последовательности. Как правило, по мере выполнения обработки положение этого блока относительно начала последовательности постоянно меняется, словарь «скользит» по входному потоку данных.

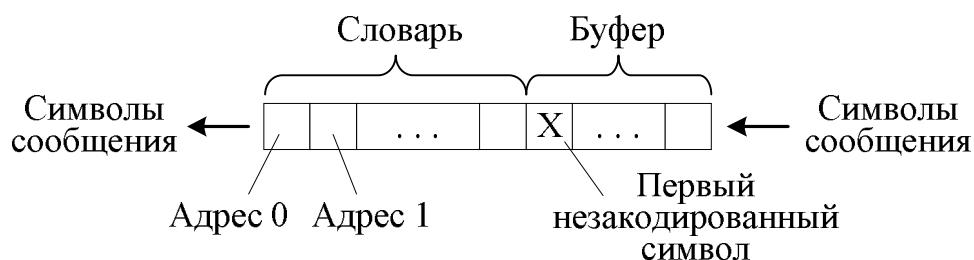


Рис. 10. Скользящее окно

Скользящее окно имеет длину N , т.е. в него помещается N символов, и состоит из 2 частей (см. рис. 10):

- 1) последовательности длины $W = N - n$ уже закодированных символов, которая и является словарем;
- 2) *упреждающего буфера*, или *буфера предварительного просмотра (lookahead)*, длины n . Как правило, n на порядки меньше W (обычно словарь содержит несколько тысяч символов, а буфер – несколько десятков).

Пусть к текущему моменту времени уже закодировано t символов $s_1, s_2, \dots, s_t$. Тогда словарем будут являться W предшествующих символов $s_{t-(W-1)}, s_{t-(W-2)}, \dots, s_t$. Соответственно, в буфере находятся ожидающие кодирования символы $s_{t+1}, s_{t+2}, \dots, s_{t+n}$. Очевидно, что если $W > t$, то словарем будет являться вся уже обработанная часть входной последовательности.

Идея алгоритма заключается в поиске самого длинного совпадения между строкой буфера, начинающейся с символа s_{t+1} и всеми фразами словаря. Эти фразы могут начинаться с любого символа $s_{t-(W-1)}, s_{t-(W-2)}, \dots, s_t$ и выходить за пределы словаря, вторгаясь в область буфера, но должны лежать в окне. Следовательно, фразы не могут начинаться с s_{t+1} поэтому буфер не может сравниваться сам с собой. Длина совпадения не должна превышать размер буфера. Если в результате поиска была найдена фраза $s_{t-(i-1)}, s_{t-(i-1)+1}, \dots, s_{t-(i-1)+(j-1)}$, то она кодируется с помощью трех информационных полей (A, L, s) :

- 1) адрес начала фразы в словаре $A = W - i$;
- 2) длины соответствия, или совпадения (*match length*), $L = j$; смещение и длина соответствия играют роль указателя (ссылки), однозначно определяющего фразу.
- 3) символ s , непосредственно следующий за совпавшей строкой буфера.

Затем окно смещается на $(j+1)$ символов вправо и осуществляется переход к новому циклу кодирования. Такая величина сдвига объясняется тем, что реально был закодирован именно $j+1$ символ: j с помощью указателя на фразу в словаре, и 1 с помощью копирования. Если строка в словаре не была найдена, то единица записи выдается в виде $(0, 0, s)$, где s – первый символ буфера, а окно смещается на одну позицию.

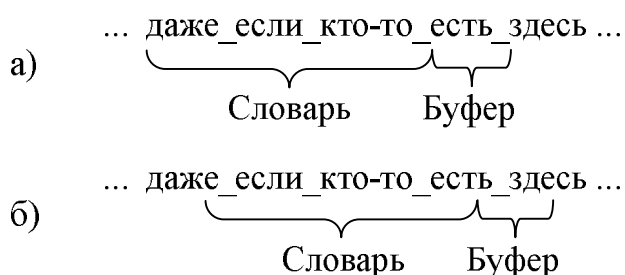


Рис. 11. Пример кодирования алгоритмом LZ77

Например, если окно кодера находится в положении, приведенном на рис. 11а, то кодер обнаружит в словаре по адресу 5 совпадающую с началом буфера фразу из 2 символов «ес» и выдаст запись $(5, 2, \text{«т»})$. После этого скользящее окно переместится по сообщению на 3 символа, см. рис. 11б. Следующей записью,

которую выдаст кодер, будет тройка (0, 0, «Ь»), а окно переместится на одну позицию.

Декодирование сжатых данных осуществляется путем простой замены кода на блок символов, состоящий из фразы словаря и явно передаваемого символа. Декодер должен выполнять те же действия по изменению окна, что и кодер. Фраза словаря элементарно определяется по смещению и длине, поэтому важным свойством LZ77 и прочих алгоритмов со скользящим окном является очень быстрая работа декодера.

Поиск подстрок в словаре представляет собой вычислительно сложную процедуру. Кроме того, в том случае, когда сообщение содержит малое количество повторяющихся серий символов, кодер LZ77 работает неэффективно. Например, если взять объем словаря в 4096 символов (12-битная адресация), а объем буфера – 16 символов (для кодирования длины подстроки требуется 4 бита), то при 256 символах алфавита сообщения появление символа *s*, отсутствующего в словаре, потребует выдачу 24 бит для записи (0, 0, *s*) вместо 8 бит, которые требовались при исходном представлении символа *s*.

Метод LZSS

LZSS является развитием LZ77 и был предложен в 1982 году Стопером (Storer) и Шиманским (Szimansky). В методе LZSS особое внимание было уделено организации структуры словаря в виде лексикографически упорядоченного дерева для ускорения алгоритма поиска строк (на деталях реализации структуры словаря и алгоритмах поиска мы останавливаться не будем). Кодер LZSS выдает записи двух видов: (Адрес, Длина) – в том случае, если строка из буфера найдена в словаре, или непосредственно код символа. Для того, чтобы различать тип записи, перед ней добавляется один бит-признак (префикс записи). Такой способ кодирования позволяет получить заметный выигрыш в сжатии по сравнению с методом LZ77.

Методы LZ77 и LZSS основаны на предположении, что повторяющиеся последовательности символов находятся в сообщении недалеко друг от друга, т.е. при очередном повторении строки ее еще можно найти в постоянно обновляемом словаре. Может случиться, что повторившаяся строка находится уже вне скользящего по сообщению словаря. Желательно было бы сохранять часто встречавшиеся строки символов вне зависимости от того, как давно они встретились. Естественным выходом здесь кажется увеличение объема скользящего словаря, но это означает увеличение как времени поиска в словаре, так и битовых затрат на кодирование адреса строки в словаре, что, наоборот, может привести к ухудшению сжатия. То же касается и увеличения количества символов в упреждающем буфере (усложнение поиска в словаре, увеличение битовых затрат

на кодирование длины строки). С целью устранения указанных проблем Лемпел и Зив предложили в 1978 году другой метод словарного кодирования – LZ78.

Метод LZ78

Если в LZ77 словарь представлял собой множество подстрок символьной строки конечной длины – последней части кодируемого сообщения, то в LZ78 словарь $\mathbf{P} = \{P(i)\}_i$ представляет собой потенциально бесконечный массив из фраз $P(i)$ различной длины, которые встретились при обработке сообщения. В начале работы и у кодера, и у декодера массив словаря состоит из одного элемента – пустой строки. Словарь пополняется в процессе обработки сообщения.

Считывая очередной незакодированный символ сообщения s из входного потока, кодер присоединяет его к концу текущей строки S посредством операции конкатенации: $S = S + s$ (в начале работы строка S пустая). До тех пор, пока текущая строка соответствует какой-либо фразе из словаря, т.е. $\exists i: S = P(i)$, $P(i) \in \mathbf{P}$, процесс присоединения очередного считанного символа сообщения к строке S продолжается. В какой-то момент присоединение символа дает строку, отсутствующую в словаре, т.е. $S = P(i) + s: S \notin \mathbf{P}, P(i) \in \mathbf{P}$. Тогда кодер:

- 1) выдает в выходной поток запись, содержащую два поля: (i, s) , где i – номер последней найденной фразы $P(i)$ в массиве-словаре $\mathbf{P} = \{P(i)\}_i$, s – символ, присоединение которого дало текущую строку S , отсутствующую в словаре;
- 2) строку $S = P(i) + s$ добавляет в словарь в качестве нового (очередного) элемента массива: $\mathbf{P} = \mathbf{P} + S$;
- 3) текущую строку S устанавливает в пустую.

Затем кодер продолжает работу по описанному алгоритму, считывая очередной символ сообщения, присоединяя его к текущей строке и проверяя наличие этой строки в словаре.

Декодер использует те же правила построения словаря.

Поскольку размер словаря $\mathbf{P}$ в методе LZ78 ничем не ограничен, то при работе алгоритма кодирования может возникнуть переполнение – нехватка памяти для хранения словаря, или нехватка производительности вычислительной системы для реализации поиска по словарю. Оригинальный метод LZ78 не определяет, что нужно делать в случае переполнения словаря. Поэтому метод LZ78 является скорее теоретической конструкцией. Для ее использования на практике необходимо определиться с тем, как избежать переполнения словаря. Рассмотрим некоторые возможные пути решения этой проблемы.

1. «Замораживание словаря». Как только объем словаря достиг максимально возможного значения, он перестает пополняться, становится статическим. Если распределение повторяющихся фраз в сообщении достаточно однородно, то

сформированный по начальным данным словарь будет эффективным и для обработки всего сообщения.

2. Сброс словаря в начальное состояние (одна пустая строка) при переполнении. При этом кодируемое сообщение, по сути дела, разбивается на блоки символов, для каждого из которых в процессе кодирования формируется свой словарь. Такой подход более предпочтителен, если состав повторяющихся фраз в сообщении меняется от блока к блоку. Как и в LZ77, здесь неявно предполагается, что похожие данные будут группироваться в сообщении близко друг к другу.
3. Когда словарь заполнился, удалить из него некоторые самые старые строки. К сожалению, не существует универсального алгоритма, который определял бы, какие строки и в каком количестве удалять.

Метод LZW

Этот словарный метод кодирования представляет собой модификацию LZ78 и был предложен Уэлчем (Welch) в 1984 году.

Основное отличие метода LZW от LZ78 состоит в том, что кодер LZW выдает *только* ссылки на словарь **P**, который при инициализации перед началом работы заполняется всеми возможными фразами длиной в один символ, т.е. всеми символами алфавита. Когда в процессе работы кодера присоединение очередного символа *s* дает текущую строку $S = P(i) + s$, отсутствующую в словаре ($S \notin \mathbf{P}, P(i) \in \mathbf{P}$), кодер LZW обрабатывает эту ситуацию следующим образом:

- 1) выдает в выходной поток индекс *i* последней найденной фразы $P(i)$ из массива-словаря $\mathbf{P} = \{P(i)\}_i$;
- 2) строку $S = P(i) + s$ запоминает для последующего добавления в словарь $St = S$;
- 3) присваивает текущей строке значение $S = s$.

Строка *St* будет добавлена в словарь **P** сразу после того, как в следующий раз кодер выдаст (а декодер прочитает) очередной индекс *k* некоторой фразы $P(k)$ – это сделано для того, чтобы и кодер, и декодер работали с одинаковыми словарями, пополняя их синхронно. До момента выбора следующей фразы $P(k)$ из словаря декодер не имеет информации о том, какой символ (он является первым в $P(k)$ символом) нужно приписать к строке $P(i)$ для формирования фразы *St*, подлежащей добавлению в словарь.

Так же, как и для LZ78, ключевым для метода LZW является предотвращение переполнения словаря. Для этого используются те же описанные выше подходы. Определение правил для «чистки» словаря (удаления фраз) во многом определяет эффективность компрессии данных и по-прежнему остается актуальной задачей.

Задание

1. Изучить исходные коды прилагаемой к лабораторной работе реализации алгоритма LZW.
2. Разобраться в механизме работы со словарём в данной реализации.
3. Сгенерировать файл размером 100 Кбайт, на котором можно получить минимальную степень сжатия.
4. Исследовать работу алгоритма на данных различного типа (русский и английский текст, исполняемые файлы, программный код и т.д., всего не менее 3-х примеров). Сравнить результаты, выраженные в средних битовых затратах на символ, с результатами работы одного из стандартных архиваторов (ZIP, RAR, ...), а также с оценкой энтропии $H = -\sum v_k \log_2 v_k$, где v_k – частоты появления символов в сообщении.
5. Исследовать эффективность и производительность алгоритма в зависимости от размера словаря.

Контрольные вопросы

1. На какие группы делятся алгоритмы сжатия без потерь?
2. Назовите несколько методов сжатия данных без потерь.
3. В чем заключается основная идея словарных методов сжатия?
4. Расскажите об алгоритмах LZ77, LZSS.
5. Расскажите об алгоритме LZ78.
6. Расскажите об алгоритме LZW.

Литература

1. Ватолин Д., Ратушняк А., Смирнов М., Юкин В. Методы сжатия данных. Устройство архиваторов, сжатие изображений и видео. – М.: ДИАЛОГ-МИФИ, 2002.
2. www.compression.ru

Лабораторная работа №5. Арифметическое кодирование

Цель работы

Изучить алгоритм адаптивного арифметического кодирования и исследовать особенности его реализации.

Теоретические сведения

Метод арифметического кодирования был предложен в 70-х годах прошлого века и представляет собой альтернативу методам Шэннона-Фано и Хаффмана. Суть его состоит в том, что последовательности символов $\{x(1), x(2), \dots, x(M)\}$, созданных дискретным источником сообщений X с известным алфавитом и вероятностями появления символов $\{(x_k, p_k)\}_{k=1}^N$, ставится в соответствие некоторое число, однозначно задающее данную последовательность. Следующий пример иллюстрирует идею метода.

Рассмотрим источник, алфавит которого и соответствующие вероятности появления символов заданы в таблице. Определим число, соответствующее сообщению «DABAC».

Сначала по таблице распределения вероятностей необходимо построить *интервалы вероятностей* $\Delta_k = [L_{x_k}; U_{x_k})$ для каждого символа x_k алфавита $\{x_k\}_{k=1}^N$ по

A	B	C	D
0,5	0,2	0,2	0,1

правилу: $U_{x_k} = \sum_{j=1}^k p_j$, $L_{x_k} = U_{x_k} - p_k$. То есть интервал $[0;1)$ разбивается на N

непересекающихся интервалов с длинами, равными вероятностям появления соответствующих символов: $|\Delta_k| = U_{x_k} - L_{x_k} = p_k$. В нашем случае имеем четыре интервала, см. табл.:

Символ	A	B	C	D
Интервал вероятности	$[0,0; 0,5)$	$[0,5; 0,7)$	$[0,7; 0,9)$	$[0,9; 1,0)$

Далее выбираются границы *текущего интервала* $[L, U)$: $L = 0$ (нижняя граница), $U = 1$ (верхняя граница).

Затем начинается последовательная обработка символов. Получая очередной символ x_k , кодер производит разбиение текущего интервала пропорционально начальным интервалам вероятностей символов и в качестве следующего текущего интервала выбирает тот, которому соответствует символ x_k . При этом каждый последующий текущий интервал вложен в предыдущий. Так, для нашего случая кодирование сообщения «DABAC» сводится к процедуре построения последовательности текущих интервалов, отраженной на рис. 12. Получили окончательный интервал $[0,9285; 0,9295)$. Из этого интервала можно взять любое

число, например, $B = 0,929$ (как содержащее наименьшее количество цифр), и восстановить по нему исходную последовательность из 5 символов «DABAC».

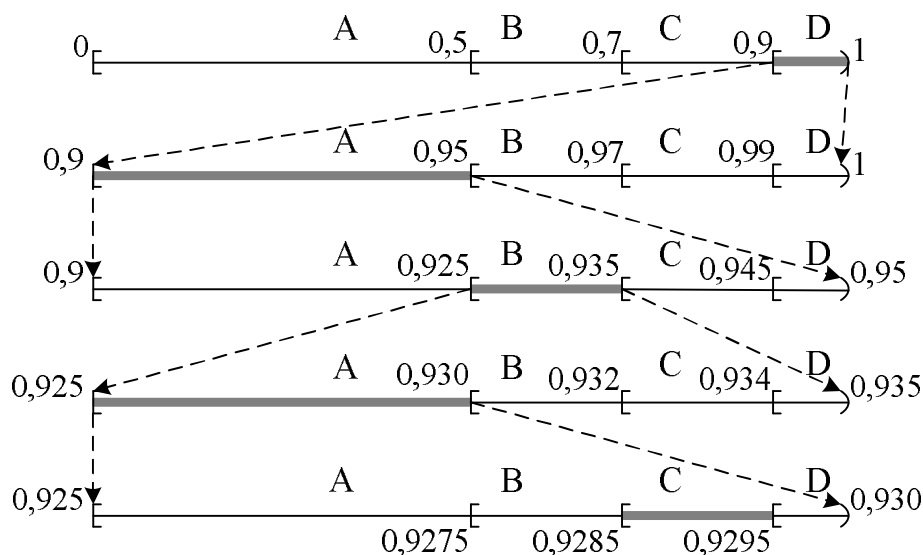


Рис. 12. Кодирование последовательности «DABAC»

Для восстановления последовательности на стороне декодера должно быть известно распределение вероятностей символов, т.е. разбиение на интервалы вероятностей. Изначально выбираем текущий интервал $[0;1)$. Далее разбиваем текущий интервал пропорционально интервалам вероятностей символов, проверяем, в интервал какого символа попадает число B . Так находим очередной декодированный символ, и выбираем его интервал вероятности в качестве следующего текущего интервала. И так далее.

Заметим, что появление того или иного символа можно отождествить с выбором наудачу числа из текущего интервала. Попадание числа в интервал вероятности некоторого символа означает событие, состоящее в появлении этого символа. Подобная трактовка хорошо известна как «геометрическая вероятность».

Процедура кодирования

Рассмотрим формализованное описание процедуры нахождения числа B , которое ставится в соответствие последовательности $\{x(1), x(2), \dots, x(M)\}$, созданной источником с алфавитом $\{x_k\}_{k=1}^N$, для которого задано (известно) разбиение на интервалы вероятностей $\Delta_k = [L_{x_k}; U_{x_k})$, $k=1, \dots, N$.

Шаг 1.

Установить границы текущего интервала: $L = 0, U = 1$. Установить счетчик символов $j = 1$.

Шаг 2.

Определить новые границы текущего интервала:

$$W = U - L,$$

$$U = L + W \times U_{x(j)},$$

$$L = L + W \times L_{x(j)}.$$

Шаг 3.

Увеличить счетчик символов $j = j + 1$. Если $j > M$, то перейти на шаг 4, иначе – на шаг 2.

Шаг 4.

Выдать число $B \in [L, U)$, содержащее наименьшее количество цифр.

Наличие арифметических операций на шаге 2 послужило причиной того, что описанный метод получил название *арифметического кодирования*.

Декодирование сообщения из M символов, которому соответствует число B , можно описать в виде следующей процедуры, также основанной на выполнении арифметических операций.

Шаг 1.

Установить границы текущего интервала: $L = 0, U = 1$. Ввести число B .

Установить счетчик символов $j = 1$.

Шаг 2.

Найти интервал Δ_k^j , в который попадает число $B \in \Delta_k^j$.

Выдать символ $x(j) = x_k$.

Шаг 3.

Определить новые границы текущего интервала:

$$W = U - L,$$

$$U = L + W \times U_{x(j)},$$

$$L = L + W \times L_{x(j)}.$$

Шаг 4.

Увеличить счетчик символов: $j = j + 1$. Если $j \leq M$, то перейти на шаг 2, иначе закончить работу.

Процедура пересчёта границ интервала на шаге 3 называется *нормализацией*.

Практическая часть

При практической реализации алгоритма возникают некоторые проблемы. Кратко опишем принципы их решения.

Проблема «плавающей точки»

Арифметический кодер сложно реализовать на числах и операциях с плавающей точкой для произвольной длины входных данных, так как числа

наибольшей доступной точности (double) имеют минимальную различимую длину интервала порядка 10^{-15} . В то же время при кодировании сообщения из 100 символов, каждый из которых имеет вероятность появления 0,01, длина интервала после завершения кодирования будет равна $0,01^{100} = 10^{-200}$.

Арифметический кодер реализуют с использованием чисел с фиксированной точкой, представляемых в памяти обычными целыми числами.

Проблема ограниченной разрядности

Использование целочисленных операций приводит к проблеме, аналогичной предыдущей: разрядность целочисленных регистров процессора также ограничена. Однако ее можно решить. Заметим, что после того, как в процессе работы кодера несколько старших разрядов (в примере на рис. 13 это $U0$ и $L0$, $U1$ и $L1$) регистров U и L совпали, они уже не могут измениться и не участвуют в дальнейшей работе кодера.

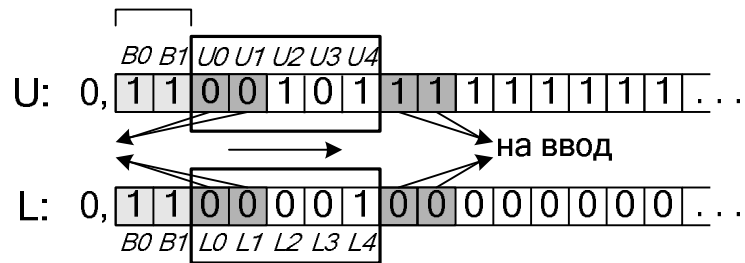


Рис. 13. Обработка совпавших старших битов в регистрах U и L

Поэтому кодер может выдвинуть совпавшие разряды из регистра и записать их в выходной поток данных (а декодер – просто отбросить), освободив место в регистре. В начале работы кодера нижняя граница интервала представляется в виде дроби с бесконечным числом нулей $L_0 = 0 = 0,0000\dots$, а верхняя – в виде дроби с бесконечным числом единиц $U_0 = 1 = 0,1111\dots$. Поэтому, когда значения в разрядах $U0$ и $L0$, $U1$ и $L1$ выдвинутся, значения регистров сдвинутся налево на два разряда, в освободившиеся младшие разряды $U3$, $U4$ вдвинутся две единицы, а в разряды $L3$, $L4$ – два нуля.

Следует обратить внимание, что из-за ограниченной разрядности регистров при практической реализации арифметического кодера необходимо использовать числа с фиксированной запятой. Так, верхняя граница U представляется при помощи целого r -битового числа $U_r \in \{0, \dots, 2^r - 1\}$ в виде $U = (U_r + 1) \cdot 2^{-r}$ (прибавлять единицу нужно для того, чтобы можно было представить начальное значение U в виде $0,1111\dots$ – дроби с бесконечным числом единиц). Тогда изначально для $U = 1$ задаем значение $U_r = 2^r - 1 = 111\dots 1_{(2)}$. Нижнюю границу и другие промежуточные

числа $T \in [0; 1)$ представляем как $T = T_r \cdot 2^{-r}$, где целое число $T_r \in \{0, \dots, 2^r - 1\}$. После перехода к числам с фиксированной запятой по вышеприведенным формулам шаг 2 процедуры кодирования (определение границ текущего интервала) при обработке j -ого символа сообщения будет выглядеть в целочисленной арифметике следующим образом:

$$\begin{aligned} W_r &= U_r - L_r + 1, \\ U_r &= L_r + (W_r \times U_{x(j),r}) 2^{-r} - 1, \\ L_r &= L_r + (W_r \times L_{x(j),r}) 2^{-r}. \end{aligned}$$

Проблема сближения границ

Другая проблема возникает, если в процессе кодирования верхняя и нижняя границы интервала отличаются в старших разрядах, но длина интервала недопустимо мала.

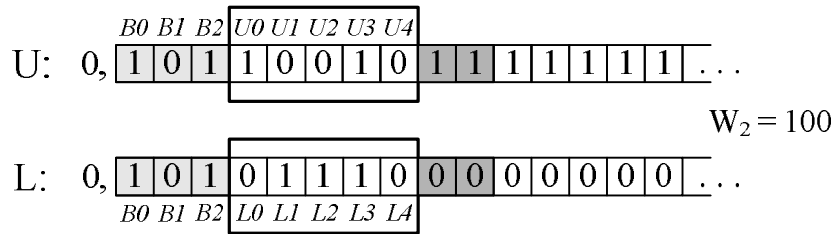


Рис. 14. Недопустимое сближение границ интервала

В этом случае есть опасность, что слишком маленькая длина интервала может не позволить закодировать очередной символ, так как при выполнении целочисленной нормализации значения регистров U и L не изменятся:

$$\begin{aligned} L_{i+1} &= L_i + \left[\frac{(U_i - L_i) \cdot L_{x(i)}}{W} \right] = L_i \\ U_{i+1} &= U_i + \left[\frac{(U_i - L_i) \cdot U_{x(i)}}{W} \right] = U_i \end{aligned}$$

и алгоритм заикнется. С этой проблемой можно бороться, отслеживая сближение верхней и нижней границы. Если длина интервала W на очередной итерации стала недопустимо малой, то выполняется принудительное расширение интервала с помощью выдвигания из регистров одного или нескольких старших разрядов (за исключением самого старшего), ещё не готовых к выводу в выходной поток (в случае, показанном на рис. 14, разряды $U1$, $U2$ в регистре U и разряды $L1$, $L2$ в регистре L выдвинутся, а разряды $U0$ и $L0$ останутся на своих местах).

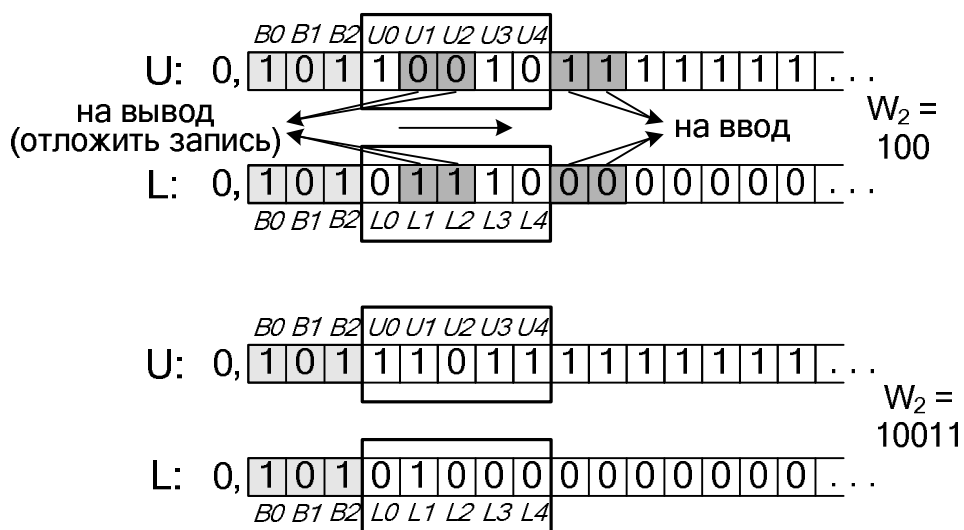


Рис. 15. «Отложенный» вывод старших разрядов

При этом сама запись выдвинутых разрядов в поток откладывается, так как их окончательные значения ещё неизвестны и станут известны только после того, как совпадут старшие разряды регистров, не участвовавшие в выдвигании, то есть когда $U_0 = L_0$.

Теперь выясним, каким должно быть критическое значение W , при котором следует проводить принудительное расширение интервала. Для этого удобно при каждом запуске процедуры нормализации проверять, выполняются ли одновременно равенства $L_1 = 1$ и $U_1 = 0$. Если они выполняются, то границы интервала находятся на недопустимо близком расстоянии и его нужно расширять.

Заметим, что при таком принципе определения W в «отложенных» разрядах могут быть либо все нули, либо все единицы, в зависимости от того, к какому значению из двух сойдутся старшие разряды регистров. В примере, показанном на рисунке 4 (в регистре 5 двоичных разрядов), два выдвинутых «без очереди» разряда будут нулями, если $L_0 = U_0 = 1$ и единицами, если $L_0 = U_0 = 0$. Это следует из того, что при обработке очередного символа интервал может только сужаться, то есть L может только увеличиваться, а U – только уменьшаться (см. шаг 3 процедуры кодирования). При этом должно соблюдаться неравенство $L < U$. Следовательно, если первоначально $L = 011...$ и $U = 100...$, то в итоге совпадение трех старших бит произойдет либо когда U станет меньше числа 10000 ($U = 011..., L = 011...$), либо когда L станет больше его ($L = 100..., U = 100...$). Ситуация, в которой, например, L станет равным числу 01010 или U – числу 10110, невозможна.

Следует обратить внимание, что «отложенные» разряды нужно выводить в том порядке, в котором они должны были бы следовать в нормальной ситуации, то есть сразу после того, как в выходной поток будет выведен совпавший разряд $L_0 = U_0$. Продолжая пример, показанный на рис.4, после вывода «отложенных» разрядов можем получить в выходном потоке либо последовательность «100», либо «011».

Накопленная гистограмма и окончание кодирования

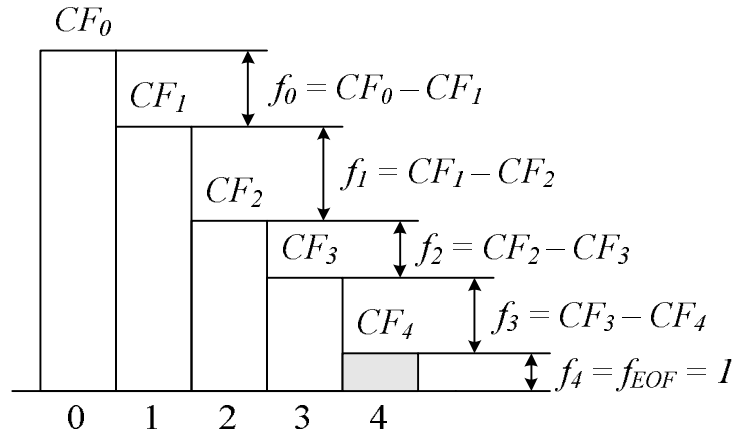


Рис. 16. Накопленная гистограмма частот

Статистику арифметического кодера удобно хранить в виде накопленной (кумулятивной) гистограммы, как показано на рис. 16. Здесь f_k – количество

появлений k -го символа, а $CF_k = \sum_{i=k}^N f_i$ – «накопленная ненормированная частота»

символа, с помощью которой удобно представлять границу интервала. Значение частоты символа (равное длине интервала W) можно получить из разности соседних накопленных частот. Величина CF_0 равна длине всего «единичного» отрезка, на котором производится кодирование. Реальная частота (оценка вероятности) символа вычисляется по формуле $p_k = \frac{CF_k - CF_{k-1}}{CF_0}$.

Обычно накопленная гистограмма состоит не из N , а из $N+1$ столбцов. В последнем столбце хранится частота специального символа, который помещается кодером в выходной поток после всех символов, подлежащих кодированию. Этот символ всегда имеет частоту 1, так как встречается в выходном потоке всего один раз и обозначает конец файла. Он необходим для корректной работы декодера, который, декодируя его, завершает работу. Очевидно, символ конца файла (обычно его называют EOF-символом) не нужен, если декодеру заранее известно, сколько символов находится в закодированном потоке, но обычно декодер такой информации не имеет.

Адаптивное моделирование

В рассмотренных методах кодирования источников сообщений предполагалось, что распределение вероятностей $\{p(x_k)\}_{k=1}^N$ (для источника без памяти) задано. Вместе с тем, статистическая модель источника не всегда известна или изменяется в процессе обработки. Тогда построение модели осуществляется одновременно с кодированием.

Покажем, как осуществляется адаптивное статистическое моделирование на примере источника без памяти. В качестве начального распределения вероятностей источника можно выбрать равномерное: $p_k = 1/N$, $k = 1, \dots, N$, а затем, закодировав первый символ сообщения $x(1) = x_j$, нужно внести изменения в модель, повысив вероятность символа x_j : $p_j = 2/(N+1)$, $\forall k \neq j$ $p_k = 1/(N+1)$. Декодер, начиная работать с тем же распределением вероятностей, что и кодер, после декодирования первого символа сообщения затем повышает в модели источника соответствующую вероятность по тому же правилу, которое использовал кодер. Такая адаптация статистических моделей производится после каждого кодирования/декодирования очередного символа. Фактически, в качестве статистической модели при этом используется гистограмма частот, полученная по выборке из обработанных символов. По мере обработки (кодирования или декодирования) сообщения объем выборки растет, и накопленная гистограмма все более точно описывает истинное распределение вероятностей стационарного источника. Очевидно, что для адекватного описания распределения вероятностей объем выборки должен намного превышать количество ячеек в гистограмме, т.е. количество символов N в алфавите источника. Поэтому чем меньше символов в алфавите, тем быстрее «настраивается» модель источника.

С точки зрения практической реализации адаптивное арифметическое кодирование отличается от неадаптивного только наличием функции обновления накопленной гистограммы частот.

Отметим, что увеличение частоты одного символа в накопленной гистограмме влечёт за собой увеличение накопленной частоты всех следующих за ним символов, поэтому адаптивное моделирование является вычислительно сложной задачей и замедляет работу арифметического кодера.

Кроме того, при достижении максимальной накопленной частотой некоторого наибольшего значения необходимо проводить нормализацию всей накопленной гистограммы. Самый простой способ нормализации – деление частоты каждого символа на 2.

Задание

1. Изучить исходные коды неадаптивного арифметического кодера, прилагаемые к методической разработке.
2. На основе предлагаемых исходных кодов реализовать адаптивную модель для арифметического кодера. Кроме того, необходимо предусмотреть специальную функцию для загрузки первоначальной статистики из файла
3. Реализовать функцию для подсчёта энтропии входных данных по всему файлу, а также функцию, подсчитывающую «мгновенную» энтропию по текущей гистограмме адаптивного кодера.

4. С помощью функций, реализованных в п. 3 (но не обязательно ограничиваясь ими), исследовать эффективность работы адаптивного и неадаптивного кодера на различных типах данных, полученных как естественным (текстовые, исполняемые файлы, исходные коды и т.д.), так и искусственным путем. В последнем случае необходимо сгенерировать входные данные с различной статистикой символов, а также входные данные с резко изменяющейся на протяжении файла (нестационарной) статистикой. Например, создав наборы данных $x_1, x_2, \dots, x_N$ и $y_1, y_2, \dots, y_N$ с существенно различающимися статистиками, изучить характеристики адаптивного кодера при обработке последовательностей $x_1, x_2, \dots, x_N, y_1, y_2, \dots, y_N$ и $x_1, y_1, x_2, y_2, \dots, x_N, y_N$. Исследование должно быть выполнено в общей сложности не менее чем на 5 различных файлах.
5. Сравнить производительность адаптивного и неадаптивного кодера (по времени) на различных входных данных, использованных в п. 4.
6. Исследовать статистику индексов слов LZW-словаря в файлах, сжатых алгоритмом LZW из предыдущей лабораторной работы. Реализовать дополнительное статистическое сжатие выходного потока индексов слов LZW-словаря с помощью арифметического кодера. Исследовать эффективность такого сжатия для различных типов данных.

Альтернативное задание

На основе предложенных исходных кодов реализовать *многомодельный* адаптивный арифметический кодер. Он отличается от обычного кодера тем, что работает не с одной гистограммой частот, а с несколькими. Выбор гистограммы осуществляется по номеру модели, который дополнительно передаётся в процедуру кодирования (декодирования) символа. Таким образом, если и кодеру, и декодеру известны номера моделей для каждого символа, арифметический кодер может сжимать и мультиплексировать несколько независимых потоков данных в один битовый поток.

Альтернативное задание выполняется по предварительному согласованию с преподавателем.

Приложение. Формат входных и выходных данных кодера

1. Кодер запускается из командной строки, на входе четыре аргумента: переключатель режима кодирования/декодирования, файл статистики, входной файл, выходной файл.
2. Файл статистики содержит в себе распределение частот символов, которое загружается в статическую модель перед началом работы. Файл двоичный,

содержит 2...256 записей типа unsigned short (16 бит), образующих гистограмму частот.

3. Для кодирования используется двоичный файл, в каждом байте которого содержится символ, подлежащий кодированию. Таким образом, максимальная длина алфавита кодера равна 256 символам.

Контрольные вопросы

1. Покажите на примере принцип работы арифметического кодера.
2. Какие проблемы возникают при практической реализации арифметического кодера?
3. Как можно ускорить предложенную реализацию алгоритма?

Литература

1. Ватолин Д., Ратушняк А., Смирнов М., Юкин В. Методы сжатия данных. Устройство архиваторов, сжатие изображений и видео. – М.: ДИАЛОГ-МИФИ, 2002. - 384 с.
2. Mark Nelson, Jean-Loup Gailly. The Data Compression Book. – M&T Books; 2nd Edition.
3. www.compression.ru

Лабораторная работа №6. Сжатие изображений с использованием дискретного косинусного преобразования

Цель работы

Изучить алгоритм сжатия изображений по методу JPEG, основанный на применении дискретного косинусного преобразования (ДКП).

Теоретические сведения

Компрессия изображений на основе двумерного ДКП

Широкий класс линейных дискретных преобразований, используемых для цифровой обработки изображений, составляют двумерные (т.е. переводящие матрицу в матрицу) сепарабельные преобразования, которые получают из одномерных (переводящих вектор в вектор) следующим образом.

Пусть преобразование матрицы $\mathbf{X} = \{x_{j,m}\}$, $j = 0, \dots, N-1$; $m = 0, \dots, M-1$, в матрицу $\mathbf{Y}$ той же размерности производится по формуле

$$y_{k,l} = \sum_{j=0}^{N-1} w_{k,j}^{(N)} \sum_{m=0}^{M-1} w_{l,m}^{(M)} x_{j,m} \quad (25)$$
$$k = 0, \dots, N-1; l = 0, \dots, M-1$$

где $\mathbf{W}_K = \{w_{j,n}^{(K)}\}_{j,n=0}^{K-1}$ – матрица одномерного преобразования вида $\mathbf{u} = \mathbf{W} \cdot \mathbf{v}$:

$u_j = \sum_{n=0}^{K-1} w_{j,n}^{(K)} v_n$, $j = 0, \dots, K-1$. Тогда двумерное преобразование (25) называется

дискретным *сепарабельным* преобразованием.

Из формулы (25) видно, что вычисление сепарабельного преобразования сводится к выполнению одномерных дискретных преобразований размерности M вдоль строк $j = 0, \dots, N-1$ матрицы $\mathbf{X}$:

$$z_{j,l} = \sum_{m=0}^{M-1} x_{j,m} w_{l,m}^{(M)}, l = 0, 1, \dots, M-1$$

(или $\mathbf{Z} = \mathbf{XW}_M^T$), и последующим одномерным преобразованиями размерности N вдоль столбцов $l = 0, \dots, M-1$ полученной матрицы $\mathbf{Z} = \{z_{j,l}\}$:

$$y_{k,l} = \sum_{j=0}^{N-1} w_{k,j}^{(N)} z_{j,l}, k = 0, 1, \dots, N-1$$

(или $\mathbf{Y} = \mathbf{W}_N \mathbf{Z}$). Причем сначала можно выполнить одномерные преобразования по столбцам матрицы, а потом по строкам, т.к. порядок суммирования в выражении

(25) можно поменять. Из сказанного следует, что *обратить* двумерное сепарабельное преобразование можно, выполнив одномерные обратные преобразования сначала вдоль столбцов, а потом вдоль строк матрицы $\mathbf{Y}$ (или наоборот).

Запишем сепарабельное преобразование (25) в эквивалентном матричном виде: $\mathbf{Y} = \mathbf{W}_N \mathbf{X} \mathbf{W}_M^T$. Тогда, если соответствующее одномерное преобразование является ортогональным, то обратное к (25) преобразование записывается как $\mathbf{X} = \mathbf{W}_N^T \mathbf{Y} \mathbf{W}_M$, а обращение формулы (25) принимает вид:

$$x_{j,m} = \sum_{k=0}^{N-1} w_{k,j}^{(N)} \sum_{l=0}^{M-1} w_{l,m}^{(M)} y_{k,l} \quad (26)$$

$$j = 0, \dots, N-1; m = 0, \dots, M-1$$

В частности, прямое и обратное двумерные ДКП задаются соответственно следующими формулами:

$$y_{k,l} = \frac{2}{\sqrt{MN}} c(k) c(l) \sum_{j=0}^{N-1} \cos\left(\frac{pk}{N} \left(j + \frac{1}{2}\right)\right) \sum_{m=0}^{M-1} x_{j,m} \cos\left(\frac{pl}{M} \left(m + \frac{1}{2}\right)\right), \quad (27)$$

$$k = 0, 1, \dots, N-1; l = 0, 1, \dots, M-1$$

$$x_{j,m} = \frac{2}{\sqrt{MN}} \sum_{k=0}^{N-1} c(k) \cos\left(\frac{pk}{N} \left(j + \frac{1}{2}\right)\right) \sum_{l=0}^{M-1} c(l) y_{k,l} \cos\left(\frac{pl}{M} \left(m + \frac{1}{2}\right)\right), \quad (28)$$

$$j = 0, 1, \dots, N-1; m = 0, 1, \dots, M-1$$

$$c(n) = \begin{cases} 1/\sqrt{2}, & \text{при } n = 0 \\ 1, & \text{при } n \neq 0 \end{cases}.$$

Стандарт JPEG

Именно двумерное ДКП было положено в основу появившегося в начале 90-х годов прошлого века международного *стандарта JPEG* (Joint Photographic Experts Group), который определяет методы эффективного представления фотографических изображений. Несмотря на то, что с 2001 года официально действует расширенная спецификация стандарта JPEG 2000, которая допускает использование альтернативной схемы кодирования на основе вейвлет-преобразований, кодирование с применением ДКП остается основным и наиболее распространенным методом, реализованным в многочисленных программных продуктах и различной аппаратуре. Рассмотрим кратко основной вариант схемы JPEG на примере обработки полутонового цифрового изображения, которое можно представить в виде матрицы, элементами которой являются значения яркости точек раstra. Точки изображения (элементы матрицы) называют также пикселями.

Поскольку фотографические изображения в большинстве случаев представляют собой нестационарные двумерные сигналы, для адаптации способа обработки под

локально изменяющиеся характеристики изображения оно обрабатывается небольшими блоками размерности 8×8 пикселей. При такой размерности, с одной стороны, декоррелирующие свойства ДКП уже хорошо выражены, а с другой стороны, в столь малой области статистические характеристики изображения можно считать локально стационарными, с достаточной точностью подчиняющимися марковской модели сигнала. Фрагменты обрабатываются последовательно (очередной обрабатываемый блок является соседним по отношению к предыдущему), общая схема обработки имеет вид, представленный на рис. 17, и включает в себя четыре основных шага.

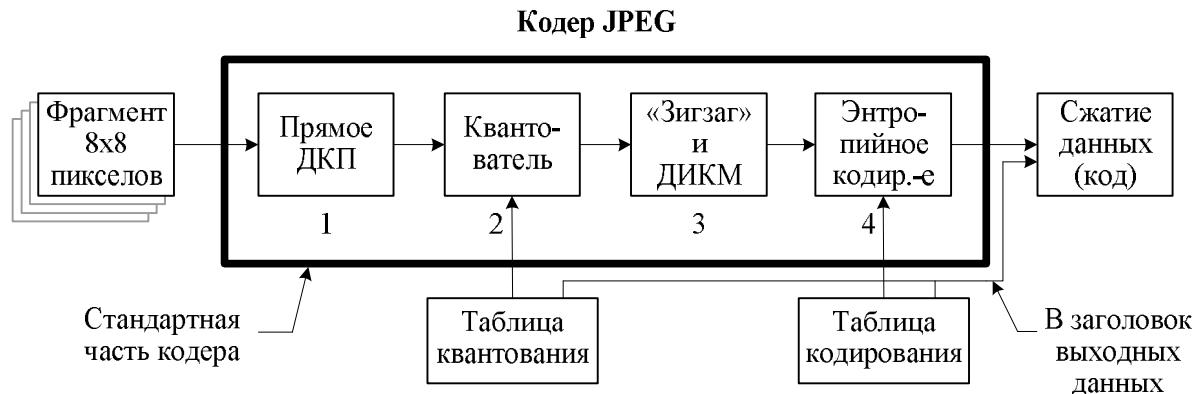


Рис. 17. Схема сжатия изображения по стандарту JPEG

Шаг 1. Вычисление двумерного ДКП блока размерности 8×8 .

Шаг 2. Скалярное квантование спектра ДКП: каждый элемент спектра равномерно квантуется с округлением до целого:

$$\tilde{y}_{k,l} = \text{round}(y_{k,l} / q_{k,l}).$$

Целые числа $q_{k,l}$ определяются по *таблице (матрице) квантования Q* (размера 8×8). Матрица квантования стандартом не регламентируется (но есть рекомендуемый JPEG набор таблиц) и передается в заголовке выходных данных. Диапазон возможных значений, которые могут принимать элементы спектра, после выполнения процедуры квантования существенно уменьшается, появляется большое количество нулей.

Шаг 3. Коэффициент спектра $\tilde{y}_{0,0}$ (постоянная составляющая¹ яркости фрагмента изображения) подвергается дифференциальной импульсно-кодовой модуляции (ДИКМ), т.е. $\tilde{y}_{0,0}$ заменяется значением $\Delta\tilde{y}_{0,0} = \tilde{y}_{0,0}(j) - \tilde{y}_{0,0}(j-1)$, где j – номер текущего фрагмента 8×8. ДИКМ целесообразно применять по причине того, что средние значения яркости соседних фрагментов, как правило, имеют близкие значения. Полученный после ДИКМ коэффициента $\tilde{y}_{0,0}$ двумерный набор из 64 чисел зигзагообразно (см. рис. 18) считывается в

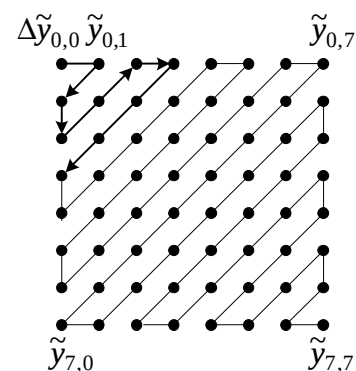


Рис. 18. Зигзаг-сканирование

одномерную последовательность, образуя промежуточный поток данных, который является входным для 4-го шага. Смысл такого зигзагообразного считывания заключается в том, чтобы сгруппировать основную часть нулевых элементов спектра, которые оказываются сосредоточенными в конце 64-элементной последовательности промежуточного потока данных.

Шаг 4.1. Удаление нулей. Нули удаляются путем преобразования промежуточного потока данных в два выходных потока. Запись в выходные потоки данных производится при появлении во входном промежуточном потоке ненулевого элемента $S \neq 0$.

Диапазон значений числа S	B
$\{-1, 1\}$	1
$\{-3, -2\} \cup \{2, 3\}$	2
$\{-7, -6, -5, -4\} \cup \{4, 5, 6, 7\}$	3
$\{-15, -14, \dots, -8\} \cup \{8, 9, \dots, 15\}$	4
$\{-31, \dots, -16\} \cup \{16, \dots, 31\}$	5
...	...
$\{-2^{15}+1, \dots, -2^{14}\} \cup \{2^{14}, \dots, 2^{15}-1\}$	15

Первый выходной поток состоит из байтов-ключей L : в полубайте L_1 записывается число нулей, встретившихся перед элементом S после предыдущего появления ненулевого элемента, а в полубайте L_2 записывается число B , которое равно количеству двоичных разрядов, необходимых для представления элемента S , см. таблицу. Во второй поток заносится B бит двоичного кода элемента S . Таким образом, первый поток данных – байтовый, второй – битовый. После записи последнего ненулевого элемента из «змейки», в байтовый поток заносится специальный символ признака конца блока ненулевых коэффициентов (EOB – end of block). Битовый поток данных дополнительно не обрабатывается, а поток байтов-ключей кодируется статистически.

¹ В англоязычной литературе для обозначения постоянной составляющей используется аббревиатура DC, а для остальных элементов ДКП-блока – AC, по аналогии с обозначениями постоянного и переменного тока.

Шаг 4.2. Статистическое кодирование байтов-ключей. Для этой цели используется префиксное кодирование. Таблицы кодов Хаффмана должен строить сам разработчик системы сжатия, поэтому таблицы необходимо помещать в заголовок выходных данных (имеются рекомендуемые JPEG коды, которые сохранять в файле необязательно).

Второй шаг схемы сжатия JPEG вносит потерю информации. Задание таблицы квантования Q и является тем инструментом, при помощи которого выбирается компромисс между сжатием информации и достоверностью ее представления: чем грубее квантование (чем больше значения шага равномерного квантования $q_{k,l}$), тем большая ошибка возникнет в изображении после восстановления, но тем выше степень сжатия данных.

Каждому шагу в схеме сжатия (рис. 17) соответствует шаг в схеме восстановления изображения (рис. 19), на котором выполняются обратные операции. За исключением квантования, все операции являются обратимыми. «Деквантование» представляет собой масштабирование $\hat{y}_{k,l} = \tilde{y}_{k,l} \cdot q_{k,l}$, задаваемое той же матрицей квантования Q .

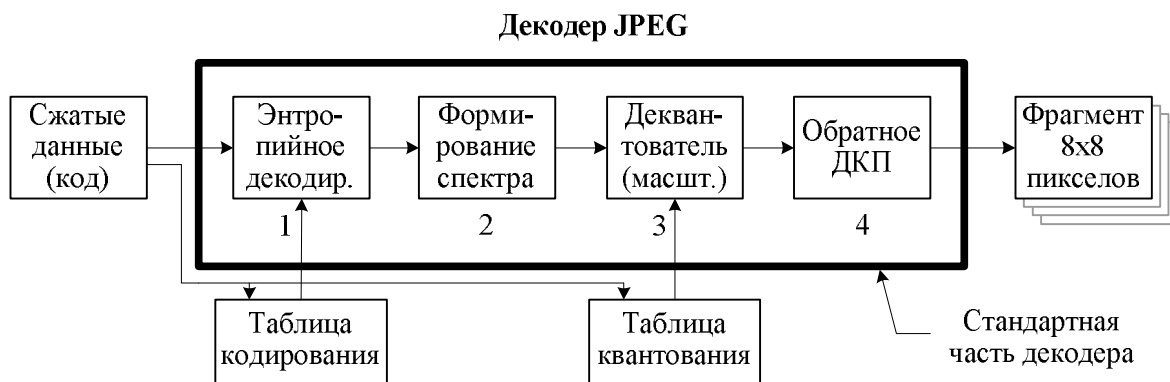


Рис. 19. Схема восстановления изображений по стандарту JPEG

При обработке цветных изображений каждый пиксел из цветового трехкомпонентного пространства (Red, Green, Blue) – сокращенно (R,G,B) – сначала переводится в представление:

$$\begin{pmatrix} Y \\ Cr \\ Cb \end{pmatrix} = \begin{pmatrix} 0,2990 & 0,5870 & 0,1140 \\ 0,5000 & -0,4187 & -0,0813 \\ -0,1687 & -0,3313 & 0,5000 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix} + \begin{pmatrix} 0 \\ 128 \\ 128 \end{pmatrix},$$

где компонента Y называется яркостной, а компоненты Cr , Cb – цветоразностными составляющими. По сравнению с яркостной составляющей чувствительность человеческого глаза к компонентам Cr , Cb значительно ниже, что учитывается при построении алгоритмов компрессии цветных изображений. Для цветоразностных компонент в методе JPEG применяется дополнительное прореживание массивов и затем используется более грубая обработка по той же

общей схеме, которая была рассмотрена выше для яркостной компоненты. При восстановлении изображений используется обратное преобразование цветового пространства:

$$\begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 1 & 1,4020 & 0 \\ 1 & -0,7141 & -0,3441 \\ 1 & 0 & 1,772 \end{pmatrix} \begin{pmatrix} Y \\ Cr-128 \\ Cb-128 \end{pmatrix}.$$

Задание

1. Пользуясь прилагаемыми к лабораторной работе исходными кодами метода сжатия полутоновых изображений на основе ДКП, аналогичного описанному в теоретической части методу JPEG, реализовать его кодирующую часть (`im_encode.m`). Убедиться в корректности работы алгоритма.
2. Исследовать качество работы полученного кодека по сравнению с любой реализацией метода JPEG на 2-3 стандартных тестовых изображениях.
3. Исследовать качество работы кодека на разных матрицах квантования (одна из стандартных матриц, рекомендуемых в методе JPEG, приведена в файле `quant_mtx.m`). В набор исследуемых матриц следует включить матрицу со всеми одинаковыми элементами.
4. В соответствии со своим вариантом выполнить одно из следующих заданий:
 - а. Исследовать результаты работы кодека при повторном кодировании нескольких изображений, ранее сжатых и восстановленных с помощью метода, реализованного в п.1 задания.
 - б. Исследовать распределение битовых затрат в сжатом изображении по различным массивам данных (постоянная составляющая, таблицы Хаффмана, массивы L и S) при сжатии 4-5 тестовых изображений с различными средними битовыми затратами.
 - в. Модифицировать кодек так, чтобы он работал как с таблицами Хаффмана, строящимися по статистике каждого кодируемого изображения, так и с одной универсальной таблицей, построенной при обработке 5-7 тестовых изображений. Сравнить качество сжатия алгоритма с универсальной таблицей и таблицами, адаптированными под конкретное изображение.

Альтернативное задание

Альтернативное задание заменяет п. 4 обычного задания и выполняется по согласованию с преподавателем.

Вариант 1. Оптимизировать формат хранения данных в заголовке файла (`encode_header.m`, `decode_header.m`).

Вариант 2. Исследовать отличия предложенного в лабораторной работе метода сжатия изображений от стандарта JPEG.

Вариант 3. Реализовать алгоритм постобработки декодированного изображения, устраняющий блочные артефакты с помощью НЧ-фильтрации изображения вдоль вертикальных и горизонтальных границ блоков.

Вариант 4. Реализовать метод сжатия с использованием арифметического кодирования вместо префиксных кодов. Арифметический кодер реализовать в виде MEX-библиотеки с использованием исходных кодов из предыдущей лабораторной работы. Исследовать качество работы модифицированного кодека, провести сравнение.

Вариант 5. Реализовать поддержку сжатия полноцветных (заданных в формате RGB) изображений.

Примечание

Поскольку встроенные функции MATLAB плохо приспособлены к работе с битовыми потоками, для ускорения работы программы предлагается написанная на языке C библиотека, выполняющая шаги 4.1 и 4.2 алгоритма. MEX-функции библиотеки можно вызывать из MATLAB как обычные m-функции, соблюдая формат вызова. Для Windows библиотека поставляется как в бинарном виде (dll, 32 бит), так и в исходных кодах; для других операционных систем – только в исходных кодах. Руководство по написанию и компиляции MEX-функций можно найти в справке MATLAB (doc mex).

Приложение. Методика тестирования алгоритмов сжатия изображений

Качество восстановленного после сжатия изображения оценивается с помощью пикового отношения сигнал-шум (PSNR): $PSNR = 10 \lg \frac{255^2}{MSE}$, где среднеквадратическое отклонение (MSE, mean square error) вычисляется по формуле

$$MSE = \frac{1}{MN} \sum_{i,j} (x_{i,j} - \hat{x}_{i,j})^2.$$

Здесь M, N – размеры изображения в пикселах, $x_{i,j}$ – значение яркости пиксела исходного изображения, $\hat{x}_{i,j}$ – значение яркости пиксела восстановленного изображения.

Изменяя значения скалярных квантователей, можно получить график зависимости качества восстановленного изображения от битовых затрат на его кодирование. Битовые затраты выражаются в битах на пиксел (bpp, bits per pixel):

$bpp = \frac{S}{MN}$, где S – общее число бит, необходимое для кодирования изображения.

Тестирование алгоритма следует проводить на стандартных и дополнительных изображениях (прилагаются к лабораторной работе) при диапазоне битовых затрат от 0,3 до 1,2 bpp.

Контрольные вопросы

1. Какие свойства ДКП позволяют использовать его для сжатия изображений?
2. Как можно улучшить эффективность предложенного метода сжатия изображений на основе ДКП?
3. Каким образом алгоритмы кодирования полутоновых изображений можно обобщить на случай цветных изображений?

Литература

1. Умняшкин С.В. Теоретические основы цифровой обработки и представления сигналов.
2. *Ватолин Д., Ратушняк А., Смирнов М., Юкин В.* Методы сжатия данных. Устройство архиваторов, сжатие изображений и видео. – М.: ДИАЛОГ-МИФИ, 2002. - 384 с.
3. *G. K. Wallace.* The JPEG still picture compression standard. IEEE Trans. Consumer Electron., vol. 38, no. 1, pp. 18–34, Feb 1992.

Лабораторная работа №7. Знакомство с дискретным вейвлет-анализом

Цель работы

Изучить способы построения масштабирующих функций и вейвлетов, связь дискретных вейвлет-преобразований и квадратурно-зеркальных фильтров. Познакомиться с основными идеями алгоритмов сжатия изображений с использованием дискретного вейвлет-преобразования.

Теоретическая часть

Общие сведения

Последовательность подпространств $\{V_m\} \subset L_2(\mathbb{R})$, $m \in \mathbb{Z}$, образует *кратно-масштабный анализ (КМА)*, если выполняются следующие свойства:

1. Подпространства вложены, $\forall m \in \mathbb{Z}: V_m \subset V_{m+1}$.
2. Если функция $f(x) \in V_m$, то $f(2x) \in V_{m+1}$, и наоборот, т.е. $\forall m \in \mathbb{Z}: f(x) \in V_m \Leftrightarrow f(2x) \in V_{m+1}$.
3. Найдется некоторая функция $f(x) \in V_0$, целочисленные сдвиги которой $\{f(x-n)\}_{n \in \mathbb{Z}} \subset V_0$ образуют ортонормированный базис подпространства V_0 . Функция $f(x)$ называется *масштабирующей*.
4. Пересечением всех подпространств является множество, содержащее единственный (нулевой) элемент: $\bigcap_{m \in \mathbb{Z}} V_m = \{q\}$.
5. После замыкания объединение подпространств совпадает с пространством $L_2(\mathbb{R})$: $\overline{\bigcup_{m \in \mathbb{Z}} V_m} = L_2(\mathbb{R})$.

На основании свойств 5 и 1 можно сделать вывод, что произвольную функцию $f(x) \in L_2(\mathbb{R})$ можно с любой наперёд заданной точностью представить элементом наилучшего приближения $f_m(x) \in V_m$.

$$\{q\} \xleftarrow[\text{точности приближения}]{\text{Понижение}} \dots \subset V_m \subset V_{m+1} \subset \dots \xrightarrow[\text{точности приближения}]{\text{Повышение}} L_2(\mathbb{R})$$

Лемма. Полученная по масштабирующей функции $f(x) \in V_0$ система функций $\{f_{m,n}(x) = 2^{m/2} f(2^m x - n)\}_{n \in \mathbb{Z}}$ образует ортонормированный базис в подпространстве V_m .

Функцию $f(x) = f_{0,0}(x) \in V_0$ можно разложить по базису $\{f_{1,n}(x)\} \in V_1$:

$$f(x) = \sqrt{2} \sum_{n \in \mathbb{Z}} h_n f(2x - n), \quad (29)$$

где $h_n = \langle f, f_{1,n} \rangle$. Уравнение (29) называется *масштабирующим* и может быть записано в более общем виде следующим образом:

$$f_{m,0}(x) = \sum_{n \in \mathbb{Z}} h_n f_{m+1,n}(x),$$

причём для коэффициентов Фурье h_n верна та же формула, что и в уравнении (29):

$$h_n = \langle f_{m,0}, f_{m+1,n} \rangle = 2^{\frac{m+1}{2}} \int_{-\infty}^{\infty} f(2^m x) f(2^{m+1} x - n) dx = \sqrt{2} \int_{-\infty}^{\infty} f(x) f(2x - n) dx = \langle f, f_{1,n} \rangle.$$

Обозначив через W_m ортогональное дополнение пространства V_m до V_{m+1} , можно записать:

$$V_m \oplus W_m = V_{m+1}, \quad V_m \perp W_m,$$

откуда на основании свойства 5 КМА следует:

$$L_2(\mathbb{R}) = \bigoplus_{j \in \mathbb{Z}} W_j. \quad (30)$$

Одно из основополагающих утверждений КМА состоит в том, что для масштабирующей функции (29) найдется такая функция $y(x) \in W_0$, называемая *материнским вейвлетом*, что множество функций

$$\{y_{m,n}(x) = 2^{m/2} y(2^m x - n)\}_{n \in \mathbb{Z}} \subset W_m$$

образует ортонормированный базис в подпространстве W_m . При этом подпространства вейвлетов $\{W_j\}_{j \in \mathbb{Z}}$ наследуют масштабирующее свойство, аналогичное свойству 2, а именно,

$$\forall m \in \mathbb{Z}: f(x) \in W_m \Leftrightarrow f(2x) \in W_{m+1}.$$

Представив материнский вейвлет в базисе пространства V_1 , получаем *масштабирующее уравнение для вейвлетов*:

$$y(x) = \sqrt{2} \sum_{n \in \mathbb{Z}} g_n f(2x - n), \quad (31)$$

и в более общем виде:

$$y_{m,0}(x) = \sum_{n \in \mathbb{Z}} g_n f_{m+1,n}(x),$$

где $g_n = \langle y, f_{1,n} \rangle$.

Особый интерес для практического применения в цифровой обработке сигналов представляют случаи, когда КМА строится на основе масштабирующей функции с ограниченным носителем. В этом случае в формулах (29) и (31) суммы будут также

конечными, что даёт возможность обрабатывать их с помощью вычислительной техники.

Дискретное вейвлет-преобразование (ДВП)

С любой наперед заданной точностью произвольную функцию $f(x) \in L_2(R)$ можно представить ее проекцией на некоторое подпространство $V_M \subset L_2(R)$. Т.е. $\forall \epsilon > 0, \forall f \in L_2(R) \exists V_M \subset L_2(R): \|f(x) - f_M(x)\| < \epsilon$. Поэтому на практике вместо исходного объекта анализа – функции $f(x) \in L_2(R)$ – можно рассматривать ее проекцию $f_M(x) \in V_M$ на некоторое подпространство $V_M \subset L_2(R)$.

Разложим функцию $f_M(x) \in V_M$ по базису функций $f_{M,n}(x)$ из того же подпространства V_M :

$$f_M(x) = \sum_{n=-\infty}^{\infty} a_{M,n} f_{M,n}(x), \quad (32)$$

где

$$a_{M,n} = \langle f_M, f_{M,n} \rangle = \int_{-\infty}^{\infty} f_M(x) f_{M,n}(x) dx.$$

Можно показать, что при использовании масштабирующих функций с ограниченным носителем набор коэффициентов $\{a_{M,n}\}$ можно найти с помощью обычной равномерной дискретизации непрерывного сигнала $f(x)$: $a_{M,n} \approx f(x_{M,n}) \cdot 2^{-M/2}$, $x_{M,n+1} = x_{M,n} + 2^{-M}$.

Поскольку $V_{m-1} \oplus W_{m-1} = V_m$, то любую функцию $f_M(x) \in V_M$ можно представить в виде суммы двух функций: $f_M = f_{M-1} + y_{M-1}$, $f_{M-1} \in V_{M-1}$, $y_{M-1} \in W_{M-1}$. В этой формуле f_{M-1} представляет собой более «грубый» вариант функции f_M , подвергнутой низкочастотной фильтрации, а y_{M-1} – высокочастотные составляющие, т.е. «детали», отброшенные при фильтрации.

Процесс дискретного вейвлет-преобразования заключается в рекуррентном разложении функции

$$f_M = f_{M-1} + y_{M-1} = (f_{M-2} + y_{M-2}) + y_{M-1} = \dots = \sum_{m=-\infty}^{M-1} y_m = \sum_{m=-\infty}^{M-1} \sum_{n=-\infty}^{\infty} c_{m,n} y_{m,n}(x),$$

где $c_{m,n} = \langle f, y_{m,n} \rangle$ – коэффициенты Фурье разложения функции y_m по базису вейвлетов в пространстве W_m .

Для вычисления вейвлет-преобразования необходимо найти коэффициенты

$$c_{m,n} = \langle f, y_{m,n} \rangle = \int_{-\infty}^{\infty} f(x) y_{m,n}(x) dx,$$

что является весьма трудоёмкой задачей. Чтобы не решать её напрямую, необходимо сначала заменить функцию f на некоторый элемент наилучшего приближения $f_M(x) \in V_M$.

В [1] описан процесс вывода формул, позволяющих вычислять очередной коэффициент $c_{m,n}$ с помощью наборов коэффициентов $\{g_n\}$, $\{h_n\}$ и $\{a_{m,n}\}$:

$$a_{m-1,k} = \sum_{n=-\infty}^{\infty} a_{m,n} h_{n-2k}, \quad c_{m-1,k} = \sum_{n=-\infty}^{\infty} a_{m,n} g_{n-2k}. \quad (33)$$

Формулы (33) задают вычислительную процедуру, которая рекуррентно применяется к последовательностям $\{a_{m,n}\}_{n \in \mathbb{Z}}$ для $m = M, M-1, M-2, \dots$, и определяет дискретное вейвлет-преобразование функции $f(x)$, заданной своим элементом наилучшего приближения $f_M(x) \in V_M$. Схематически эту процедуру можно представить следующим образом:

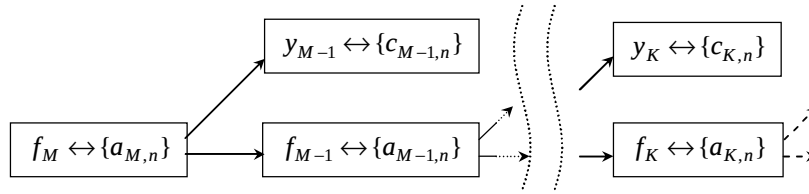


Рис. 20. Схема вычисления ДВП

На практике число шагов алгоритма вейвлет-преобразования должно быть конечно. Для некоторого числа $m = K$ вычисления прекращаются, то есть мы ограничиваемся представлением:

$$\begin{aligned} f_M(x) &= \sum_{n=-\infty}^{\infty} a_{K,n} f_{K,n}(x) + \sum_{m=K}^{M-1} \sum_{n=-\infty}^{\infty} c_{m,n} y_{m,n}(x) = \\ &= f_K(x) + y_K(x) + y_{K+1}(x) + \dots + y_{M-1}(x). \end{aligned} \quad (34)$$

Обратное вейвлет-преобразование вычисляется по следующей формуле (см. [1]):

$$\forall k: a_{m,k} = \sum_{n \in \mathbb{Z}} a_{m-1,n} h_{k-2n} + \sum_{n \in \mathbb{Z}} c_{m-1,n} g_{k-2n}, \quad m = K+1, K+2, \dots, M. \quad (35)$$

Этой формуле соответствует схема, представленная на рис. 21.

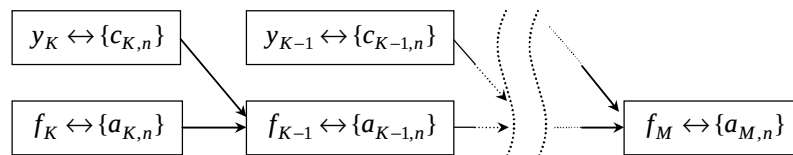


Рис. 21. Схема вычисления обратного ДВП

Квадратурно-зеркальные фильтры

Один шаг алгоритма вычисления коэффициентов вейвлет-разложения можно представить как параллельную обработку фильтрами с импульсными характеристиками $\{\tilde{h}(n)=h_{-n}\}$, $\{\tilde{g}(n)=g_{-n}\}$ и последующее прореживание выходных последовательностей, см. рис. 22.

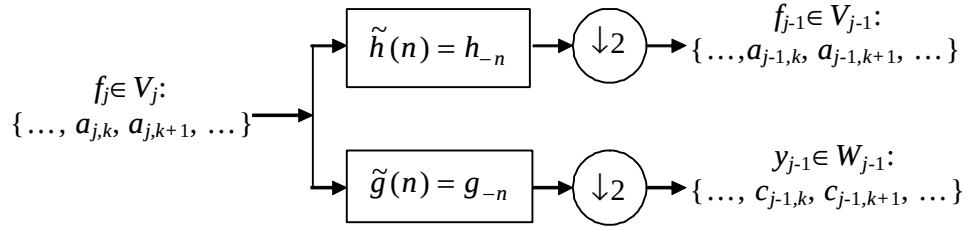


Рис. 22. Реализация одного шага вейвлет-преобразования с помощью системы КЗФ

Операцию вставки нулевого элемента на каждую вторую позицию в последовательности обозначим $\uparrow 2$. Тогда вычислениям по формуле (35) соответствует схема обработки, которая реализуется при помощи пары фильтров с импульсными характеристиками $h(n)$ и $g(n)$, см. рис. 23.

Для импульсных характеристик фильтров верны равенства: $\tilde{g}(n)=g(-n)$, $\tilde{h}(n)=h(-n)$, то есть их импульсные характеристики являются *зеркальными*. Все вместе эти четыре фильтра называют *квадратурно-зеркальными фильтрами* (КЗФ). Описать данные фильтры можно не только при помощи ИХ, но и задав передаточные функции $\tilde{H}(z)$, $\tilde{G}(z)$, $H(z)$, $G(z)$ или частотные характеристики $K_{\tilde{H}}(w)$, $K_{\tilde{G}}(w)$, $K_H(w)$, $K_G(w)$.

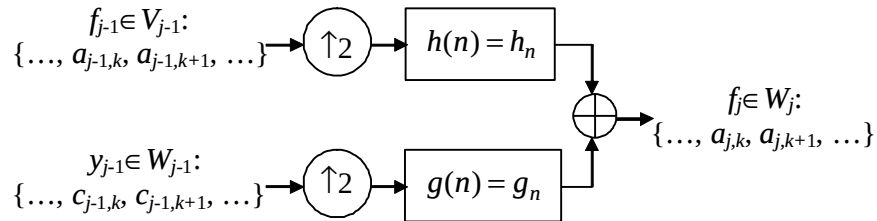


Рис. 23. Реализация одного шага обратного вейвлет-преобразования с помощью системы КЗФ

Для того чтобы сигнал мог быть точно восстановлен после фильтрации с использованием КЗФ, должно выполняться следующее соотношение:

$$g_n = a h_{1-n-2k} (-1)^{1-n}, \quad (36)$$

где $k \in \mathbb{Z}$ – произвольное число, множитель $a \in \{-1, 1\}$ выбирается одним и тем же для всех значений индекса $n \in \mathbb{Z}$. При этом для ЧХ фильтров имеем

$$K_G(w) = a e^{i(2k-1)w} \overline{K_H(w+p)}, \quad (37)$$

а для передаточных функций

$$G(z) = a z^{2k-1} H(-z^{-1}). \quad (38)$$

Построение масштабирующих функций и вейвлетов по масштабирующим уравнениям

Набор коэффициентов $\{h_n\}$ уравнения (29) позволяет задать масштабирующую функцию $f(x)$. Рассмотрим два рекуррентных численных метода, позволяющих найти функцию $f(x)$ по заданным коэффициентам уравнений (29) и (31). Поскольку коэффициенты масштабирующего уравнения для вейвлетов (31) выражаются через $\{h_n\}$ при помощи выражения (36), то задания набора $\{h_n\}$ достаточно и для построения функции вейвлета.

Как упоминалось выше, особый интерес для практики представляют КМА, порождаемые масштабирующей функцией с ограниченным носителем. Положим, что носитель $\Delta_{0,0} = \text{supp } f(t)$ содержится в некотором отрезке $[0; M-1] \supset \Delta_{0,0}$, где целое число $M \geq 2$, причем на полуинтервале $t \in [0; 1)$ масштабирующая функция отлична от тождественного нуля – этого всегда можно добиться, выполнив при необходимости целочисленный сдвиг аргумента масштабирующей функции. Тогда при $k \notin [0; M-1]$ носители $\Delta_{1,k} = \text{supp } f(2t-k) \subset [k/2; (M-1+k)/2]$ таковы, что $\Delta_{1,k} \not\subset [0; M-1]$ (возможны только отдельные общие точки: $t = 0$ или $t = M-1$) и, следовательно, $\Delta_{1,k} \not\subset \Delta_{0,0}$. Поэтому уравнение (29) можно записать в виде конечной суммы:

$$f(x) = \sqrt{2} \sum_{n=0}^{M-1} h_n f(2x-n). \quad (39)$$

Возьмем в качестве M четное число: $M = 2L$. Тогда, обозначив $2k = 2 - M$, на основании (36) для уравнения (31) получаем:

$$y(x) = \sqrt{2} \sum_{n=0}^{M-1} g_n f(2x-n) = \sqrt{2} \sum_{n=0}^{M-1} (-1)^n h_{M-1-n} f(2x-n). \quad (40)$$

Заметим, что запись (40) означает также, что $\text{supp } y(t) \subset [0; M]$.

Перейдем к описанию методов нахождения функции $f(x)$.

Первый метод

Для отсчетов функции $f(x_n)$, взятых с шагом $h = 2^{-N}$, можно записать [1]:

$$f(x_n) = 2^{N/2} a_{N,n} + O(2^{-N}). \quad (41)$$

Отсюда видно, что, имея набор коэффициентов $\{a_{N,n}\}_{n \in \mathbb{Z}}$, который определяет проекцию $f_N = A_N(f) \in V_N$, можно с погрешностью порядка $O(h)$ найти значения

функции $f(x)$ в точках x_n . Т.е. увеличивая N , можно как угодно точно описать функцию $f(x)$: погрешность определения значения функции в отсчетах $f(x_n)$ по формуле (41) будет уменьшаться, а сами отсчеты будут располагаться на числовой оси все более часто.

С учетом (41) процедура нахождения масштабирующей функции $f(x)$ при известном наборе коэффициентов $\{h_n\}$ состоит в следующем.

Сначала заметим, что $f(x) \in V_0 \subset V_1 \subset \dots$ и $f(x) \perp W_0, f(x) \perp W_1, \dots$. Так как $f(x) = f_{0,0}(x)$ – базисная функция пространства V_0 , то в разложении (32) функции $f(x) = f_0(x) \in V_0$ получим $a_{0,0} = 1$ и $\forall k \neq 0: a_{0,k} = 0$. При этом в представлении для V_1 : $f(x) = f_0(x) + y_0(x)$, где $f_0(x) \in V_0, y_0(x) \in W_0$, получим $y_0(x) = q$ и $\forall k: c_{0,k} = 0$. Поэтому с использованием формулы (35) можно найти коэффициенты $\{a_{1,k}\}_{k \in \mathbb{Z}}$ разложения масштабирующей функции $f(x) \in V_0$ по базису пространства V_1 .

Рекуррентно применяя полученную из (35) формулу

$$a_{m,k} = \sum_{n \in \mathbb{Z}} a_{m-1,n} h_{k-2n}$$

для $m = 2, 3, \dots, N$, можно получить как угодно точное описание масштабирующей функции $f(x)$ через ее отсчеты в точках $x_n \in \Delta_{N,n}$. Для этого необходимо воспользоваться выражением (41), положив $f(x) = f(x)$.

Второй метод

Запишем масштабирующее уравнение (39) для целых значений аргумента $j = 0, \dots, M-1$:

$$f(j) = \sqrt{2} \sum_{n=0}^{M-1} h_n f(2j-n).$$

Или в матричном виде:

$$\begin{pmatrix} f(0) \\ f(1) \\ \mathbf{L} \\ f(M-1) \end{pmatrix} = \mathbf{B} \begin{pmatrix} f(0) \\ f(1) \\ \mathbf{L} \\ f(M-1) \end{pmatrix}, \quad (42)$$

где

$$\mathbf{B} = \sqrt{2} \begin{pmatrix} h_0 & 0 & 0 & 0 & 0 & \dots & 0 & 0 & 0 & 0 & 0 \\ h_2 & h_1 & h_0 & 0 & 0 & \dots & 0 & 0 & 0 & 0 & 0 \\ h_4 & h_3 & h_2 & h_1 & h_0 & \dots & 0 & 0 & 0 & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ h_{M-4} & h_{M-5} & h_{M-6} & h_{M-7} & h_{M-8} & \dots & h_1 & h_0 & 0 & 0 & 0 \\ h_{M-2} & h_{M-3} & h_{M-4} & h_{M-5} & h_{M-6} & \dots & h_3 & h_2 & h_1 & h_0 & 0 \\ 0 & h_{M-1} & h_{M-2} & h_{M-3} & h_{M-4} & \dots & h_5 & h_4 & h_3 & h_2 & h_1 \\ 0 & 0 & 0 & h_{M-1} & h_{M-2} & \dots & h_7 & h_6 & h_5 & h_4 & h_3 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & 0 & 0 & \dots & h_{M-1} & h_{M-2} & h_{M-3} & h_{M-4} & h_{M-5} \\ 0 & 0 & 0 & 0 & 0 & \dots & 0 & 0 & h_{M-1} & h_{M-2} & h_{M-3} \\ 0 & 0 & 0 & 0 & 0 & \dots & 0 & 0 & 0 & 0 & h_{M-1} \end{pmatrix}$$

Лемма. Матрица $\mathbf{B}$ уравнения (42) имеет собственное число $l = 1$.

Данная лемма и уравнения (39), (42) определяют следующий способ рекуррентного построения масштабирующей функции $f(x)$ по набору коэффициентов $\{h_n\}_{n=0}^{M-1}$.

Сначала находим собственный вектор $\Phi_0 = (f(0), f(1), \dots, f(M-1))^T$ матрицы $\mathbf{B}$, который соответствует собственному числу $l = 1$ и определяет значения масштабирующей функции для целочисленных значений аргумента. Так как собственный вектор Φ_0 матрицы $\mathbf{B}$ определяется с точностью до масштабного множителя c ($c\Phi_0$ также будет собственным вектором матрицы $\mathbf{B}$), то для выбора $\Phi_0 = (f(0), f(1), \dots, f(M-1))^T$ из собственных векторов матрицы $\mathbf{B}$, соответствующих собственному числу $l = 1$, введем дополнительное требование, которое мы сформулируем в виде следующей леммы.

Лемма. Масштабирующая функция $f(x)$ должна удовлетворять условию

$$\sum_{n \in \mathbb{Z}} f(x-n) = 1. \quad (43)$$

Таким образом, искомый собственный вектор $\Phi_0 = (f(0), f(1), \dots, f(M-1))^T$ матрицы $\mathbf{B}$ из уравнения (42) должен удовлетворять условию $\sum_{n=0}^{M-1} f(n) = 1$. После определения собственного вектора $\Phi_0 = (f(0), f(1), \dots, f(M-1))^T$, отвечающего данному требованию, далее последовательно находим значения масштабирующей функции в точках $t_j = j2^{-m}$, $j \in \mathbb{Z}$, рекуррентно используя уравнение (39) для $m = 1, 2, \dots$:

$$f(j2^{-m}) = \sqrt{2} \sum_{n=0}^{M-1} h_n f(j2^{-m+1} - n). \quad (44)$$

В результате получаем значения масштабирующей функции на множестве двоично-рациональных значений аргумента. Реально вычисления по формуле (44) прекращаются при достижении достаточно большого значения $m = K$, когда для масштабирующей функции известен вектор отсчетов $\Phi_K = (f(0), f(2^{-K}), \dots, f(M - 2^{-K}))^T$, заданных на сетке значений аргумента t с шагом $\Delta t = 2^{-K} : t_j = j \Delta t, j = 0, 1, \dots, M 2^k - 1$.

По аналогии с описанными процедурами итерационного построения масштабирующей функции строятся процедуры нахождения материнского вейвлета $y(x)$ по заданному набору коэффициентов $\{h_n\}$ масштабирующих уравнений (39) и (40).

Сжатие изображений с использованием ДВП

Основная идея большинства алгоритмов вейвлет-компрессии состоит в следующем. Для исходного дискретного сигнала f_M , заданного конечным набором (т.е. вектором) коэффициентов $\{a_{M,n}\}$, сначала выполняется дискретное вейвлет-преобразование. Количество отсчетов в сглаженной проекции $f_K \leftrightarrow \{a_{K,n}\}$ будет существенно меньшим, чем в исходном векторе $\{a_{M,n}\}$, поскольку каждый шаг, соответствующий снижению уровня разрешения проекции f_j , $j = M - 1, M - 2, \dots, K$, соответствует двукратному сокращению числа компонент вектора $\{a_{j,n}\}$. Коэффициенты сглаженной проекции $\{a_{K,n}\}$ далее квантуются и статистически кодируются, обычно независимо друг от друга, поскольку представляют собой практически не коррелированные данные.

Вейвлет-коэффициенты $\{c_{K,n}\}, \{c_{K+1,n}\}, \dots, \{c_{M-2,n}\}, \{c_{M-1,n}\}$ обрабатываются по схеме, которая основана на специфических свойствах локализации функциональных вейвлет-базисов. Базисы вейвлет-функций $\{y_{m,n}(t)\}$ и соответствующие им коэффициенты $\{c_{m,n}(t)\}$ разложения обрабатываемого сигнала удобно упорядочить в виде древовидной структуры, как показано на рис. 24.

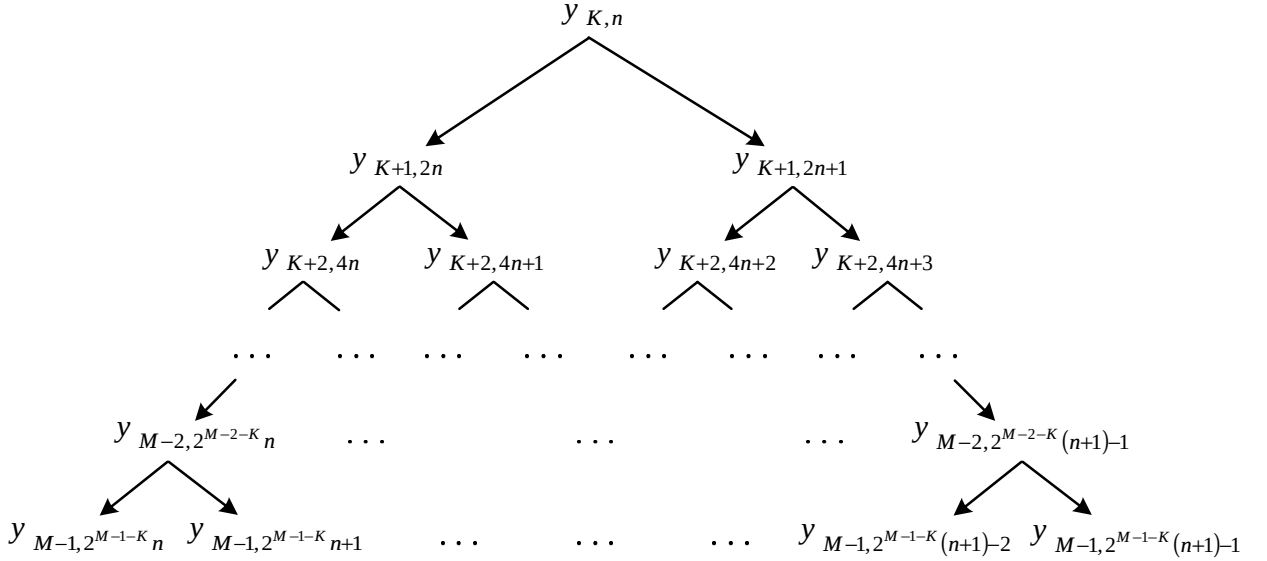


Рис. 24. Схема упорядочивания вейвлет-коэффициентов в виде древовидной структуры

При таком упорядочивании каждый узел $y_{j,m}$ дерева (за исключением листьев) имеет по два непосредственных (ближайших) потомка $\{y_{j+1,2m}, y_{j+1,2m+1}\}$, причем носители $\Delta_{j+1,2m} = \text{supp } \psi_{j+1,2m}(t)$ и $\Delta_{j+1,2m+1} = \text{supp } \psi_{j+1,2m+1}(t)$ соответствующих им базисных функций представляют собой области, лежащие внутри области носителя $\Delta_{j,m} = \text{supp } \psi_{j,m}(t)$ базисной функции-родителя (см. рис. 25): $\Delta_{j+1,2m} \subset \Delta_{j,m}$, $\Delta_{j+1,2m+1} \subset \Delta_{j,m}$. Действительно, пусть $\Delta_{0,0} = \text{supp } \psi(t) = [0; A]$. Заметим, что $A \geq 1$, т.к. в противном случае целочисленные сдвиги $\{\psi(t-n)\}_{n \in \mathbb{Z}}$ не покрывают полностью числовую ось $t \in \mathbb{R}$. Тогда $\Delta_{j,m} = \left[\frac{m}{2^j}; \frac{m}{2^j} + \frac{A}{2^j} \right]$,

$$\Delta_{j+1,2m} = \left[\frac{2m}{2^{j+1}}; \frac{2m+A}{2^{j+1}} \right] = \left[\frac{m}{2^j}; \frac{m}{2^j} + \frac{A}{2^{j+1}} \right] \subset \Delta_{j,m},$$

$$\Delta_{j+1,2m+1} = \left[\frac{2m+1}{2^{j+1}}; \frac{2m+1+A}{2^{j+1}} \right] = \left[\frac{m}{2^j} + \frac{1}{2^{j+1}}; \frac{m}{2^j} + \frac{A+1}{2^{j+1}} \right] \subset \Delta_{j,m},$$

т.к. для $A \geq 1$ справедливо: $\frac{A+1}{2^{j+1}} \leq \frac{A}{2^j}$.

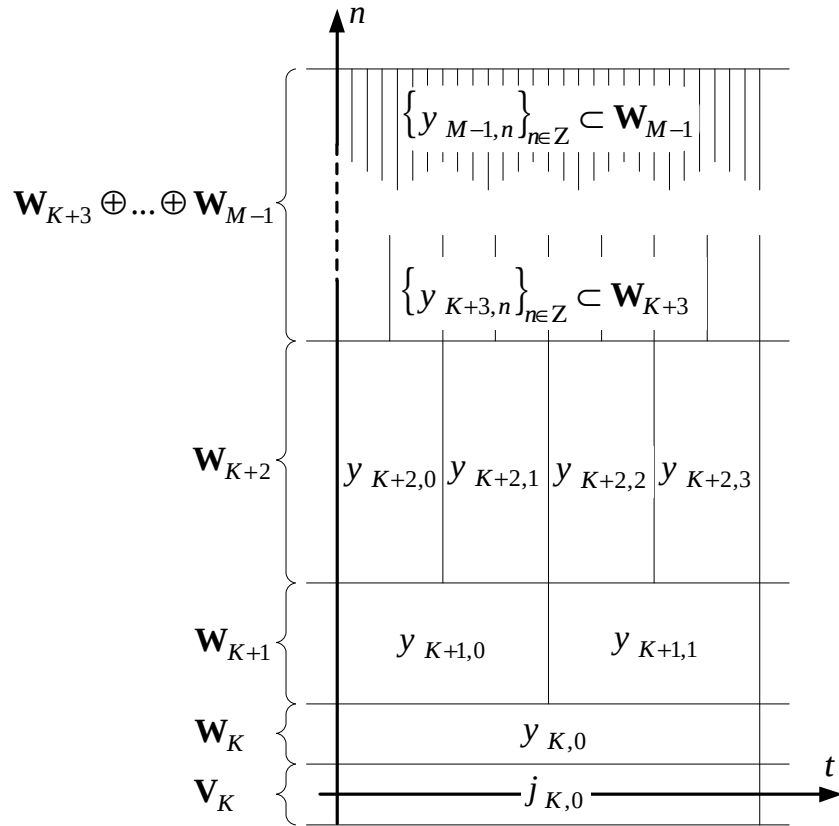


Рис. 25. Время-частотная локализация базисных функций, используемых для представления сигнала в виде (34)

Основное предположение, лежащее в основе многих алгоритмов вейвлет-кодирования, состоит в том, что если некоторый коэффициент $y_{j,m} = \langle f(t), y_{j,m}(t) \rangle$ мал по абсолютной величине, то в области аргумента $t \in \Delta_{j,m} = \text{supp } \psi_{j,m}(t)$ сигнал $f(t)$, скорее всего, обладает малой энергией (и несет малую информацию), и все вейвлет-коэффициенты, которые являются прямыми (не только непосредственными) потомками узла $y_{j,m}$, также малы по абсолютной величине. Тогда «ветвь дерева» $T_{j,m}$, выходящую из узла $y_{j,m}$ и включающую в себя всех его прямых потомков, можно «подрезать», не кодируя входящие в нее вейвлет-коэффициенты, а при декодировании заменить отброшенные коэффициенты нулями.

Если используемое вейвлет-преобразование является ортогональным, то евклидова норма ошибки, вносимая в дискретный сигнал $f_M \leftrightarrow \{\tilde{a}_{M,n}\}$ при подрезании ветви $T_{j,m}$, может быть подсчитана в области преобразования:

$$\|f_M - \tilde{f}_M\|^2 = \sum_n |a_{M,n} - \tilde{a}_{M,n}|^2 = \sum_{y_{k,l} \in T_{j,m}} y_{k,l}^2,$$

где $\tilde{f}_M \leftrightarrow \{\tilde{a}_{M,n}\}$ – восстановленный после декодирования и обратного вейвлет-преобразования дискретный сигнал. Различные алгоритмы вейвлет-компрессии

различаются, в основном, правилами, по которым принимается решение о подрезании ветвей, и способами кодирования структуры подрезанного дерева, а также процедурами квантования и статистического кодирования тех вейвлет-коэффициентов, которые остались после подрезания ветвей.

Для обработки двумерных дискретных сигналов, которыми описываются цифровые изображения, используется следующая схема, обобщающая одномерный случай. При помощи одномерной масштабирующей функции $f(x)$ и соответствующего ей материнского вейвлета $y(x)$ определяются двумерная масштабирующая функция $f(x, y) = f(x)f(y)$ и три двумерных вейвлета:

$$y^H(x, y) = f(x)y(y), y^V(x, y) = y(x)f(y), y^D(x, y) = y(y)y(x).$$

Соответственно, вместо двух фильтров декомпозиции получаем четыре. На практике это означает, что сначала двумерная матрица дискретного изображения построчно подвергается одномерной фильтрации (отклик $\tilde{h}$ -фильтра записывается в левую половину вектора-строки, отклик $\tilde{g}$ -фильтра – в правую), а затем одномерная фильтрация проводится вдоль столбцов (отклик $\tilde{h}$ -фильтра записывается в верхнюю половину вектора-столбца, отклик $\tilde{g}$ -фильтра – в нижнюю). В результате получаем из матрицы изображения 4 группы коэффициентов разложения, см. рис. 26. Восстановить исходное изображение после такой обработки можно, последовательно обработав столбцы и строки при помощи пары фильтров реконструкции.

Процедура декомпозиции может быть продолжена над левой верхней четвертью полученных коэффициентов, которые соответствуют двумерным масштабирующим функциям (см. рис. 27). Коэффициенты, лежащие в оставшихся трех наборах, относятся к функциям двумерных вейвлетов и далее не обрабатываются. Соответствующие блоки коэффициентов называем V, H, D (vertical, horizontal, diagonal) или HL, LH, HH (high-low, low-high, high-high) *поддиапазонами (суббэндами, subband)*. Три вейвлет-поддиапазона, которые могут быть получены на следующем шаге, представляют «более грубый» уровень пространственного (т.е. в области изображения) разрешения.

Аналогично одномерному случаю, базисные функции двумерного вейвлет-спектра также обладают важным свойством, широко используемым в современных алгоритмах компрессии изображений. А именно, базисные функции более высокого уровня разрешения получаются из базисных функций более низкого уровня разрешения путем сжатия и сдвига, причем «потомки» имеют носитель, не выходящий за область носителя родителя, а у каждого родителя имеется по четыре потомка. Так, если $y(x, y)$ – «вейвлет-родитель», то порождаются следующие функции-потомки:

$$y(x, y) \rightarrow \left\{ \begin{array}{l} 2y(2x, 2y), 2y(2x-1, 2y), \\ 2y(2x, 2y-1), 2y(2x-1, 2y-1) \end{array} \right\} \quad (45)$$

Схема связей «родитель-потомки» отражена на рис. 28. Коэффициенты спектра могут быть упорядочены в древовидную структуру, как это показано на рис. 29. Родителем трех вейвлетов самого «грубого» разрешения по определению считается единственная масштабирующая функция (начало дерева), дальнейшие связи устанавливаются в соответствии с (45).

Таким образом, в двумерном случае коэффициенты вейвлет-преобразования также могут быть упорядочены в древовидную структуру, которая наследует свойства одномерной структуры. Соответственно, и основные идеи алгоритмов компрессии двумерных сигналов остаются теми же, что и в одномерном случае.

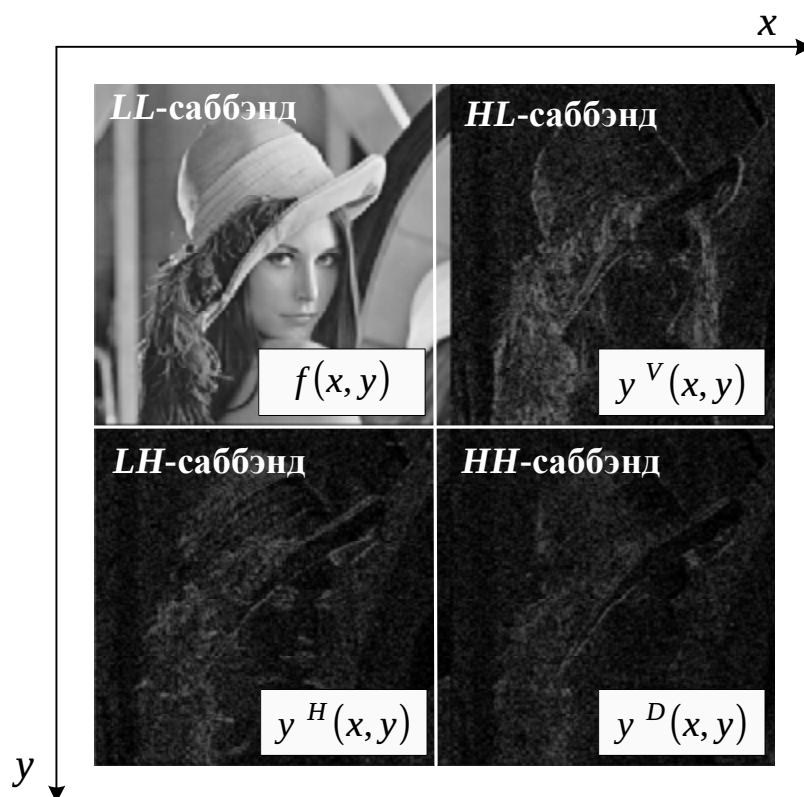


Рис. 26. Результат выполнения первого шага двумерной декомпозиции

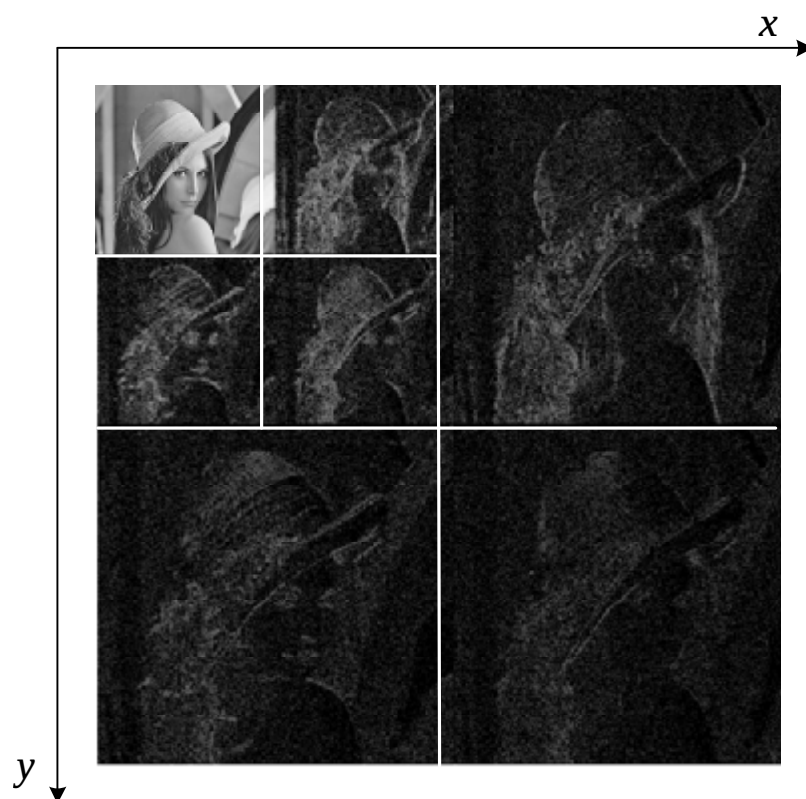


Рис. 27. Результат выполнения двух шагов двумерной декомпозиции

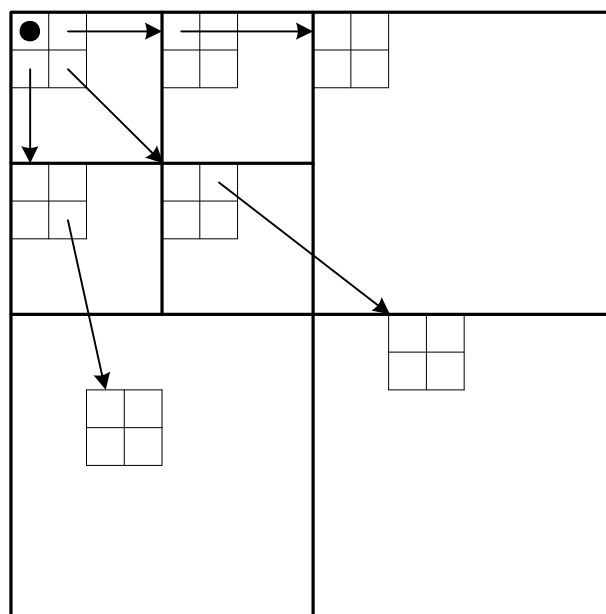


Рис. 28. Структура связей «родитель-потомки» между базисными функциями вейвлет-спектра. Выполнено три шага преобразования

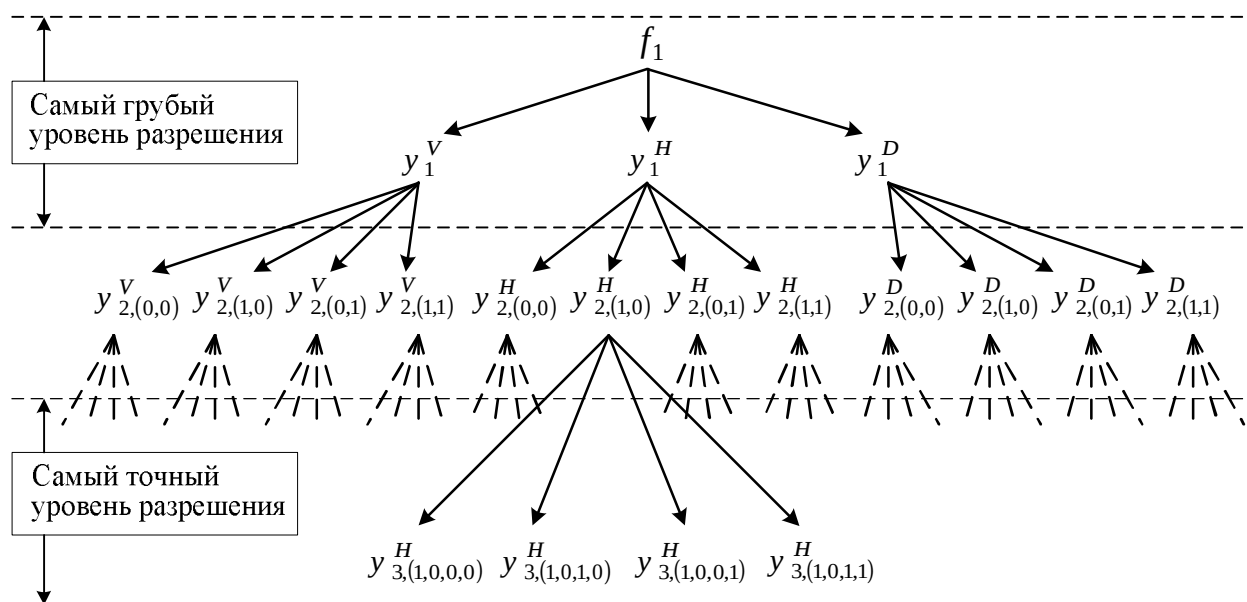


Рис. 29. Способ упорядочивания базисных функций дискретного вейвлет-преобразования в древовидную структуру

Контрольные вопросы

1. Что такое дискретное вейвлет-преобразование сигнала?
2. Как можно задать квадратурно-зеркальные фильтры?
3. Расскажите о методах построения масштабирующей функции $f(x)$.
4. В чем заключается основная идея сжатия сигналов с использованием вейвлетов?
5. Какие свойства ДВП позволяют использовать его для сжатия изображений?

Задание

1. Изучить возможности MATLAB по работе с вейвлетами (пакет Wavelet Toolbox). Изучить инструмент wavemenu (Wavelet 1-D, Wavelet Display) «вручную» и с помощью презентации wavedemo.
2. С помощью предложенных в п.1 средств проанализировать процесс разложения сигналов по базисам вейвлетов Хаара, «Добеши-2», «Добеши-8», убедиться в зеркальности импульсных характеристик соответствующих им фильтров.
3. Проанализировать квадратурно-зеркальные фильтры, соответствующие одному из вейвлетов Добеши. К какому классу фильтров (ВЧ, НЧ, полосовые) они относятся?
4. По наборам коэффициентов $\{g_n\}$ и $\{h_n\}$, полученных с помощью функции wfilters, построить:

– *вариант 1:* для вейвлетов Хаара, «Добеши-2», «Добеши-8» масштабирующую функцию 1-м способом и материнский вейвлет 2-м способом;

– *вариант 2:* для вейвлетов Хаара, «Добеши-4», «Добеши-10» масштабирующую функцию 2-м способом и материнский вейвлет 1-м способом.

5. Ознакомиться с методом сжатия изображений на основе ДВП (wavemenu → Wavelet 2D → Compress).

Литература

1. Умняшкин С.В. Теоретические основы цифровой обработки и представления сигналов.
2. И. Добеши. Десять лекций по вейвлетам. – Ижевск: РХД, 2004.
3. К. Блаттер. Вейвлет-анализ. Основы теории. – М.: Техносфера, 2004.
4. В.П. Дьяконов. Вейвлеты. От теории к практике. Изд. 2-е. – М.: СОЛОН-Пресс, 2004.

